



UNIVERSITE DE LUBUMBASHI
FACULTE POLYTECHNIQUE

Département d'électromécanique



CONTROLE INTERACTIF ET DYNAMIQUE D'UN CIRCUIT ELECTRIQUE PAR LE LANGAGE NATUREL GRACE AUX GRANDS MODELES DE LANGAGE



Par **BALEMBA MWAMI Salomon**

Travail présenté et défendu en vue de
l'obtention du grade d'ingénieur civil
Bachelier en électromécanique

OCTOBRE 2024



UNIVERSITE DE LUBUMBASHI
FACULTE POLYTECHNIQUE
Département d'électromécanique



CONTROLE INTERACTIF ET DYNAMIQUE D'UN CIRCUIT ELECTRIQUE PAR LE LANGAGE NATUREL GRACE AUX GRANDS MODELES DE LANGAGE



Par **BALEMBA MWAMI Salomon**

Directeur : Dr. Ir. Gauthier NGANDU KALALA

Co-directeur: Msc. Ir. KASIHO MUSHAGALUSA Emmanuel

Travail présenté et défendu en vue de
l'obtention du grade d'ingénieur civil
Bachelier en électromécanique

2023-2024

RESUME

Ce Travail explore l'utilisation des grands modèles de langage (LLM) pour contrôler interactivement et dynamiquement des circuits électriques par le biais du langage naturel. Les interfaces traditionnelles de contrôle électrique, basées sur des langages de programmation spécialisés ou des interfaces graphiques complexes, peuvent s'avérer difficiles d'accès pour les opérateurs non-spécialistes. L'objectif principal de ce travail est de démontrer la faisabilité d'un système de contrôle intuitif et convivial, exploitant la capacité des LLM à traduire des instructions en langage naturel en commandes exécutables par un microcontrôleur. L'hypothèse fondamentale est que les LLM peuvent comprendre les nuances du langage naturel relatives au contrôle électrique, générer du code Python fonctionnel et s'adapter aux variations des configurations de circuits. Pour valider cette hypothèse, un système de contrôle a été développé autour d'un Arduino UNO R3, pilotant des LEDs et un servomoteur. Une interface interactive permet à l'utilisateur de formuler des instructions en langage naturel, qui sont ensuite traitées par un LLM pour générer du code Python. Ce code est exécuté pour envoyer des commandes à l'Arduino via la communication série, gérée par la bibliothèque PySerial. L'Arduino exécute les commandes et renvoie un feedback au LLM, confirmant l'exécution ou signalant des erreurs. Différents LLM, incluant GPT-4o, GPT-4o mini, Mistral Large, Mistral 8X7B, Mistral 7B, Mistral Nemo, Codestral et Gemma 2 9B, ont été testés avec un jeu de données d'instructions en langage naturel. Les résultats expérimentaux démontrent la faisabilité du contrôle interactif de circuits électriques par le langage naturel via les LLM. Des taux de succès élevés, atteignant 100% pour GPT-4o mini, ont été obtenus. GPT-4o s'est distingué par sa précision temporelle avec une MAE de 4.5 secondes, tandis que GPT-4o mini offre le meilleur compromis performance-coût. L'analyse du code généré a révélé une capacité des LLM à traduire des instructions en langage naturel en structures de code Python appropriées. Cependant, des limitations persistent dans la gestion des nuances temporelles complexes, l'interprétation des boucles infinies et la synchronisation précise des actions pour certains modèles. Ce travail confirme le potentiel des LLM pour le contrôle des circuits électriques par le langage naturel, ouvrant la voie à une nouvelle ère d'interaction homme-machine. L'adaptabilité du système à différentes configurations de circuits a été démontrée, mais des améliorations sont nécessaires pour une adaptation automatique. La recherche future se concentrera sur le développement de LLM spécialisés pour le contrôle électrique, l'intégration de la multimodalité, l'amélioration du raisonnement des modèles, et l'apprentissage continu pour une collaboration homme-machine plus performante.

Mots-clés: Contrôle interactif, Circuits électriques, langage naturel, grands modèles de langage (LLM), Systèmes embarqués, interaction homme-machine, Génération de code.

ABSTRACT

This work explores the use of large language models (LLMs) for interactively and dynamically controlling electrical circuits through natural language. Traditional electrical control interfaces, based on specialized programming languages or complex graphical user interfaces, can be challenging for non-specialist operators. The primary objective of this work is to demonstrate the feasibility of an intuitive and user-friendly control system, leveraging the capability of LLMs to translate natural language instructions into commands executable by a microcontroller. The core hypothesis is that LLMs can understand the nuances of natural language related to electrical control, generate functional Python code, and adapt to variations in circuit configurations. To validate this hypothesis, a control system was developed around an Arduino UNO R3, controlling LEDs and a servo motor. An interactive interface allows the user to issue natural language commands, which are then processed by an LLM to generate Python code. This code is executed to send commands to the Arduino via serial communication, managed by the PySerial library. The Arduino executes the commands and sends feedback to the LLM, confirming execution or signaling errors. Various LLMs, including GPT-4o, GPT-4o mini, Mistral Large, Mistral 8X7B, Mistral 7B, Mistral Nemo, Codestral, and Gemma 2 9B, were tested with a dataset of natural language instructions. Experimental results demonstrate the feasibility of interactive control of electrical circuits via natural language through LLMs. High success rates, reaching 100% for GPT-4o mini, were achieved. GPT-4o excelled in temporal accuracy with a mean absolute error (MAE) of 4.5 seconds, while GPT-4o mini provided the best performance-cost tradeoff. The analysis of the generated code revealed that LLMs can effectively translate natural language instructions into appropriate Python code structures. However, limitations remain in managing complex temporal nuances, interpreting infinite loops, and achieving precise synchronization of actions for certain models. This work confirms the potential of LLMs for controlling electrical circuits through natural language, paving the way for a new era of human-machine interaction. The system's adaptability to different circuit configurations has been demonstrated, but further improvements are necessary for automatic adaptation. Future research will focus on developing LLMs specialized in electrical control, integrating multimodality, improving model reasoning, and continuous learning for more efficient human-machine collaboration.

Keywords: Interactive control, Electrical circuits, Natural language, Large language models (LLMs), Embedded systems, Human-machine interaction, Code generation.

EPIGRAPHE

« Je pense que la manière dont nous allons rendre les machines intelligentes est de les faire apprendre par l'expérience. »

— Geoffrey Hinton.

DEDICACE

Ce travail, à la croisée des chemins entre le langage naturel et la machine, est dédié à ceux qui, hier et aujourd'hui, nourrissent le rêve d'une super intelligence artificielle générale à la fois performante et bienveillante.

A mes parents ; BALEMBA VINCENT et MAPENDO REBECCA,
vos sacrifices silencieux, votre amour indéfectible et votre foi en mon potentiel m'ont porté plus loin que je n'aurais jamais pu imaginer. Ce chemin que j'ai parcouru, c'est grâce à vous que je l'ai entamé, avec la conviction que tout rêve peut devenir réalité. Je vous dédie ce travail, fruit de vos enseignements, de vos valeurs et de votre courage.

A mes frères et sœurs,
chaque sourire, chaque mot d'encouragement et chaque moment de partage avec vous a été une source inestimable de motivation. Votre amour m'a permis de garder la foi et de poursuivre mes rêves, sachant que je ne suis jamais seul sur ce chemin.

A ma famille,

A mes amis,

A mes professeurs

A ma faculté, la Polytechnique

A toutes celles et ceux qui rêvent de repousser les limites du possible,
que ce mémoire soit la preuve que la passion et la persévérance ouvrent des portes vers l'inconnu, vers l'infini.

REMERCIEMENT

Ce travail est le fruit de la contribution, tant morale que physique, de nombreuses personnes dont l'appui a été essentiel. Bien que toutes ne puissent être citées ici, il convient d'exprimer une sincère gratitude à chacune d'entre elles.

Les remerciements vont tout d'abord à Monsieur le Docteur Ingénieur Gauthier NGANDU KALALA, directeur de ce mémoire, qui a grandement contribué à la réalisation de ce projet.

Une reconnaissance particulière est également adressée au Master Ingénieur KASIHO MUSHAGALUSA Emmanuel, co-directeur de ce travail, pour sa disponibilité, son soutien constant, ses conseils avisés et ses remarques constructives tout au long du processus.

Il convient ensuite de remercier l'ensemble du corps académique de la Faculté Polytechnique de l'Université de Lubumbashi, et en particulier le département d'électromécanique, pour la qualité de l'enseignement dispensé et l'encadrement fourni tout au long de la formation.

Une profonde gratitude est adressée à mes parents, BALEMBA Vincent et MAPENDO Rebecca, pour leur amour inconditionnel, leurs sacrifices silencieux, et leur soutien indéfectible. Leur confiance et leurs encouragements ont été des moteurs essentiels tout au long de ce chemin.

Une gratitude particulière est adressée aux camarades de promotion et amis, qui ont partagé les moments de doute, de travail intense et de réussite collective. Parmi eux, il est important de mentionner : KANKENZA Emmanuel, Valery KYUNGU, NDAGANO Romain, Maurice MMLD, Denis EMONGO, Trésor KAUMBA, Placide SHAURI, Vainqueur ABEBWA, MASIMANGO Joseph, Benjamin MUHASE, et bien d'autres, dont l'appui a été précieux.

Enfin, une reconnaissance profonde est adressée à l'ensemble de ma famille, dont l'amour, le soutien inébranlable et les encouragements constants ont constitué une véritable source de motivation tout au long de ce parcours. Un remerciement tout particulier est dédié à Ben KABENA NKUNA et Angel NGOMBA KALALA, leur hospitalité, leur soutien moral et leur bienveillance ont été déterminants pour surmonter les défis rencontrés et m'ont permis de me concentrer pleinement sur l'accomplissement de ce travail.

Que chacun trouve ici l'expression de la plus profonde reconnaissance.

TABLE DES MATIERES

RESUME	I
ABSTRACT	II
EPIGRAPHE	III
DEDICACE	IV
REMERCIEMENT	V
TABLE DES MATIERES	VI
LISTE DES FIGURES.....	X
LISTE DES ABREVIATIONS.....	XI
LISTE DES TABLEAUX.....	XII
LISTE DES ANNEXES.....	XIII
INTRODUCTION GENREALE	1
CHAPITRE I. ÉVOLUTION HISTORIQUE DU CONTROLE DES CIRCUITS ELECTRIQUES ET DES MODELES D'APPRENTISSAGE AUTOMATIQUE	3
I.1 Introduction.....	3
I.2. Évolution historique du contrôle des circuits électriques	3
I.2.1 Les premiers dispositifs de contrôle électrique	4
I.2.2 L'émergence de l'électronique et des tubes à vide	5
I.2.3 Le développement de l'automatique et des systèmes asservis.....	7
I.2.4 L'avènement de l'informatique et des microcontrôleurs	9
I.2.5 Les applications actuelles et futures du contrôle électrique	10
I.3 Evolution historique des modèles d'apprentissage automatique	12
I.3.1 Les origines de l'intelligence artificielle et du test de Turing	12
I.3.2 Les premiers modèles d'apprentissage automatique	13

VII

I.3.3	La crise de l'IA et le renouveau de l'apprentissage automatique.....	14
I.3.4	L'essor des grands modèles de langage (LLM) et des Transformers	15
I.3.5	Les défis et les enjeux de l'apprentissage automatique	16
I.4	Conclusion	17
CHAPITRE II : État de l'art		18
II.1	Introduction.....	18
II.2	Revue de la littérature sur les LLM	18
II.2.1	Définition et concept des LLM	18
II.2.2	Introduction aux grand modèle de langage (LLM)	18
II.2.3	Architecture des LLM.....	20
II.2.4	Capacités et caractéristiques clés des LLM	22
II.2.5	Présentation des quelques LLM	24
II.2.6	Performances de quelques modeles LLM sur MMLU et HumanEval.....	29
II.3	Revue de la littérature sur le contrôle des circuits électriques.....	32
II.3.1	Définition et caractéristiques du contrôle électrique.....	32
II.3.2	Présentation des principaux outils et méthodes de contrôle électrique.....	33
II.3.3	Avantages et limites du contrôle électrique pour le langage naturel.....	35
II.3.4	Applications et tendances du contrôle électrique.....	35
II.4	Positionnement du travail par rapport aux travaux existants	36
II.4.1	Intégration de l'interaction en langage naturel pour le contrôle des circuits électriques.....	36
II.4.2	Utilisation d'un Arduino Uno pour la simulation des circuits électriques.....	36
II.4.3	Utilisation de la bibliothèque Open Interpreter pour l'interaction entre le langage naturel et les objets programmables.....	37
II.4.4	Comparaison des performances des différents LLM pour le contrôle des circuits électriques	37

VIII

II.5 Conclusion	37
Chapitre III. Implémentation d'un Système de Contrôle Électrique par Langage Naturel	39
III.1 Introduction.....	39
III.2 Présentation des outils et des données utilisées	39
III.2.1 L'Arduino Uno pour le prototypage et la simulation des circuits électriques.....	40
III.2.2 La bibliothèque Open Interpreter pour l'interaction entre le langage naturel et les objets programmables.....	41
III.2.3 Utilisation de la bibliothèque pySerial pour la communication série	42
III.2.3 Analyse Comparative des LLM pour des Applications de contrôle électrique.....	44
III.3 Hypothèses et méthodologie de Conception et d'Implémentation d'un système de Contrôle Électrique Par Langage Naturel	48
III.3.1 Hypothèses de Conception.....	49
III.3.2 Méthodologie de Conception et d'Implémentation	49
III.4 Conclusion	51
CHAPITRE IV. ANALYSE ET INTERPRETATION DES RESULTATS	52
IV.1 Introduction.....	52
IV.2 Évaluation des Hypothèses de Conception	52
IV.2.1 Hypothèse 1: Compréhension du Langage Naturel	52
IV.2.2 Hypothèse 2: Fiabilité des LLM	53
IV.2.3 Hypothèse 3: Compatibilité Technique.....	53
IV.2.4 Hypothèse 4: Adaptabilité.....	53
IV.3 Analyse Quantitative des Performances des Modèles de Langage (LLM).....	54
IV.3.1 Présentation des Métriques d'Évaluation	55
IV.3.2 Présentation du Jeu d'Instructions de Contrôle Dynamique.....	55
IV.3.3 Résultats Quantitatifs et Comparaison des Modèles	55

IV.4	Analyse Qualitative des Performances.....	56
IV.4.1	Remarques Générales.....	56
IV.4.2	Analyse des Difficultés Rencontrées par les Modèles	56
IV.4.3	Impact du Prompt Engineering	58
IV.4.4	Précision du Langage Naturel	58
IV.5	Architecture et Implémentation du Système	59
IV.5.1	Diagramme du Système	59
IV.5.2	Description Détaillée des Composants Matériels et Logiciels.....	61
IV.5.3	Description des Interfaces de Communication	63
IV.5.4	Caractéristiques des Modèles LLM	64
IV.6	Limites, Difficultés Rencontrées, et Solutions.....	64
IV.6.1	Limites Intrinsèques à l'Approche.....	64
IV.6.1.1	Complexité du Langage Naturel	64
IV.6.2	Difficultés Techniques et Solutions	65
IV.6.3	Contraintes de Ressources et de Temps	66
IV.7	Perspectives d'Avenir.....	67
IV.7.1	Modèles LLM Spécialisés pour le Contrôle de Circuits Électriques	67
IV.7.2	Intégration de la Multimodalité.....	67
IV.7.3	Contrôle de Bas Niveau par les LLM	68
IV.7.4	Amélioration du Raisonnement des Modèles LLM	68
IV.7.5	Apprentissage Continu et Collaboration	68
IV.8	Conclusion	69
	CONCLUSION GENERALE.....	70
	BIBLIOGRAPHIE	i
	ANNEXES	A

LISTE DES FIGURES

Figure I- 1 : Principe d'un relais électromécanique (« Relais électromécanique », 2024)	4
Figure I- 2 : Principe des triodes à chauffage indirect et direct (« Triode (électronique) », 2023)	6
Figure I- 3 : Principe de fonctionnement d'un transistor NPN(« Bipolar junction transistor », 2024)	7
Figure I- 4: Schéma d'un système asservi(« Asservissement (automatique) », 2024).....	8
Figure I- 5: Régulateur à boules de la machine à vapeur de Scott (musée des arts et métiers, Paris)(« Régulateur à boules », 2024).....	8
Figure I- 6: l'une des premières version du premier microprocesseur Intel_C4004(« Intel 4004 », 2024)	9
Figure I- 7: Diagramme du Test de Turing Turing (« Test de Turing », 2024).....	12
Figure I- 8: le Perceptron(« Perceptron », 2024)	13
Figure I- 9: Un perceptron multicouche(« Perceptron multicouche », 2023).....	14
Figure I- 10: Architecture générale d'un Transformeur(« Transformeur », 2024)	16
Figure II- 1: Architecture GPT de base(« Transformeur génératif pré-entraîné », 2024)	21
Figure II- 2: Capacités des LLM (Minaee et al., 2024)	23
Figure II- 3: Actionnaire de vanne électrique (File, 2017)	33
Figure II- 4: Capteur d'obstacle (File, 2017)	34
Figure II- 5: Principe de fonctionnement d'un contrôleur PID(« Régulateur PID », 2024).....	34
Figure III- 1: Performances des LLM selectionner sur les benchmarks HumanEval et MMLU .	46
Figure IV- 1: Performances des modèles Evaluer sur notre jeu d'instructions comparer à HUMANEval et MMLU	54
Figure IV- 2: Diagramme Schématique du Système	60

LISTE DES ABREVIATIONS

API: Interface de Programmation d'Application (Application Programming Interface)

BERT: Bidirectional Encoder Representations from Transformers

GPT: Transformateur Génératif Pré-entraîné (Generative Pre-trained Transformer)

IA: Intelligence Artificielle

IDE: Environnement de Développement Intégré (Integrated Development Environment)

IoT: Internet des Objets (Internet of Things)

LLaMa: Large Language Model Meta AI

LLM: Grand Modèle de Langage (Large Language Model)

MAE: Erreur Absolue Moyenne (Mean Absolute Error)

MMLU: Compréhension Massive Multitâche du Langage (Massive Multitask Language Understanding)

MoE: Mixture d'Experts (Mixture of Experts)

NLP: Traitement Automatique du Langage Naturel (Natural Language Processing)

PID: Proportionnel, Intégral, Dérivé

PWM: Modulation de Largeur d'Impulsion (Pulse Width Modulation)

RLAIF: Apprentissage par Renforcement avec Feedback d'Intelligence Artificielle (Reinforcement Learning with Artificial Intelligence Feedback)

RLHF: Apprentissage par Renforcement avec Feedback Humain (Reinforcement Learning with Human Feedback)

USB: Bus Série Universel (Universal Serial Bus)

VLLM: Modèle de Langage Visuel (Visual Language Model)

LISTE DES TABLEAUX

Tableau II- 1: Paramètres de l'architecture de Mistral 8X7B (Jiang et al., 2024).....	26
Tableau II- 2: Performances des quelques LLM sur MMLU et HumanEval(M. Chen et al., 2021; Hendrycks et al., 2021)	30
Tableau IV- 1: Performances des Modèles LLM sur notre Jeu d'Instructions	55
Tableau IV- 2: Instructions de Contrôle Dynamique Exigeant des Boucles Infinies	57
Tableau IV- 3: Exemple des matrices de commande serie	61

LISTE DES ANNEXES

Annexe A: Liste du jeu d'instructions utilisé pour évaluer les modèles sur le contrôle dynamique des circuits électriques	A
Annexe B: Prompt système qui guide les modèles LLM	B
Annexe C: Performances des modèles sur l'écart temporel par instruction.....	F
Annexe D : Notebook des tests de performances des Modèles LLM sur les Instructions de Contrôle Dynamique des Circuits Électriques (BALEMBA, 2024).....	F

INTRODUCTION GENREALE

L'omniprésence des circuits électriques dans les systèmes modernes, de l'industrie à l'Internet des Objets (IoT) en passant par les systèmes embarqués, nécessite des méthodes de contrôle toujours plus performantes et intuitives. L'automatisation croissante des processus, l'optimisation énergétique et la maintenance prédictive exigent des interfaces de contrôle flexibles, adaptatives et faciles d'utilisation (Marr, 2019). Cependant, les approches traditionnelles, basées sur des interfaces graphiques complexes ou des langages de programmation spécialisés, peuvent présenter des obstacles pour les opérateurs non-spécialistes et demandent beaucoup de temps de formation pour être maîtriser. L'essor fulgurant de l'intelligence artificielle (IA), et notamment des grands modèles de langage (LLM), ouvre des perspectives inédites pour révolutionner l'interaction homme-machine et simplifier le contrôle des circuits électriques. Ces modèles, entraînés sur d'immenses corpus de données textuelles, possèdent des capacités remarquables de compréhension du langage naturel et de génération de code (Naveed et al., 2024). Ils permettent d'envisager un contrôle plus intuitif et dynamique, où l'utilisateur pourrait interagir avec les circuits électriques simplement en utilisant le langage naturel. Imaginons un technicien capable de commander un système complexe en disant "Allumer la pompe 2 pendant 5 minutes", ou un ingénieur optimisant un processus industriel en demandant " Ajuster la vitesse du moteur 3 pour minimiser la consommation d'énergie et donne-moi un graphique dynamique d'évaluation en temps réel ". Cette vision d'un contrôle conversationnel des circuits électriques est au cœur de ce mémoire. L'état de l'art actuel des LLM, notamment avec des modèles comme GPT-4o, Mistral Large et Codestral, démontre leur potentiel pour des applications variées, incluant la génération de code Python, la traduction et la réponse à des questions complexes (M. Chen et al., 2021; OpenAI et al., 2024). Ce mémoire explore précisément l'application des LLM au contrôle interactif et dynamique des circuits électriques par le biais du langage naturel. La problématique centrale est de déterminer comment exploiter la puissance des LLM pour traduire des instructions en langage naturel en commandes précises et exécutables par un microcontrôleur, permettant ainsi un contrôle fluide et adaptable des circuits électriques. L'objectif principal de ce travail est de démontrer la faisabilité d'un tel système de contrôle, offrant une interface intuitive et conviviale pour les ingénieurs et les techniciens. L'hypothèse fondamentale est que les LLM sont capables de comprendre les nuances du langage naturel relatives au contrôle électrique, de générer du code Python fonctionnel pour piloter un microcontrôleur, et de s'adapter aux variations des configurations de circuits. Pour valider cette hypothèse, une approche expérimentale a été mise en œuvre. Un système de contrôle a été développé autour d'un Arduino UNO R3, permettant de commander des composants tels que des LEDs, un servomoteur, des capteurs et des moteurs pas à pas. Une interface interactive, utilisant un programme Python personnalisé et la bibliothèque PySerial pour la communication série, a été créée pour relier le LLM à l'Arduino. Différents LLM, incluant GPT-4o, GPT-4o mini, Mistral Large, Mistral 8X7B, Mistral 7B, Mistral Nemo, Codestral, et Gemma 2 9B, ont été testés avec un jeu de données d'instructions en langage naturel. Le code Python généré par le LLM est

exécuté pour envoyer des commandes à l'Arduino, qui les exécute et renvoie un feedback au LLM. Des métriques comme le taux de succès, l'erreur temporelle et le coût d'inférence ont été utilisées pour évaluer les performances des différents modèles. Ce mémoire est structuré en cinq chapitres. Après cette introduction, le premier chapitre présente un aperçu historique du contrôle des circuits électriques et des modèles d'apprentissage automatique, contextualisant le sujet de recherche. Le deuxième chapitre détaille l'état de l'art des LLM et des techniques de contrôle électrique, justifiant les choix technologiques effectués. Le troisième chapitre décrit la méthodologie de conception et d'implémentation du système de contrôle interactif, incluant la présentation des outils, des données et des LLM utilisés. Le quatrième chapitre analyse et interprète les résultats expérimentaux, en les confrontant aux hypothèses de conception. Enfin, le dernier chapitre conclut ce travail en résumant les principales contributions et en proposant des perspectives d'avenir.

CHAPITRE I. ÉVOLUTION HISTORIQUE DU CONTRÔLE DES CIRCUITS ÉLECTRIQUES ET DES MODÈLES D'APPRENTISSAGE AUTOMATIQUE

I.1 Introduction

Ce chapitre introductif a pour objectif de dresser un panorama historique des deux domaines clés qui s'entrecroisent dans le cadre de ce mémoire : le contrôle des circuits électriques et les modèles d'apprentissage automatique. Loin d'être une simple juxtaposition de deux histoires parallèles, ce chapitre vise à mettre en lumière la convergence progressive de ces deux domaines, aboutissant à l'émergence de nouvelles approches pour le contrôle intelligent des systèmes électriques. La première partie de ce chapitre, intitulée "Évolution historique du contrôle des circuits électriques", retrace les grandes étapes qui ont marqué le développement des systèmes de contrôle, depuis les premiers dispositifs électromécaniques jusqu'aux architectures numériques sophistiquées que l'on retrouve dans les systèmes cyber-physiques et l'Internet des objets (IoT). Cette rétrospective historique permettra de comprendre les principes fondamentaux du contrôle électrique, ainsi que les défis technologiques qui ont jalonné son évolution. La seconde partie, "Évolution historique des modèles d'apprentissage automatique", explore l'histoire fascinante de l'intelligence artificielle et de sa branche dédiée à l'apprentissage à partir de données. De la naissance du concept d'intelligence artificielle avec le test de Turing aux premiers modèles de réseaux de neurones, en passant par l'essor de l'apprentissage profond et des grands modèles de langage, cette partie met en lumière les avancées conceptuelles et technologiques qui ont permis aux machines d'atteindre des performances remarquables dans un large éventail de tâches cognitives. En retraçant ces deux histoires parallèles, ce chapitre introductif permettra de mieux appréhender la pertinence et l'originalité de ce travail de mémoire, qui se positionne à l'intersection de ces deux domaines en pleine mutation. L'objectif ultime est de démontrer le potentiel des grands modèles de langage (LLM) pour révolutionner l'interaction homme-machine et ouvrir la voie à un contrôle plus intuitif et flexible des circuits électriques.

I.2. Évolution historique du contrôle des circuits électriques

L'évolution historique du contrôle des circuits électriques est marquée par de nombreuses inventions et développements technologiques qui ont révolutionné les industries et les systèmes de contrôle électrique. Dans cette section, nous allons explorer en détail l'évolution des dispositifs de contrôle électrique, de l'émergence de l'électronique et des tubes à vide, en passant par le développement de l'automatique et des systèmes asservis, jusqu'à l'avènement de l'informatique et des microcontrôleurs. Nous terminerons par une discussion sur les applications actuelles et futures du contrôle électrique dans divers domaines tels que l'industrie, l'Internet des objets (IoT), et les systèmes embarqués.

I.2.1 Les premiers dispositifs de contrôle électrique

Les premiers dispositifs de contrôle électrique ont été développés au cours du 19^e siècle, avec l'invention du relais électromécanique par Joseph Henry en 1835 (W. Aspray, 1990). Les relais électromécaniques sont des interrupteurs commandés par un électroaimant, qui permettent de contrôler des circuits électriques à distance et de développer les réseaux de télégraphie. Au fil du temps, les relais et les contacteurs ont évolué et se sont améliorés, avec l'introduction de relais à mercure, à semi-conducteurs et à état solide (W. Aspray, 1990).

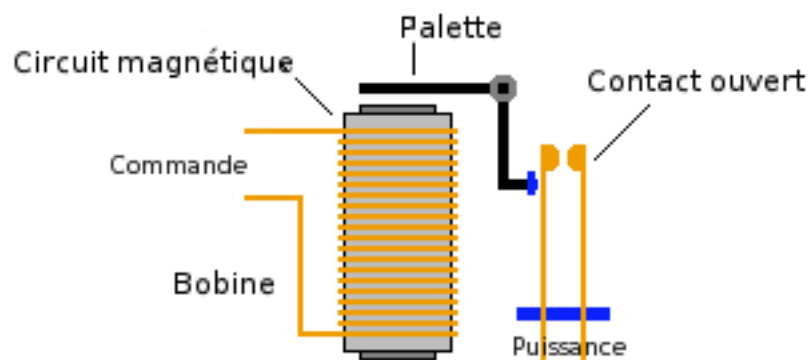


Figure I- 1 : Principe d'un relais électromécanique (« Relais électromécanique », 2024)

Les relais et les contacteurs ont rapidement trouvé leur place dans les systèmes de contrôle industriels, tels que les machines-outils, les ascenseurs et les systèmes de chauffage. Les télégraphes et les réseaux électriques sont des exemples d'applications historiques qui ont révolutionné la communication et la distribution d'énergie (Hughes, 1993). En fait, le télégraphe électrique a été l'une des premières applications pratiques de l'électromagnétisme, permettant la transmission de messages sur de longues distances à l'aide de codes électriques. Les réseaux électriques ont également connu un développement rapide grâce à l'utilisation de relais et de contacteurs pour la distribution et le contrôle de l'énergie électrique (Hughes, 1993).

I.2.1.1 Évolution des relais et des contacteurs

L'invention du relais électromécanique par Joseph Henry en 1835 a marqué le début du contrôle électrique à distance (W. Aspray, 1990). Les relais électromécaniques se composent d'un électroaimant et d'un interrupteur mécanique, qui est actionné lorsque l'électroaimant est alimenté en courant. Cette invention a permis de développer les réseaux de télégraphie, qui ont révolutionné la communication à longue distance.

Au fil du temps, les relais et les contacteurs ont évolué et se sont améliorés. Les relais à mercure, inventés au début du 20^e siècle, utilisaient du mercure comme conducteur électrique, ce qui permettait de réduire l'usure mécanique et d'améliorer la durée de vie des relais. Cependant, les relais à mercure présentaient des inconvénients environnementaux et de sécurité, ce qui a conduit à leur abandon progressif au profit des relais à semi-conducteurs et à état solide (Bennett, 1986).

Les relais à semi-conducteurs et à état solide utilisent des composants électroniques tels que des transistors, des thyristors et des diodes pour contrôler le courant électrique, sans pièces mécaniques en mouvement. Ces relais présentent plusieurs avantages par rapport aux relais électromécaniques, tels qu'une durée de vie plus longue, une vitesse de commutation plus élevée et une consommation d'énergie réduite (W. Aspray, 1990).

I.2.1.2 Utilisation des premiers dispositifs de contrôle électrique industriels

Les relais et les contacteurs ont rapidement trouvé leur place dans les systèmes de contrôle industriels, tels que les machines-outils, les ascenseurs et les systèmes de chauffage. Les relais électromécaniques permettaient de contrôler des circuits électriques à distance, ce qui facilitait la mise en œuvre de systèmes de contrôle complexes (W. Aspray, 1990).

Les télégraphes et les réseaux électriques sont des exemples d'applications historiques qui ont révolutionné la communication et la distribution d'énergie. Le télégraphe électrique, inventé par Samuel Morse en 1837, utilisait des relais électromécaniques pour transmettre des messages codés à l'aide d'impulsions électriques (Bennett, 1986). Les réseaux électriques, développés à la fin du 19^e siècle, utilisaient des relais et des contacteurs pour contrôler la distribution d'énergie électrique et assurer la sécurité des installations (Bennett, 1986).

I.2.2 L'émergence de l'électronique et des tubes à vide

L'émergence de l'électronique et des tubes à vide a marqué une étape importante dans l'évolution du contrôle électrique. Les tubes à vide, inventés par Lee De Forest en 1906, ont permis d'amplifier et de moduler les signaux électriques, ce qui a ouvert la voie au développement des premiers systèmes de contrôle électroniques.

I.2.2.1 Introduction des tubes à vide et leur impact sur le contrôle électrique

Les tubes à vide, également appelés lampes à vide ou triodes, sont des dispositifs électroniques qui permettent d'amplifier et de moduler les signaux électriques (Bennett, 1986). Les tubes à vide se composent d'une cathode chauffée, d'une grille de commande et d'une anode, placées dans un tube de verre sous vide, comme illustré dans la figure I-2. Lorsque la grille de commande est alimentée en tension, elle contrôle le flux d'électrons entre la cathode et l'anode, ce qui permet d'amplifier ou de moduler le signal électrique.

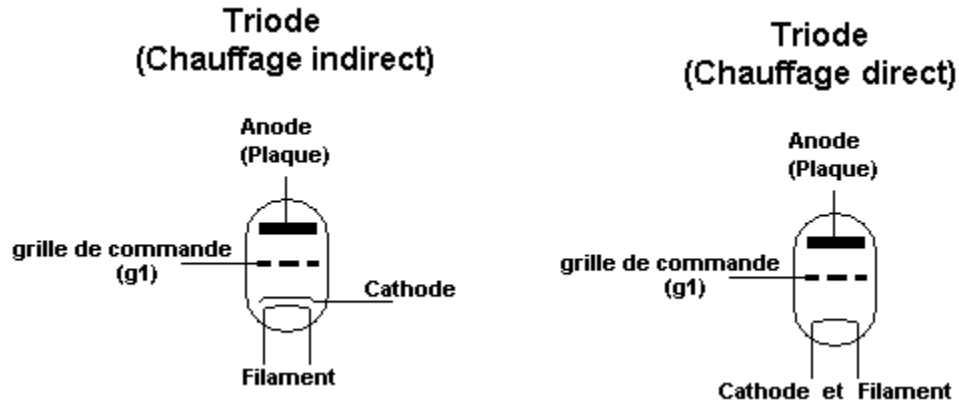


Figure I- 2 : Principe des triodes à chauffage indirect et direct (« Triode (électronique) », 2023)

Les tubes à vide ont rapidement trouvé leur place dans les premiers systèmes de contrôle électroniques, tels que les radios, les téléviseurs et les ordinateurs. Les tubes à vide ont permis de développer des systèmes de contrôle plus complexes et plus performants que les relais électromécaniques, en offrant une meilleure précision et une plus grande rapidité de commutation (W. F. Aspray, 1985).

I.2.2.2 Développement de l'électronique et ses applications dans le contrôle électrique

Le développement de l'électronique a été marqué par l'introduction de nouveaux composants électroniques tels que les transistors, les diodes et les circuits intégrés, qui ont révolutionné les systèmes de contrôle électrique (Bennett, 1986). Les transistors, inventés en 1947 par John Bardeen, Walter Brattain et William Shockley, sont des dispositifs électroniques à semi-conducteurs qui permettent d'amplifier et de commuter des signaux électriques (Riordan & Hoddeson, 1997). Les transistors présentent plusieurs avantages par rapport aux tubes à vide, tels qu'une taille réduite, une consommation d'énergie plus faible et une durée de vie plus longue (Riordan & Hoddeson, 1997).

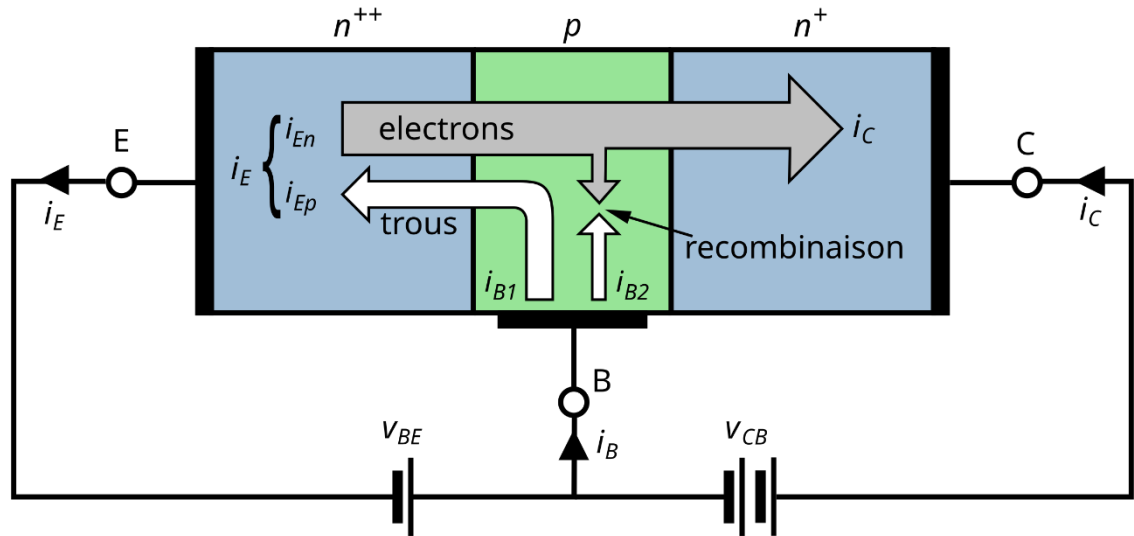


Figure I- 3 : Principe de fonctionnement d'un transistor NPN(« Bipolar junction transistor », 2024)

La transition des tubes à vide vers les transistors et les circuits intégrés a permis de miniaturiser et d'améliorer les performances des systèmes de contrôle électrique(Riordan & Hoddeson, 1997). Les circuits intégrés, développés à partir des années 1960, permettent d'intégrer plusieurs composants électroniques sur une seule puce de silicium, ce qui réduit la taille et le coût des systèmes électroniques (Riordan & Hoddeson, 1997).

I.2.3 Le développement de l'automatique et des systèmes asservis

Le développement de l'automatique et des systèmes asservis a joué un rôle crucial dans l'évolution du contrôle électrique. Les systèmes asservis permettent de réguler et de maintenir une grandeur physique (vitesse, température, etc.) à une valeur désirée, en utilisant des boucles de rétroaction (Ogata, 2010).

I.2.3.1Évolution des systèmes asservis et leur importance dans le contrôle électrique

Les systèmes asservis sont basés sur le principe de la rétroaction, qui consiste à utiliser la sortie d'un système pour contrôler son entrée (Ogata, 2010). Les systèmes asservis permettent de réguler et de maintenir une grandeur physique à une valeur désirée, en ajustant automatiquement les paramètres du système en fonction des variations de l'environnement(Ogata, 2010).

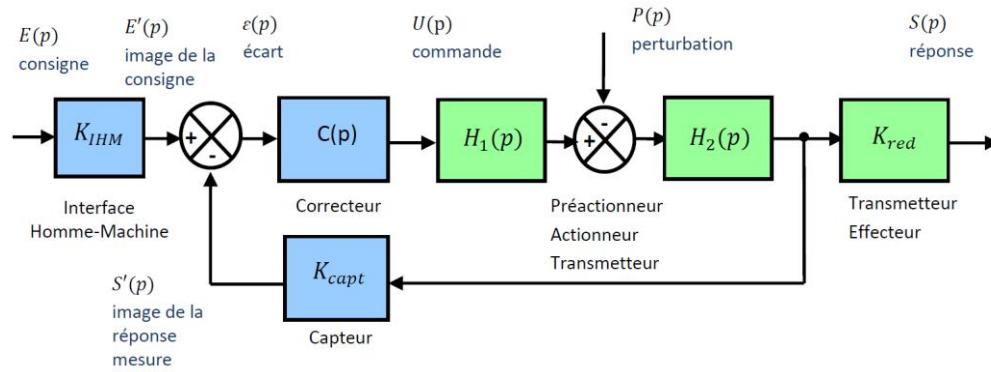


Figure I- 4: Schéma d'un système asservi (« Asservissement (automatique) », 2024)

L'histoire du développement des systèmes asservis remonte à James Watt, qui a inventé le régulateur à boules (Figure I-5) en 1788 pour contrôler la vitesse des machines à vapeur (Bennett, 1986). Au cours du 19^e siècle, de nombreux chercheurs ont contribué au développement des systèmes asservis, tels que Maxwell, Vyshnegradsky et Poincaré (Bennett, 1986).

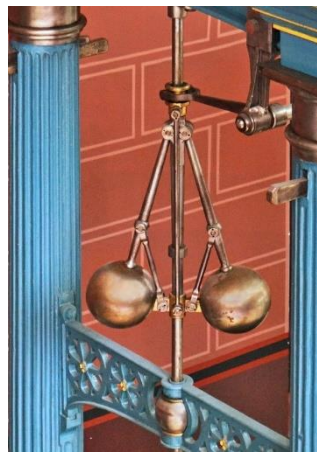


Figure I- 5: Régulateur à boules de la machine à vapeur de Scott (musée des arts et métiers, Paris) (« Régulateur à boules », 2024)

I.2.3.2 Développement de l'automatique et son influence sur le contrôle des circuits électriques

L'automatique est une discipline scientifique qui étudie les systèmes asservis et leur contrôle (Ogata, 2010). L'émergence de l'automatique comme discipline scientifique est étroitement liée au développement des systèmes asservis et à leur utilisation dans les systèmes de contrôle électrique (Bennett, 1986).

Les travaux de Norbert Wiener sur la cybérnétique, publiés en 1948, ont jeté les bases de l'automatique moderne. Wiener a introduit le concept de rétroaction négative, qui permet de stabiliser les systèmes asservis et de réduire les erreurs de contrôle. Les travaux de Rudolf Kalman

sur le filtrage et l'estimation d'état, publiés dans les années 1960, ont également contribué au développement de l'automatique et à son application dans les systèmes de contrôle électrique (*Kalman1960.pdf*, s. d.).

Les applications de l'automatique dans les systèmes de contrôle électrique sont nombreuses et variées, telles que la régulation de vitesse des moteurs, la commande de position des servomécanismes, et la gestion de l'énergie dans les réseaux électriques.

I.2.4 L'avènement de l'informatique et des microcontrôleurs

L'avènement de l'informatique et des microcontrôleurs a révolutionné le contrôle électrique en permettant de traiter et de stocker l'information numérique de manière rapide et efficace (Bennett, 1986). Les microprocesseurs et les microcontrôleurs ont permis d'intégrer l'informatique dans les systèmes de contrôle électrique, avec l'utilisation d'automates programmables, de cartes d'acquisition et de logiciels de commande (W. Aspray, 1990).

I.2.4.1 Révolution de l'informatique et son impact sur le contrôle électrique

L'invention du microprocesseur par Intel en 1971 a marqué le début de la révolution de l'informatique (W. Aspray, 1990). Les microprocesseurs sont des circuits intégrés qui permettent de traiter l'information numérique de manière rapide et efficace, en exécutant des instructions stockées en mémoire (W. Aspray, 1990). La figure I-6 montre un aperçu de l'un des premières versions des microprocesseurs Intel

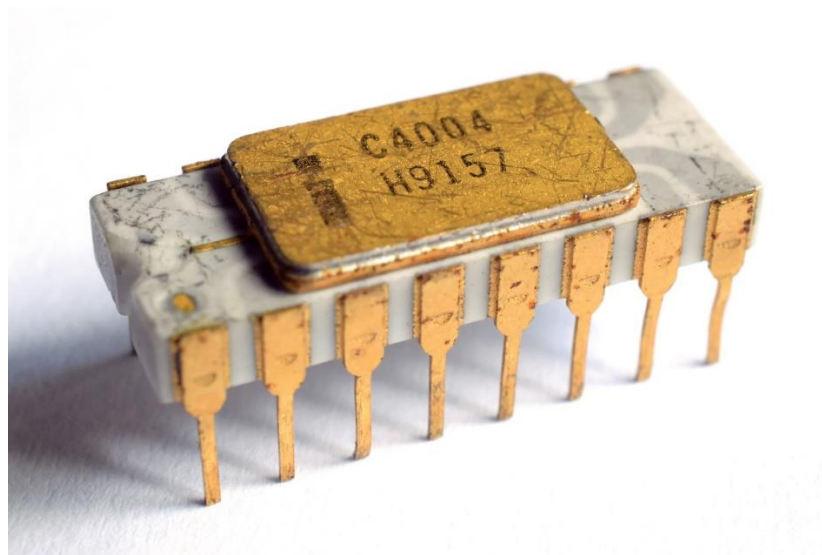


Figure I- 6: l'une des premières version du premier microprocesseur Intel_C4004(« Intel 4004 », 2024)

L'intégration de l'informatique dans les systèmes de contrôle électrique a permis de développer des systèmes de contrôle plus complexes et plus performants, en utilisant des automates programmables, des cartes d'acquisition et des logiciels de commande (Bennett, 1986). Les automates programmables sont des dispositifs électroniques qui permettent de contrôler des processus industriels en exécutant des programmes stockés en mémoire (W. Aspray, 1990). Les cartes d'acquisition permettent de convertir des signaux analogiques en signaux numériques, qui peuvent être traités par un microprocesseur (Ceruzzi, 1998). Les logiciels de commande permettent de programmer et de contrôler des systèmes de contrôle électrique à l'aide d'un ordinateur (W. Aspray, 1990).

I.2.4.2 Développement des microcontrôleurs et leur rôle dans le contrôle des circuits électriques

Les microcontrôleurs sont des circuits intégrés qui combinent un microprocesseur, une mémoire et des périphériques d'entrée/sortie sur une seule puce. Les microcontrôleurs sont largement utilisés dans les systèmes de contrôle électrique modernes, tels que les variateurs de vitesse, les onduleurs et les convertisseurs d'énergie.

Les microcontrôleurs permettent de contrôler des circuits électriques de manière précise et efficace, en exécutant des programmes stockés en mémoire. Les microcontrôleurs présentent plusieurs avantages par rapport aux relais électromécaniques et aux tubes à vide, tels qu'une taille réduite, une consommation d'énergie plus faible et une durée de vie plus longue (W. Aspray, 1990).

I.2.5 Les applications actuelles et futures du contrôle électrique

Le contrôle électrique est aujourd'hui omniprésent dans de nombreux domaines, tels que l'industrie, l'Internet des objets (IoT), et les systèmes embarqués. Dans cette section, nous allons explorer quelques-unes des applications actuelles et futures du contrôle électrique dans ces domaines.

I.2.5.1 Contrôle électrique dans l'industrie 4.0 et les systèmes cyber-physiques

L'industrie 4.0, également appelée quatrième révolution industrielle, est une tendance actuelle qui vise à intégrer les technologies numériques dans les processus de production, afin d'améliorer la flexibilité, l'efficacité et la qualité (Kagermann et al., 2013). Les systèmes cyber-physiques sont des systèmes qui associent des composants physiques (capteurs, actionneurs, etc.) et des composants numériques (logiciels, réseaux, etc.) pour interagir avec leur environnement (Lee, 2008).

Le contrôle électrique joue un rôle crucial dans les systèmes cyber-physiques et l'industrie 4.0, en permettant de contrôler et de réguler les processus de production en temps réel (Kagermann et al., 2013). Les systèmes de contrôle électrique modernes utilisent des capteurs, des actionneurs et des microcontrôleurs pour contrôler les machines et les équipements industriels, en utilisant des algorithmes avancés de contrôle et de régulation (Kagermann et al., 2013).

I.2.5.2 Internet des objets (IoT) et contrôle électrique embarqué

L'Internet des objets (IoT) est un réseau d'objets connectés, qui communiquent entre eux et avec l'environnement à l'aide de capteurs, d'actionneurs et de microcontrôleurs (Atzori et al., 2010). Le contrôle électrique embarqué est une application importante de l'IoT, qui permet de contrôler et de réguler les objets connectés en temps réel (Atzori et al., 2010).

Les systèmes de contrôle électrique embarqués sont utilisés dans de nombreuses applications de l'IoT, telles que les systèmes de surveillance de la santé, les systèmes de contrôle de l'éclairage, et les systèmes de contrôle de la température (Atzori et al., 2010). Les systèmes de contrôle électrique embarqués présentent plusieurs défis, tels que la consommation d'énergie, la sécurité, la fiabilité et la maintenance des systèmes (Atzori et al., 2010).

I.2.5.3 Perspectives d'avenir pour le contrôle électrique

Les perspectives d'avenir pour le contrôle électrique sont vastes et variées, avec de nombreuses tendances et innovations qui émergent dans ce domaine. Parmi ces tendances, on peut citer l'intelligence artificielle, l'apprentissage automatique, la réalité virtuelle et augmentée, et les matériaux avancés (Atzori et al., 2010). L'intelligence artificielle et l'apprentissage automatique sont des technologies émergentes qui ont le potentiel de révolutionner le contrôle électrique en permettant aux systèmes de contrôle d'apprendre et de s'adapter à leur environnement (Atzori et al., 2010). Les systèmes de contrôle électrique basés sur l'intelligence artificielle et l'apprentissage automatique peuvent améliorer la précision, la rapidité et l'efficacité des systèmes de contrôle électrique.

La réalité virtuelle et augmentée est une autre tendance émergente dans le domaine du contrôle électrique, qui permet de visualiser et d'interagir avec les systèmes de contrôle électrique de manière immersive et intuitive (Lascar, 2022). Les matériaux avancés, tels que les matériaux composites et les matériaux intelligents, offrent également de nouvelles possibilités pour le contrôle électrique, en permettant de concevoir des systèmes de contrôle plus légers, plus compacts et plus performants (Lascar, 2022).

Les applications actuelles et futures du contrôle électrique sont vastes et variées, allant de l'industrie 4.0 et les systèmes cyber-physiques à l'Internet des objets (IoT) et les systèmes embarqués. Les perspectives d'avenir pour le contrôle électrique sont prometteuses, avec de

nombreuses tendances et innovations qui émergent dans ce domaine, telles que l'intelligence artificielle, l'apprentissage automatique, la réalité virtuelle et augmentée, et les matériaux avancés.

I.3 Evolution historique des modèles d'apprentissage automatique

L'apprentissage automatique, ou machine learning, est un domaine de l'intelligence artificielle qui vise à développer des algorithmes permettant aux machines d'apprendre à partir de données. L'évolution des modèles d'apprentissage automatique a été marquée par de nombreuses avancées technologiques et scientifiques, qui ont permis de développer des algorithmes de plus en plus performants et adaptés à un large éventail d'applications. Cette section présente un aperçu détaillé de l'histoire de l'apprentissage automatique, depuis ses origines jusqu'aux récents développements des grands modèles de langage et des Transformers.

I.3.1 Les origines de l'intelligence artificielle et du test de Turing

L'intelligence artificielle (IA) trouve ses racines dans les travaux de pionniers tels que Alan Turing, John McCarthy, et Marvin Minsky, qui ont cherché à créer des machines capables d'imiter ou de dépasser les performances humaines dans des tâches cognitives. L'un des premiers concepts clés de l'IA (Intelligence Artificielle) est le test de Turing, proposé par Alan Turing en 1950 (A.M Turing, 1950). Ce test consiste à évaluer la capacité d'une machine à imiter la conversation humaine de manière convaincante, en la faisant interagir avec un interlocuteur humain à travers un terminal texte, comme illustré dans la figure I-7. Si l'interlocuteur humain ne parvient pas à distinguer la machine d'un véritable humain, on considère que la machine a réussi le test.

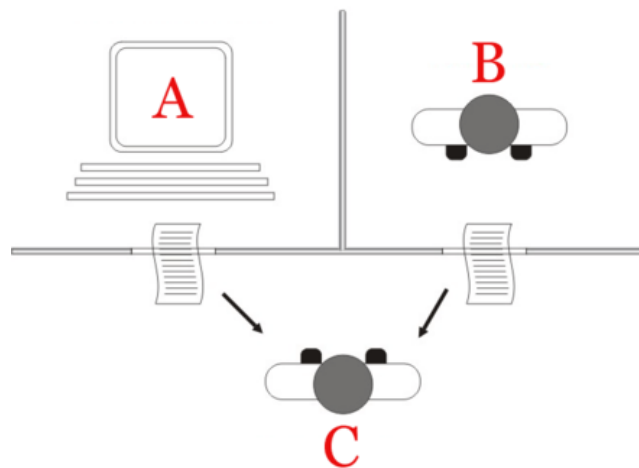


Figure I- 7: Diagramme du Test de Turing Turing (« Test de Turing », 2024)

Les premiers modèles d'apprentissage automatique ont commencé à émerger dans les années 1950 et 1960, avec le développement de techniques telles que le perceptron (Rosenblatt, 1958), la régression linéaire, et les machines à vecteurs de support (Goodfellow et al., 2016). Ces

modèles ont jeté les bases de l'apprentissage automatique, en permettant aux machines d'apprendre à partir de données et d'améliorer leurs performances au fil du temps.

I.3.2 Les premiers modèles d'apprentissage automatique

L'un des premiers modèles d'apprentissage automatique développés est le perceptron, proposé par Frank Rosenblatt en 1957 (Rosenblatt, 1958). Le perceptron est un algorithme d'apprentissage supervisé qui permet de classer des données linéairement séparables en deux catégories. Il est basé sur un modèle simple de réseau de neurones à une couche, où chaque neurone calcule une combinaison linéaire des entrées, suivie d'une fonction d'activation. Le perceptron comme l'est illustré sur la figure 1-8 a pour entrées i_1 à i_n qui sont multipliées avec les poids W_1 à W_n , puis sommées avant qu'une fonction d'activation soit appliquée

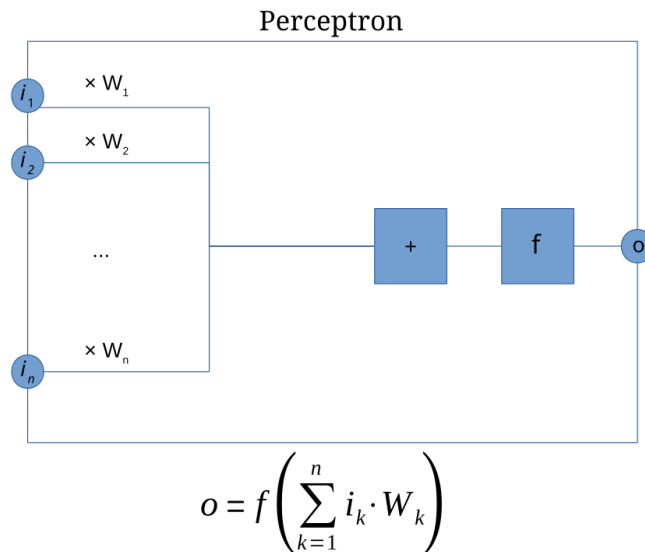


Figure I- 8: le Perceptron(« Perceptron », 2024)

Les réseaux de neurones artificiels (RNA) sont une extension du perceptron, qui permettent de représenter et d'apprendre des relations non linéaires entre les entrées et les sorties. Les RNA sont composés de plusieurs couches de neurones interconnectés, où chaque couche traite les informations provenant de la couche précédente et transmet le résultat à la couche suivante. L'apprentissage dans les RNA se fait par rétropropagation du gradient, qui consiste à ajuster les poids des connexions entre les neurones de manière à minimiser l'erreur de prédiction (Rumelhart et al., 1986).

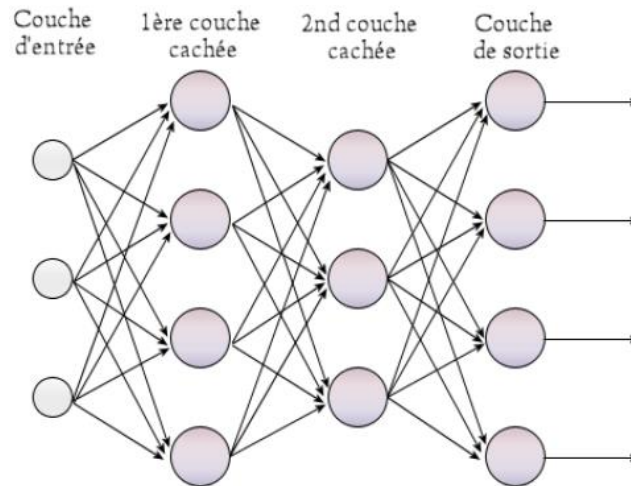


Figure I- 9: Un perceptron multicouche (« Perceptron multicouche », 2023)

Au fil des années, les modèles d'apprentissage automatique se sont diversifiés et complexifiés, avec le développement de techniques telles que les arbres de décision (Quinlan, 1986), les réseaux de Bayes (Pearl, 1988), et les machines à vecteurs de support. Ces modèles ont permis d'aborder un large éventail de problèmes d'apprentissage, tels que la classification, la régression, le clustering, et la détection d'anomalies.

I.3.3 La crise de l'IA et le renouveau de l'apprentissage automatique

Dans les années 1970 et 1980, l'IA a connu une période de stagnation, souvent appelée "hiver de l'IA", due à plusieurs facteurs tels que les limites des premiers modèles d'apprentissage, les attentes excessives, et le manque de financement (Crevier, 1993). Cette crise a incité les chercheurs à explorer de nouvelles approches et à développer des méthodes d'apprentissage plus robustes et adaptatives.

Un tournant majeur dans l'histoire de l'apprentissage automatique a été l'introduction des méthodes d'apprentissage profond (deep learning), qui ont permis d'améliorer les performances des réseaux de neurones et d'élargir leur domaine d'application (LeCun et al., 2015). Les réseaux de neurones profonds sont caractérisés par un grand nombre de couches cachées, qui leur permettent d'apprendre des représentations hiérarchiques des données, en extrayant automatiquement les caractéristiques pertinentes à différents niveaux d'abstraction.

Le développement des méthodes d'apprentissage profond a été rendu possible grâce à plusieurs avancées technologiques et méthodologiques, telles que l'augmentation de la puissance de calcul, la disponibilité de grandes quantités de données d'entraînement, et l'amélioration des algorithmes d'optimisation (Schmidhuber, 2015). Ces avancées ont permis de surmonter les limitations des modèles d'apprentissage traditionnels et d'ouvrir la voie à de nouvelles applications

dans divers domaines, tels que la reconnaissance d'images, la synthèse vocale, et la traduction automatique.

I.3.4 L'essor des grands modèles de langage (LLM) et des Transformers

Les grands modèles de langage (LLM) sont une classe de modèles d'apprentissage profond spécialisés dans le traitement et la génération de langage naturel. Ces modèles sont capables de comprendre et de produire du texte cohérent et pertinent, en s'appuyant sur des connaissances linguistiques et factuelles acquises lors de leur pré-entraînement sur de vastes corpus de données textuelles (Devlin et al., 2019; Radford et al., s. d.).

Les grands modèles linguistiques (LLM) ont attiré beaucoup d'attention en raison de leurs solides performances sur un large éventail de tâches en langage naturel, depuis la sortie de ChatGPT en novembre 2022. La capacité des LLM à comprendre et à générer un langage général s'acquiert par la formation des milliards de paramètres de modèle sur des quantités massives de données textuelles, comme le prédisent les lois d'échelle. Les LLM ont connu un essor spectaculaire ces dernières années, avec le développement de modèles tels que GPT-3. Ces modèles ont battu de nombreux records de performance dans diverses tâches de traitement du langage naturel, telles que la compréhension de lecture, la réponse aux questions, et la génération de texte.

L'un des facteurs clés du succès des LLM est l'utilisation des Transformeurs, une architecture (figure I-10) de réseau de neurones introduite par (Vaswani et al., 2023). Les Transformeurs se distinguent des architectures précédentes, telles que les réseaux récurrents (RNN) et les réseaux convolutifs (CNN), par leur capacité à traiter les séquences de données en parallèle, grâce à l'utilisation de mécanismes d'attention. Cette propriété permet aux Transformers de traiter des séquences de longueur variable de manière efficace, en se focalisant sur les parties les plus pertinentes de la séquence à chaque étape de traitement.

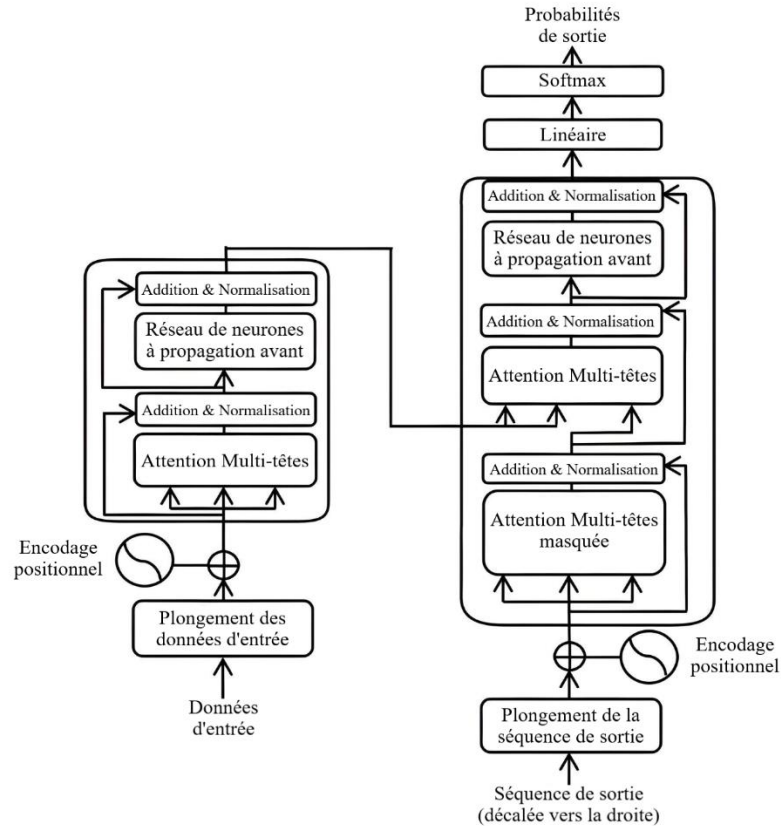


Figure I- 10: Architecture générale d'un Transformeur(« Transformeur », 2024)

Les Transformers ont rapidement été adoptés dans de nombreux modèles d'apprentissage profond, en particulier dans le domaine du traitement du langage naturel, où ils ont permis d'améliorer considérablement les performances des LLM (Brown et al., 2020; Devlin et al., 2019; Radford et al., s. d.). Les Transformers ont également ouvert la voie à de nouvelles applications, telles que la traduction automatique en temps réel, la génération de résumés, et la modélisation de dialogues.

I.3.5 Les défis et les enjeux de l'apprentissage automatique

Malgré les avancées spectaculaires réalisées dans le domaine de l'apprentissage automatique, de nombreux défis et enjeux subsistent, tant sur le plan technique que sur le plan éthique et social. Parmi ces défis, on peut citer le coût et la consommation d'énergie liés à l'entraînement des modèles d'apprentissage, la protection de la vie privée, et la nécessité de réguler et d'encadrer l'utilisation de l'IA dans la société.

Le coût et la consommation d'énergie associés à l'entraînement des modèles d'apprentissage automatique, en particulier des LLM, sont devenus un enjeu majeur, compte tenu de l'augmentation rapide de la taille et de la complexité de ces modèles (Strubell et al., 2019). Des

solutions sont actuellement explorées pour réduire ces coûts, telles que l'optimisation des algorithmes d'entraînement, la compression des modèles, et le transfer learning (M. Chen et al., 2021; Howard & Ruder, 2018).

Sur le plan éthique, l'utilisation de l'apprentissage automatique soulève de nombreuses questions, telles que la protection de la vie privée, la transparence, l'équité, et la non-discrimination (Jobin et al., 2019). Il est essentiel de développer des méthodes d'apprentissage éthiques et responsables, qui prennent en compte ces enjeux et garantissent un usage respectueux des droits et des libertés fondamentales des individus.

Enfin, l'encadrement et la régulation de l'IA dans la société sont des enjeux cruciaux, afin d'éviter les dérives et les abus liés à l'utilisation de ces technologies. Des initiatives sont en cours au niveau national et international pour établir des cadres juridiques et éthiques adaptés à l'IA, et pour promouvoir un développement et une utilisation responsables de ces technologies (Jobin et al., s. d.).

I.4 Conclusion

Ce premier chapitre a retracé l'évolution historique parallèle du contrôle des circuits électriques et des modèles d'apprentissage automatique, deux domaines distincts mais inexorablement liés dans le contexte de ce mémoire. De la simplicité des premiers dispositifs de contrôle, tels que les relais électromécaniques, à la sophistication des systèmes cyber-physiques et de l'Internet des objets (IoT), le contrôle des circuits électriques a connu des avancées fulgurantes, nourries par les progrès constants de l'électronique et de l'informatique. Simultanément, le domaine de l'apprentissage automatique a émergé de ses balbutiements, marqué par le test de Turing et les premiers réseaux de neurones, pour atteindre des sommets de performance avec l'avènement de l'apprentissage profond et des grands modèles de langage (LLM) basés sur l'architecture des Transformers. Ces LLM, capables de comprendre et de générer du langage naturel avec une fluidité sans précédent, ouvrent un champ des possibles dans l'interaction homme-machine. C'est dans ce contexte de convergence technologique que s'inscrit ce mémoire. En effet, l'objectif principal de ce travail est d'explorer et d'exploiter le potentiel des LLM pour révolutionner le contrôle des circuits électriques, en permettant une interaction intuitive et flexible basée sur le langage naturel. La combinaison de ces deux domaines, autrefois distincts, promet de transformer radicalement la manière dont les ingénieurs et les techniciens interagissent avec les systèmes électriques, ouvrant la voie à une nouvelle ère de contrôle intelligent et adaptatif. Le chapitre suivant s'attachera à examiner en détail l'état de l'art des LLM et leurs applications dans divers domaines, afin de mieux cerner leur potentiel disruptif pour le contrôle des circuits électriques.

CHAPITRE II : ETAT DE L'ART

II.1 Introduction

Ce chapitre propose une analyse approfondie des fondements sur lesquels repose ce mémoire, à savoir les Large Language Models (LLM) et les techniques de contrôle des circuits électriques. Dans un premier temps, nous explorerons en détail les LLM, en nous attardant sur leur architecture, leurs capacités et leurs limites. Une sélection de modèles représentatifs sera présentée et comparée afin de justifier le choix des LLM retenus pour l'implémentation du système de contrôle interactif développé dans ce travail. Ensuite, nous nous pencherons sur le contrôle des circuits électriques, en passant en revue les concepts fondamentaux, les types de contrôle, les outils et méthodes traditionnels, ainsi que les applications et tendances actuelles. Cette analyse permettra de contextualiser le présent travail par rapport aux solutions existantes. Enfin, nous conclurons ce chapitre par un positionnement clair de notre recherche, en soulignant son originalité et sa contribution potentielle au domaine du contrôle des circuits électriques par le biais de l'interaction en langage naturel. En d'autres termes, ce chapitre a pour objectif de démontrer la pertinence et le caractère novateur de l'utilisation des LLM pour le contrôle interactif et dynamique des circuits électriques par le langage naturel.

II.2 Revue de la littérature sur les LLM

II.2.1 Définition et concept des LLM

Un grand modèle de langage, grand modèle linguistique, grand modèle de langue, modèle de langage de grande taille ou encore modèle massif de langage (abrégé LLM de l'anglais large language model) est un modèle de langage possédant un grand nombre de paramètres (généralement de l'ordre du milliard de poids ou plus).(Naveed et al., 2024)

Ce sont des réseaux de neurones profonds entraînés sur de grandes quantités de texte non étiqueté utilisant l'apprentissage auto-supervisé ou l'apprentissage semi-supervisé. Les LLM sont apparus vers 2018 et ont été utilisés pour la mise en œuvre d'agents conversationnels.(Naveed et al., 2024)

II.2.2 Introduction aux grand modèle de langage (LLM)

Les Large Language Models (LLM) représentent une avancée significative dans le domaine de l'intelligence artificielle, excellent dans un large éventail de tâches liées au langage naturel. Contrairement aux modèles de langage traditionnels, qui sont souvent spécialisés pour des tâches spécifiques telles que l'analyse des sentiments, la reconnaissance d'entités nommées ou le raisonnement mathématique, les LLM sont conçus pour générer des suites de texte probables en

réponse à une entrée donnée.(Naveed et al., 2024) La performance de ces modèles dépend fortement de la quantité et de la qualité des ressources utilisées pour leur entraînement, notamment la taille des paramètres, la puissance de calcul, et les données disponibles.

Les modèles de langage possédant un grand nombre de paramètres s'avèrent capable de capturer une grande partie de la syntaxe et de la sémantique du langage humain. Ils font également preuve d'une connaissance générale considérable sur le monde, et sont capables de « mémoriser » une grande quantité de faits lors de l'entraînement (Naveed et al., 2024).

Les LLM sont capables d'apprendre des représentations linguistiques riches et complexes, ce qui leur permet de s'adapter à diverses tâches liées au langage naturel (Naveed et al., 2024).

II.2.2.1 Différence entre LLM et les modèles de langage traditionnels

Les LLM se distinguent des modèles de langage traditionnels par leur taille, leur capacité à généraliser et leur aptitude à apprendre des représentations linguistiques plus riches. Alors que les modèles de langage traditionnels sont généralement entraînés sur des corpus de données spécifiques à un domaine, les LLM sont entraînés sur des corpus de données beaucoup plus vastes et diversifiés, ce qui leur confère une plus grande adaptabilité aux différents contextes et tâches (Devlin et al., 2019).

II.2.2.2 Importance des LLM dans le domaine de l'intelligence artificielle

Les LLM jouent un rôle crucial dans le domaine de l'intelligence artificielle, car ils permettent d'améliorer les performances des systèmes de traitement automatique du langage naturel (NLP) et d'étendre leurs applications à divers domaines. En effet, les LLM sont capables de comprendre et de générer du langage naturel de manière cohérente et pertinente, ce qui les rend aptes à interagir avec les humains et à effectuer des tâches complexes (Brown et al., 2020).

Les progrès récents en matière de grands modèles de langage (LLM) basés sur des transformateurs, pré-entraînés sur des corpus de textes à l'échelle du Web, ont considérablement étendu les capacités des modèles de langage (LLM). Par exemple, ChatGPT et GPT-4 d'OpenAI peuvent être utilisés non seulement pour le traitement du langage naturel, mais également pour la génération de code et le raisonnement intégrant des capacités cognitives.(OpenAI et al., 2024)

Les LLM peuvent également être complétés en utilisant des connaissances et des outils externes (W. Chen, 2023) afin qu'ils puissent interagir efficacement avec les utilisateurs et l'environnement , et s'améliorer continuellement en utilisant les données de feedback collectées via les interactions (par exemple via l'apprentissage par renforcement avec feedback humain (RLHF)) (Naveed et al., 2024).

Grâce à des techniques avancées d'utilisation et d'augmentation, les LLM peuvent être déployés en tant qu'agents d'IA : des entités artificielles qui détectent leur environnement, prennent des décisions et agissent. Des recherches antérieures se sont concentrées sur le développement d'agents pour des tâches et des domaines spécifiques (Xia et al., 2023). Les capacités émergentes démontrées par les LLM permettent de construire des agents d'IA polyvalents basés sur les LLM. Alors que les LLM sont formés pour produire des réponses dans des environnements statiques, les agents d'IA doivent prendre des mesures pour interagir avec un environnement dynamique. Par conséquent, les agents basés sur LLM ont souvent besoin d'augmenter les LLM pour, par exemple, obtenir des informations mises à jour à partir de bases de connaissances externes, vérifier si une action du système produit le résultat attendu et faire face aux cas où les choses ne se passent pas comme prévu, etc (Xia et al., 2023).

II.2.3 Architecture des LLM

Les LLM sont basés sur des réseaux de neurones profonds, qui sont des modèles d'apprentissage automatique capables d'apprendre des représentations hiérarchiques des données. Les réseaux de neurones profonds sont composés de plusieurs couches de neurones artificiels interconnectés, qui permettent au modèle d'extraire des caractéristiques de plus en plus abstraites à partir des données d'entrée (LeCun et al., 2015).

II.2.3 .1 La structure Transformer et son rôle dans les LLM

Un modèle de langage reçoit typiquement en entrée des données séquentielles de longueur variable. Pendant longtemps, l'architecture utilisée préférentiellement pour ce genre de données était celle dite de réseaux de neurones récurrents. Cette architecture présentait comme inconvénient majeur de mal se prêter à la parallélisation des calculs nécessaires à l'entraînement.

C'est l'avènement de l'architecture transformer, et surtout les gains en performance qu'elle procure, qui ont permis aux chercheurs d'augmenter considérablement le nombre de paramètres de leurs modèles, d'où le qualificatif « grand » les concernant. La structure Transformer est un type d'architecture de réseau de neurones introduite par (Vaswani et al., 2023) pour améliorer l'efficacité et la qualité de l'apprentissage des modèles de langage. Elle repose sur des mécanismes d'attention qui permettent au modèle de se concentrer sur les parties pertinentes du texte lors du traitement du langage naturel. La structure Transformer est devenue un élément clé des LLM, car elle permet d'améliorer leur capacité à traiter de longues séquences de texte et à capturer des dépendances linguistiques à long terme (Vaswani et al., 2023).

L'architecture Transformer générative pré-entraîné (GPT pour Generative Pre-trained Transformer en anglais) utilisée actuellement par les LLM les plus performant a été popularisée par OpenAI ; cet architecture (figure II-1) a abouti à des performances de transfert robustes sur diverses tâches. L'apprentissage auto-supervisé utilisé par OpenAI pour entraîner ses grands

modèles de langage commence par une étape de « pré-entraînement », où le modèle est entraîné à prédire le token suivant (un token étant une séquence de caractères, typiquement un mot, une partie d'un mot, ou de la ponctuation). Cet entraînement à prédire ce qui va suivre, répété pour un grand nombre de textes, permet à ces modèles d'accumuler des connaissances sur le monde (He et al., 2021, p.12).

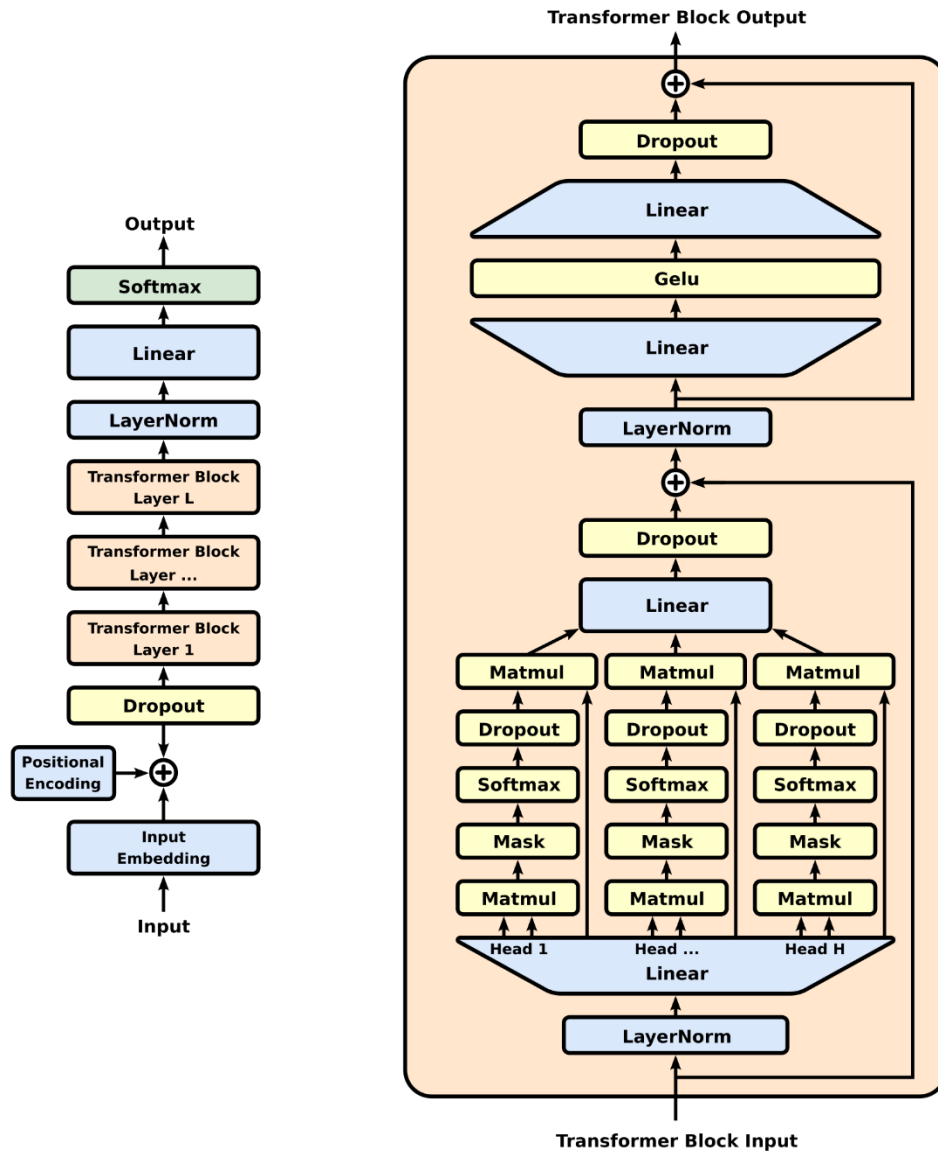


Figure II- 1: Architecture GPT de base (« Transformeur génératif pré-entraîné », 2024)

Il y a ensuite parfois une étape d'apprentissage supervisé où le modèle est ajusté pour une tâche donnée, par exemple pour obtenir des réponses selon un format ou un style d'assistant. Il y a également souvent une étape d'apprentissage par renforcement (telle que RLHF ou RLAIIF) permettant de rendre le modèle plus véridique, utile et inoffensif.

II.2.3.2 L'entraînement des LLM sur de larges corpus de données

Les LLM sont entraînés sur de vastes corpus de données textuelles pour apprendre à prédire la probabilité d'occurrence d'un mot ou d'une phrase donnée. Cette approche permet aux modèles d'acquérir des connaissances linguistiques étendues et de s'adapter à diverses tâches. L'entraînement des LLM nécessite des ressources informatiques importantes et des techniques d'optimisation avancées pour gérer efficacement les grandes quantités de données et réduire le temps d'apprentissage (Radford et al., s. d.).

La plupart des LLM sont entraînés par pré-entraînement génératif, c'est-à-dire qu'étant donné un ensemble de données d'entraînement de jetons de texte, le modèle prédit les jetons dans l'ensemble de données. Il existe deux styles généraux de pré-entraînement pour la génération (Naveed et al., 2024) :

- Autorégressif (style GPT, "prédire le mot suivant") : étant donné un segment de texte comme "J'aime manger", le modèle prédit les jetons suivants, comme "crème glacée".
- Masqué ("style BERT", "test de cloze") : Étant donné un segment de texte comme "J'aime [MASQUE] [MASQUE] glacée", le modèle prédit les jetons masqués, comme "manger de la crème".

Les LLM peuvent être entraînés sur des tâches auxiliaires qui testent leur compréhension de la distribution des données, telles que la prédiction de la phrase suivante, dans laquelle des paires de phrases sont présentées et le modèle doit prédire si elles apparaissent consécutivement dans le corpus d'entraînement.

II.1.4 Capacités et caractéristiques clés des LLM

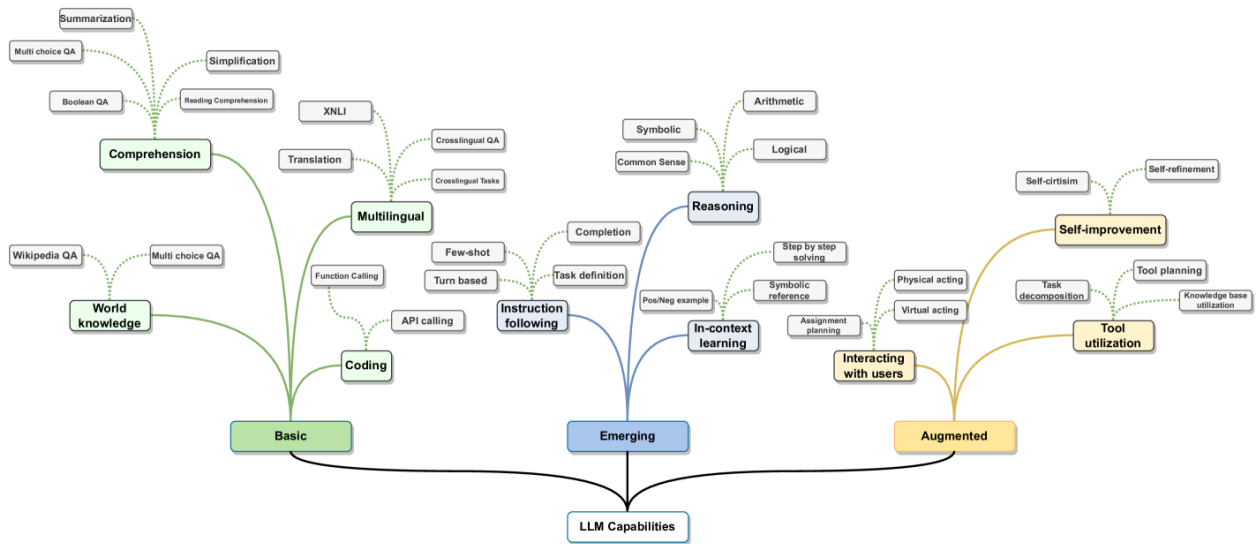


Figure II- 2: Capacités des LLM (Minaee et al., 2024)

Généralement les performances de grands modèles de langage sur diverses tâches peuvent être extrapolées sur la base des performances de modèles plus petits similaires. Cependant, les grands modèles subissent parfois un « déphasage discontinu » où le modèle acquiert soudainement des capacités substantielles non vues dans les modèles plus petits comme l'illustre la figure II-2 ci-dessus. Celles-ci sont connues sous le nom de « capacités émergentes » et ont fait l'objet d'études approfondies. Les chercheurs notent que de telles capacités « ne peuvent pas être prédites simplement en extrapolant les performances de modèles plus petits » (Naveed et al., 2024). Ces capacités sont découvertes plutôt que programmées ou conçues, dans certains cas seulement après le déploiement public du LLM. Des centaines de capacités émergentes ont été décrites. Les exemples incluent l'arithmétique en plusieurs étapes, la passation d'examens de niveau universitaire, l'identification du sens voulu d'un mot, l'incitation à la chaîne de pensée, le décodage de l'alphabet phonétique international, le décryptage des lettres d'un mot pas clair.

- Généralisation et adaptation aux différents domaines : Les LLM sont capables de généraliser leurs connaissances linguistiques à différents domaines et tâches, ce qui les rend polyvalents et adaptables. Cette capacité de généralisation est due à leur entraînement sur de vastes corpus de données textuelles diversifiés, qui leur permet d'apprendre des représentations linguistiques riches et adaptables (Brown et al., 2020).
- Capacité à comprendre et générer du langage naturel : Les LLM peuvent comprendre et générer du langage naturel de manière cohérente et pertinente, ce qui les rend aptes à interagir avec les humains et à effectuer des tâches complexes. Ils sont capables de traiter diverses tâches liées au langage naturel, telles que la compréhension de texte, la génération de texte et la traduction automatique (Devlin et al., 2019).

- Traitement des tâches de compréhension, de génération et de traduction : Les LLM sont capables de traiter diverses tâches liées au langage naturel, telles que la compréhension de texte, la génération de texte et la traduction automatique. Leur capacité à apprendre des représentations linguistiques riches et à capturer des dépendances linguistiques à long terme leur permet de s'adapter à ces différentes tâches et d'obtenir des performances élevées (Radford et al., s. d.).

II.2.5 Présentation des quelques LLM

II.2.5.1 Présentation de la famille de modèles Mistral

II.2.5.1.1 Présentation de Mistral 7B

Dans le domaine en rapide évolution du traitement du langage naturel (NLP), l'optimisation des performances des modèles implique souvent une augmentation de leur taille, ce qui peut engendrer des coûts computationnels élevés et une latence d'inférence accrue. Cela constitue un obstacle majeur à leur déploiement dans des scénarios pratiques (Ainslie et al., 2023). Dans ce contexte, Mistral 7B émerge comme un modèle capable de concilier haute performance et efficacité d'inférence, répondant ainsi à la nécessité d'une approche équilibrée.

Mistral 7B a démontré sa capacité à surpasser le précédent modèle de référence, Llama 2 13B, sur l'ensemble des benchmarks testés, ainsi qu'à devancer le modèle de 34B LLaMa dans des domaines tels que la génération de code et les mathématiques (Jiang et al., 2023). L'un des aspects novateurs de Mistral 7B est l'utilisation de mécanismes d'attention tels que l'attention à requête groupée (Grouped-Query Attention, GQA) et l'attention à fenêtre glissante (Sliding Window Attention, SWA). GQA permet d'accélérer la vitesse d'inférence tout en réduisant les besoins en mémoire lors du décodage, favorisant ainsi des tailles de lot plus importantes et un débit accru, des caractéristiques essentielles pour les applications en temps réel (Ainslie et al., 2023).

L'attention à fenêtre glissante, quant à elle, est conçue pour traiter des séquences plus longues de manière plus efficace, tout en réduisant le coût computationnel, ce qui atténue une limitation courante des modèles de langage (Ainslie et al., 2023). Ensemble, ces mécanismes contribuent à améliorer tant la performance que l'efficacité de Mistral 7B.

Mistral 7B est construit sur une architecture de transformateur et est doté de plusieurs paramètres clés. Cette architecture présente des modifications par rapport à des modèles antérieurs, optimisant ainsi l'utilisation de la mémoire et la vitesse d'inférence. Par exemple, l'utilisation d'un cache à tampon circulaire permet de maintenir une taille de cache fixe, ce qui limite l'augmentation de la mémoire cache tout en garantissant que la qualité du modèle est préservée (Ainslie et al., 2023).

En conclusion, Mistral 7B représente une avancée significative dans l'équilibre entre haute performance et efficacité des modèles de langage. Ce modèle est publié sous la licence Apache 2.0, accompagné d'une mise en œuvre de référence qui facilite son déploiement sur des plateformes locales ou cloud, tel que AWS (Amazon Web Services) ou GCP (Google Cloud Platform) (Jiang et al., 2023). Son adaptabilité et sa performance supérieure en font un candidat idéal pour des applications variées dans le domaine de l'IoT et du contrôle des circuits électriques programmables.

II.2.5.1.2 Présentation de Mistral 8X7B

Mistral 8X7B est un modèle de mixture éparsée d'experts (Sparse Mixture of Experts, SMOE) avec des poids ouverts, sous licence Apache 2.0. Il surpasse Llama 2 70B et GPT-3.5 dans la majorité des benchmarks, offrant une vitesse d'inférence plus rapide à faible taille de batch et un débit plus élevé à grande taille de batch (Jiang et al., 2024).

- **Architecture :** Mistral 8X7B utilise une architecture de transformer, spécifiquement un modèle uniquement décodeur où le bloc de feedforward sélectionne parmi un ensemble de huit groupes distincts de paramètres. À chaque couche, pour chaque token, un réseau de routage choisit deux de ces groupes (les "experts") pour traiter le token et combine leur sortie de manière additive. Cette technique permet d'augmenter le nombre de paramètres d'un modèle tout en contrôlant les coûts et la latence, car le modèle n'utilise qu'une fraction de l'ensemble total des paramètres par token (Jiang et al., 2024).
- **Pré-entraînement et Performance :** Mistral 8X7B est pré-entraîné avec des données multilingues en utilisant une taille de contexte de 32k tokens. Il égale ou dépasse la performance de Llama 2 70B et GPT-3.5 sur plusieurs benchmarks, démontrant des capacités supérieures en mathématiques, en génération de code et dans des tâches nécessitant une compréhension multilingue. Des expériences montrent que Mistral 8X7B peut récupérer avec succès des informations de sa fenêtre de contexte de 32k tokens, indépendamment de la longueur de la séquence et de l'emplacement de l'information dans la séquence (Jiang et al., 2024).
- **Fine-Tuning et Applications :** Mistral 8X7B – Instruct, un modèle de chat affiné pour suivre des instructions, utilise un affinement supervisé et l'optimisation de préférence directe. Sa performance dépasse notablement celle de GPT-3.5 Turbo, Claude-2.1, et Llama 2 70B dans des évaluations humaines. Mistral – Instruct montre également des biais réduits et un profil de sentiment plus équilibré dans des benchmarks comme BBQ et BOLD (Jiang et al., 2024).
- **Libération et Accessibilité :** Les modèles Mistral 8X7B et Mistral 8X7B – Instruct sont disponibles sous la licence Apache 2.0, gratuits pour un usage académique et commercial, assurant une large accessibilité et un potentiel pour des applications diverses. Pour permettre à la communauté d'exécuter Mistral avec une pile entièrement open-source, des modifications ont été soumises au projet VLLM, intégrant des kernels CUDA Megablocks

pour une inférence efficace. Skypilot permet également le déploiement des points de terminaison VLLM sur n'importe quelle instance dans le cloud (Mistral, 2024).

- Les paramètres de l'architecture de Mistral 8X7B sont résumés dans le tableau II-1 (Jiang et al., 2024) :

Tableau II- 1: Paramètres de l'architecture de Mistral 8X7B (Jiang et al., 2024)

Paramètre	Valeur
Dim	4096
n_layers	32
head_dim	128
hidden_dim	14336
n_heads	32
n_kv_heads	8
context_len	32768
vocab_size	32000
num_experts	8
top_k_experts	2

- MoE (Mixture of Experts) Layer : La couche MoE est une composante clé de Mistral 8X7B. Chaque vecteur d'entrée est assigné à deux des huit experts par un routeur, et la sortie de la couche est la somme pondérée des sorties des deux experts sélectionnés. Cette approche, similaire à l'architecture GShard, remplace tous les sous-blocs feed-forward (FFN) par des couches MoE, avec une stratégie de gating plus simple pour le deuxième expert assigné à chaque token (Jiang et al., 2024).
- Comparaison : Mistral 8X7B surpasse Llama 2 70B dans presque tous les benchmarks populaires tout en utilisant cinq fois moins de paramètres actifs pendant l'inférence. Les résultats détaillés montrent que Mistral excelle particulièrement en mathématiques et en

génération de code, démontrant une supériorité significative par rapport aux modèles Llama de même taille.(Jiang et al., 2024)

II.2.5.1.3 Présentation de Mistral Large

Mistral Large est un modèle de langage avancé développé par Mistral AI, conçu pour offrir des capacités exceptionnelles en génération de code, en mathématiques et en raisonnement. Ce modèle représente une avancée significative par rapport à ses prédécesseurs, notamment en termes de support multilingue et de fonctionnalités avancées de gestion des fonctions.

- **Caractéristiques Techniques :** Mistral Large 2, la dernière génération du modèle, dispose d'une fenêtre de contexte de 128k et supporte plusieurs langues, y compris le français, l'allemand, l'espagnol, l'italien, le portugais, l'arabe, l'hindi, le russe, le chinois, le japonais et le coréen. En outre, il supporte plus de 80 langages de programmation, tels que Python, Java, C, C++, JavaScript et Bash. Cette polyvalence linguistique et technique en fait un outil puissant pour diverses applications industrielles et académiques (*Mistral AI API | Mistral AI Large Language Models*, 2024).
- **Performance et Efficacité :** Mistral Large excelle dans des tâches de génération de texte, d'analyse de sentiments, de traduction automatique, de classification de texte et de complétion de code, surpassant même les plus grands modèles de GPT-4 dans plusieurs benchmarks clés. De plus, il utilise des techniques de compression pour réduire la latence et les coûts d'inférence, tout en maintenant des performances de pointe (*Mistral AI API | Mistral AI Large Language Models*, 2024).
- **Cas d'Usage :** Le modèle est particulièrement efficace dans des scénarios de chatbot, d'analyse de sentiments, de génération de code, de classification de texte et de complétion de texte. Son intégration dans des systèmes de recommandation, des moteurs de recherche, des plateformes de formation et d'évaluation, ainsi que des systèmes de contrôle de processus industriels, démontre sa capacité à fournir des solutions de pointe dans des environnements divers et complexes (*Mistral AI API | Mistral AI Large Language Models*, 2024).

II.2.5.1.4 Présentation de CodeStral

CodeStral est un modèle spécialisé dans la génération de code, conçu pour offrir des solutions optimales aux développeurs et aux ingénieurs travaillant sur des projets complexes. Sa capacité à comprendre des instructions en langage naturel et à les transformer en code fonctionnel fait de CodeStral un outil indispensable pour les équipes de développement logiciel et les ingénieurs de recherche.

- **Architecture :** Le modèle utilise une architecture de transformer, optimisée pour la génération de code. Il supporte plus de 50 langages de programmation, notamment Python, Java, C++, JavaScript, Ruby, et SQL. CodeStral est capable de générer des solutions de code complètes, de corriger les erreurs dans le code existant, et de fournir des suggestions

d'amélioration et d'optimisation (*Codestral: Hello, World! | Mistral AI | Frontier AI in your hands*, 2024)

- Performance : CodeStral a été testé sur plusieurs benchmarks de génération de code et a surpassé les modèles existants en termes de précision et de rapidité. Sa capacité à comprendre et à implémenter des concepts complexes de programmation en fait un outil puissant pour les développeurs travaillant sur des projets variés, allant de l'automatisation des tâches simples à la gestion de grands systèmes logiciels (*Codestral: Hello, World! | Mistral AI | Frontier AI in your hands*, 2024).
- Applications : Les applications de CodeStral incluent la génération de code pour des systèmes embarqués, le développement d'applications web et mobiles, l'automatisation de tâches de déploiement, et la création de scripts pour l'analyse de données. Son intégration dans des environnements de développement intégrés (IDE) permet aux développeurs de gagner du temps et d'améliorer la qualité de leur code (*Codestral: Hello, World! | Mistral AI | Frontier AI in your hands*, 2024).

II.2.5.2 Présentation de la famille de modèles GPT d'OpenAI

La famille de modèles GPT (Generative Pre-trained Transformer) développée par OpenAI représente une avancée significative dans le domaine de l'intelligence artificielle et du traitement du langage naturel (NLP). Chaque itération de ces modèles a été conçue pour améliorer les capacités de traitement, d'interaction et de compréhension des données multimodales, rendant ainsi l'interaction homme-machine plus fluide et naturelle (*Hello GPT-4o | OpenAI*, 2024).

II.2.5.2.1 Contexte et évolutions des modèles GPT

Les modèles GPT ont été initialement introduits avec GPT-1, qui a posé les bases de l'utilisation des réseaux de neurones pour générer du texte de manière autonome. Par la suite, les versions successives GPT-2, GPT-3, et enfin GPT-4 ont chacune amélioré les performances sur des tâches variées, notamment la compréhension du langage, la génération de texte cohérent et la réponse à des requêtes complexes. (OpenAI et al., 2024)

Avec la récente annonce de GPT-4o, OpenAI franchit un nouveau cap en intégrant des capacités de traitement multimodal, permettant au modèle de traiter des entrées textuelles, audio, visuelles et vidéo. Ce modèle est conçu pour interagir en temps réel avec un temps de réponse moyen de 320 millisecondes, comparable à celui des interactions humaines (*Hello GPT-4o*, 2024). Cela représente une avancée majeure, car il permet une interaction plus naturelle et dynamique entre l'utilisateur et la machine.

II.2.5.2.2 Caractéristiques de GPT-4o

Le modèle GPT-4o, où "o" signifie "omni", se distingue par sa capacité à accepter n'importe quelle combinaison de types d'entrée et à générer des sorties correspondantes, que ce soit du texte,

de l'audio ou des images. Ce modèle a été optimisé pour améliorer la compréhension du langage dans différentes langues, et il est également plus rapide et économique que ses prédécesseurs. Par exemple, il est signalé qu'il est 50 % moins cher en termes d'utilisation via l'API par rapport à GPT-4 (*Hello GPT-4o*, 2024).

L'architecture de GPT-4o a été conçue pour traiter tous les types d'entrées et de sorties au sein d'un même réseau de neurones, contrairement aux versions précédentes qui nécessitaient des pipelines distincts pour la transcription audio, le traitement du texte et la conversion en audio (*Hello GPT-4o*, 2024). Cela permet une meilleure conservation des informations contextuelles, notamment en ce qui concerne le ton, les interlocuteurs multiples et les bruits de fond.

II.2.5.2.3 Performances et évaluations

Sur des benchmarks traditionnels, GPT-4o a atteint des niveaux de performance similaires à ceux de GPT-4 Turbo en ce qui concerne le traitement du texte et la compréhension des langages. Cependant, il a établi de nouveaux records en matière de compréhension audio et visuelle. Des évaluations montrent que GPT-4o surpasse ses prédécesseurs dans des tâches telles que la traduction audio et la perception visuelle.

Les limitations identifiées lors des tests de GPT-4o incluent des risques liés à la sécurité et à l'éthique, comme l'amplification de biais dans les données de formation. OpenAI a mis en place des mécanismes de sécurité pour atténuer ces risques, notamment par des évaluations régulières et des tests de red teaming avec des experts externes (*Hello GPT-4o*, 2024).

II.2.6 Performances de quelques modèles LLM sur MMLU et HumanEval

L'évaluation des performances des Grands Modèles de Langage (LLM) est cruciale pour déterminer leur capacité à réaliser des tâches spécifiques. Dans le cadre de ce projet de recherche, deux benchmarks ont été sélectionnés pour évaluer les performances des différents LLMs utilisés : le Massive Multitask Language Understanding (MMLU) et le HumanEval.

II.1.6.1 Présentation des benchmarks MMLU et HumanEval

Le MMLU (Hendrycks et al., 2021) est un benchmark qui évalue la compréhension du langage à travers 57 tâches réparties dans des domaines variés tels que les sciences, les mathématiques, les sciences humaines et sociales. Il mesure la capacité des modèles à appliquer leurs connaissances à un large éventail de problèmes, allant des questions élémentaires aux questions de niveau professionnel.

Le HumanEval (M. Chen et al., 2021) est un benchmark spécifiquement conçu pour évaluer la capacité des LLMs à générer du code Python fonctionnel à partir de descriptions textuelles (docstrings). Il se compose de 164 problèmes de programmation accompagnés de tests unitaires permettant de vérifier automatiquement la validité du code généré.

Le choix de ces deux benchmarks s'explique par la nature même du projet de recherche, qui vise à contrôler des circuits électriques par le langage naturel.

II.2.6.2 Justification du choix des benchmarks

L'utilisation de HumanEval se justifie par la nécessité d'évaluer la capacité des LLMs à générer du code Python correct et fonctionnel. En effet, le projet repose sur la traduction d'instructions en langage naturel en code Python exécutable pour envoyer des commandes à l'Arduino UNO.

MMLU quant à lui, permet d'évaluer la capacité des LLMs à comprendre un langage naturel complexe et à mobiliser des connaissances dans différents domaines. Cette capacité est essentielle pour interpréter correctement les instructions de l'utilisateur et fournir un retour d'information pertinent.

II.2.6.3 Analyse des performances des LLMs sur MMLU et HumanEval

Le Tableau II-2 présente les performances de différents LLMs sur les benchmarks MMLU et HumanEval.

Tableau II- 2: Performances des quelques LLM sur MMLU et HumanEval(M. Chen et al., 2021; Hendrycks et al., 2021)

Nom du Modèle	Nombre de paramtres	Benchmark MMLU	Benchmark HumanEval
Mistral 7B	7B	62.5	40.2
Mistral Large 2	123B	84.0	89.0
Llama 3 .1 8B	8B	69.4	68.3
Llama 3 .1 70B	70B	83.6	80.5
Llama 3 .1 450B	450B	88.6	89.0
Mistral 8X7B	12B	79.9	75.6
Codestral	22B	-	81.1
GT-4o	-	88.7	90.2
GPT-4o mini	-	82.0	87.2

Gemini 1.5 pro	-	81.9	71.9
Gemma 2 9B	9B	71.3	40.2
Gemma 2 27B	27 B	75.2	51.8
Claude 3 Opus	-	86.8	84.9
Claude 3.5 Sonnet	-	88.3	92.0
Gemini Ultra 1.0	-	83.7	83.7
Mistral Nemo	12 B	68	60
Gemini 1.5 flash	-	78.9	77.2

Analyse :

- Corrélation entre nombre de paramètres et performances : On observe généralement une corrélation positive entre le nombre de paramètres d'un LLM et ses performances sur les deux benchmarks. Les modèles dotés d'un plus grand nombre de paramètres, tels que Llama 3.1 450B, GPT-4o et Claude 3.5 Sonnet, obtiennent des scores plus élevés, soulignant l'impact de la quantité de données d'entraînement sur la capacité des LLMs.
- Performances sur HumanEval : Les modèles spécifiquement conçus pour la génération de code, tels que Codestral, obtiennent des scores élevés sur HumanEval. Cependant, certains modèles généralistes, tels que GPT-4o et Claude 3.5 Sonnet, affichent également des performances remarquables, démontrant ainsi leur polyvalence.
- Importance du choix du modèle : Le choix du modèle le plus adapté dépendra des exigences spécifiques du projet.

Ces résultats préliminaires mettent en évidence la capacité des LLMs à appréhender le langage naturel et à générer du code, validant ainsi le bien-fondé de leur utilisation pour le contrôle de circuits électriques par le langage naturel.

II.3 Revue de la littérature sur le contrôle des circuits électriques

Dans cette section, nous approfondirons le concept de contrôle des circuits électriques, en explorant ses définitions, ses caractéristiques, ses méthodes et ses applications. Nous fournirons également des détails sur les différents types de contrôle électrique, les outils et les techniques utilisés pour assurer un fonctionnement efficace et fiable des systèmes électriques.

II.3.1 Définition et caractéristiques du contrôle électrique

II.3.1.1 Concept de base du contrôle électrique

Le contrôle électrique est une discipline qui se concentre sur la régulation et la commande des systèmes électriques et électromécaniques en utilisant des dispositifs électroniques, des capteurs et des actionneurs. Il est essentiel dans diverses applications, telles que l'industrie, l'énergie, les transports et la robotique (Craig, 2005). Le contrôle électrique permet de garantir que les systèmes fonctionnent de manière optimale, en ajustant les paramètres et en répondant aux changements de l'environnement.

II.3.1.2 Types de contrôle électrique

Il existe plusieurs types de contrôle électrique, chacun ayant ses propres caractéristiques et avantages. Les principaux types de contrôle électrique sont les suivants (Ogata, 2010) :

- Contrôle en boucle ouverte et en boucle fermée : Dans un système en boucle ouverte, la sortie n'affecte pas l'entrée, ce qui signifie que le système ne prend pas en compte les modifications apportées à la sortie pour ajuster l'entrée. En revanche, dans un système en boucle fermée, la sortie est comparée à l'entrée pour ajuster le système en conséquence.
- Contrôle analogique et numérique : Le contrôle analogique utilise des signaux continus pour réguler les systèmes, tandis que le contrôle numérique utilise des signaux discrets. Les systèmes de contrôle numérique sont généralement plus précis et polyvalents que les systèmes analogiques, car ils peuvent être programmés et adaptés à diverses applications (Franklin, 2010).

II.3.1.3 Caractéristiques du contrôle électrique

Les systèmes de contrôle électrique doivent posséder certaines caractéristiques pour assurer un fonctionnement efficace et fiable. Ces caractéristiques comprennent ((Dorf & Bishop, 2008) :

- Précision : Les systèmes de contrôle électrique doivent être capables de maintenir la sortie aussi proche que possible de la valeur souhaitée, en minimisant l'erreur entre les deux.
- Stabilité : Un système de contrôle électrique stable est un système qui, lorsqu'il est soumis à une perturbation, revient à son état d'équilibre sans oscillations excessives.

- Rapidité : Les systèmes de contrôle électrique doivent réagir rapidement aux changements de l'environnement et ajuster la sortie en conséquence.
- Robustesse : Les systèmes de contrôle électrique doivent être conçus pour fonctionner dans diverses conditions et être capables de s'adapter aux changements de l'environnement.
- Adaptabilité : Les systèmes de contrôle électrique doivent être capables de s'adapter aux variations des paramètres du système et aux changements de l'environnement.

II.3.2 Présentation des principaux outils et méthodes de contrôle électrique

Les outils et les méthodes utilisés dans le contrôle électrique sont variés et comprennent des composants matériels et logiciels. Parmi les principaux outils et méthodes, on peut citer (Sontag, 2013) :

II.3.2.1 Capteurs et actionneurs

Les capteurs et les actionneurs sont des éléments clés du contrôle électrique. Les capteurs mesurent les grandeurs physiques du système, telles que la température, la pression et la position, et convertissent ces mesures en signaux électriques qui peuvent être traités par le système de contrôle. Les actionneurs, quant à eux, agissent sur le système en fonction des signaux de commande provenant du contrôleur. La figure II-3 et la figure II-4 montre un exemple de capteur et d'actionneur utilisés dans un système de contrôle électrique.

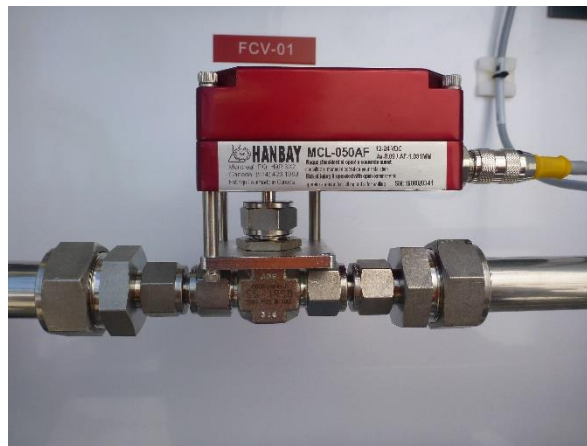


Figure II- 3: Actionnaire de vanne électrique (File, 2017)

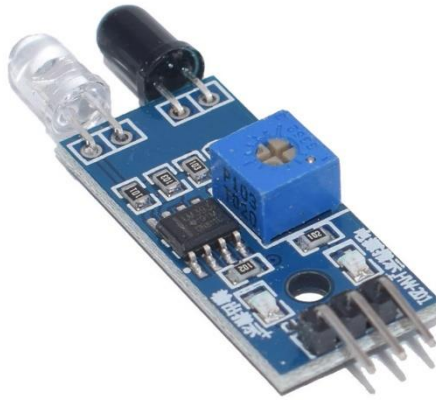


Figure II- 4: Capteur d'obstacle (File, 2017)

II.3.2.2 Contrôleurs et régulateurs

Les contrôleurs et les régulateurs sont utilisés pour ajuster les performances des systèmes électriques. Les contrôleurs les plus couramment utilisés sont les contrôleurs PID (Proportionnel, Intégral, Dérivé), qui ajustent la sortie du système en fonction de l'erreur entre la valeur souhaitée et la valeur réelle. Les régulateurs, quant à eux, maintiennent une variable du système à une valeur constante en ajustant automatiquement les paramètres du système. La figure II-4 illustre le principe de fonctionnement d'un contrôleur PID.

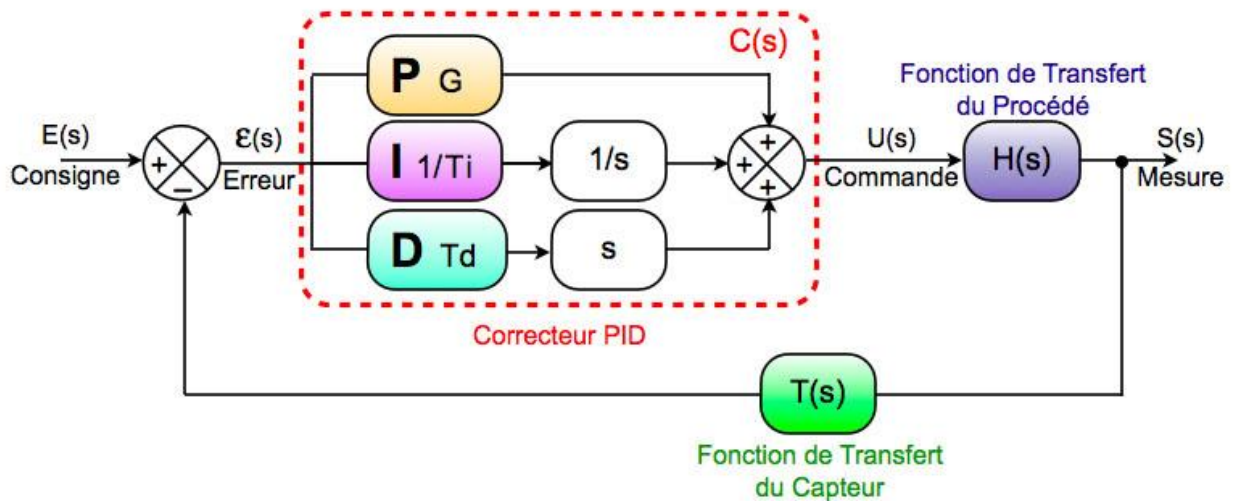


Figure II- 5: Principe de fonctionnement d'un contrôleur PID (« Régulateur PID », 2024)

II.3.2.3 Systèmes d'acquisition et de traitement de données

Les systèmes d'acquisition et de traitement de données sont essentiels pour collecter, analyser et traiter les informations provenant des capteurs et des actionneurs. Ces systèmes peuvent

être basés sur des microcontrôleurs, des FPGA (Field Programmable Gate Array) qui peuvent être programmer à plusieurs reprises après la fabrication ou des DSP (Digital Signal Processor) qui sont des processeurs optimiser pour exécuter des applications de traitement de signal numérique. Ces 2 systèmes permettent de traiter les données en temps réel et d'adapter le contrôle en conséquence.

II.3.3 Avantages et limites du contrôle électrique pour le langage naturel

L'utilisation du langage naturel dans le contrôle électrique présente à la fois des avantages et des limites. Parmi les principaux avantages, on peut citer :

- Interaction homme-machine améliorée : Le langage naturel permet une communication plus intuitive et plus facile entre les opérateurs humains et les systèmes électriques, ce qui facilite la programmation et la commande des systèmes.
- Efficacité accrue : Le contrôle électrique par langage naturel peut simplifier la programmation et la commande des systèmes, réduire les erreurs et les temps d'arrêt, et améliorer l'efficacité globale des systèmes.

Cependant, il existe également des limites et des défis associés à l'utilisation du langage naturel dans le contrôle électrique (Fakih et al., 2024):

- Ambiguïté et complexité : Le langage naturel est intrinsèquement ambigu et complexe, ce qui peut entraîner des erreurs et des malentendus dans les systèmes de contrôle électrique.
- Sécurité et fiabilité : Les systèmes de contrôle basés sur le langage naturel doivent garantir la sécurité et la fiabilité des opérations, ce qui peut être difficile à assurer en raison de l'ambiguïté et de la complexité du langage naturel.

II.3.4 Applications et tendances du contrôle électrique

Le contrôle électrique est utilisé dans de nombreuses applications et domaines, notamment (Craig, 2005; Cui et al., 2024) :

- Industrie : L'automatisation et le contrôle de processus industriels sont des applications clés du contrôle électrique. La robotique et les systèmes mécatroniques bénéficient également du contrôle électrique pour améliorer leurs performances et leur efficacité.
- Systèmes embarqués et IoT : L'intégration du contrôle électrique dans les dispositifs intelligents et connectés est une tendance croissante. Les systèmes embarqués et l'Internet des objets (IoT) utilisent le contrôle électrique pour améliorer la communication et la coordination entre les objets connectés.

Les tendances actuelles et futures du contrôle électrique incluent (Marr, 2019) :

- Intelligence artificielle et apprentissage automatique : L'IA et l'apprentissage automatique sont de plus en plus utilisés dans le contrôle électrique pour améliorer les performances et l'adaptabilité des systèmes.
- Contrôle électrique basé sur le cloud et les edge computing : Le contrôle électrique basé sur le cloud et les edge computing offrent des possibilités d'amélioration de l'efficacité et de la flexibilité des systèmes de contrôle. Elle consiste à traiter les données à la périphérie du réseau, près de la source des données, et ainsi elle minimise les besoins en bande passante entre les capteurs et les centres de traitements des données en entreprenant les analyses au plus près de données.
- Systèmes de contrôle électrique écoénergétiques et durables : Le développement de systèmes de contrôle électrique écoénergétiques et durables est un domaine de recherche important, visant à réduire la consommation d'énergie et l'impact environnemental des systèmes électriques.

II.4 Positionnement du travail par rapport aux travaux existants

Le travail proposé dans ce mémoire s'inscrit dans la continuité des recherches précédentes sur l'utilisation des LLM pour le contrôle des circuits électriques. Toutefois, il se distingue des travaux existants par plusieurs aspects, qui seront détaillés dans cette section.

II.4.1 Intégration de l'interaction en langage naturel pour le contrôle des circuits électriques

L'un des principaux apports de ce travail réside dans l'intégration de l'interaction en langage naturel pour le contrôle des circuits électriques. Alors que de nombreuses recherches se sont concentrées sur l'amélioration des performances des LLM pour diverses tâches liées au langage naturel, peu d'entre elles ont exploré leur utilisation pour le contrôle des circuits électriques en utilisant le langage naturel comme interface (Fakih et al., 2024). Le présent travail vise à combler cette lacune en proposant une méthode permettant d'utiliser les LLM pour contrôler les circuits électriques de manière interactive et dynamique à l'aide du langage naturel.

II.4.2 Utilisation d'un Arduino Uno pour la simulation des circuits électriques

Un autre aspect novateur de ce travail est l'utilisation d'un Arduino Uno pour la simulation des circuits électriques, dans le cadre d'une preuve de concept pour montrer les possibilités offertes par ces outils. Bien que l'Arduino Uno ait été largement utilisé dans divers projets de contrôle électrique, son intégration avec les LLM pour le contrôle interactif et dynamique des circuits électriques à l'aide du langage naturel n'a pas encore été explorée de manière approfondie. Ce travail présentera une méthode permettant d'utiliser l'Arduino Uno en conjonction avec les LLM

pour simuler et contrôler les circuits électriques en utilisant le langage naturel comme interface, de sorte que l'arduino Uno recoit les commandes venant du port série lorsque le code python généré par le LLM est exécuté.

II.4.3 Utilisation de la bibliothèque Open Interpreter pour l'interaction entre le langage naturel et les objets programmables

La bibliothèque Open Interpreter est un outil puissant pour l'interaction entre le langage naturel et les objets programmables. Cependant, son utilisation pour le contrôle des circuits électriques à l'aide des LLM n'a pas encore été étudiée en détail. Dans ce travail, nous présenterons une méthode permettant d'intégrer la bibliothèque Open Interpreter avec les LLM Open source et propriétaires pour faciliter l'interaction entre le langage naturel et les circuits électriques contrôlés par l'Arduino Uno.

II.4.4 Comparaison des performances des différents LLM pour le contrôle des circuits électriques

Enfin, ce travail se distinguera des recherches existantes en comparant les performances des différents LLM pour le contrôle des circuits électriques. Bien que de nombreux modèles de LLM aient été proposés, peu d'études ont évalué leur efficacité pour le contrôle des circuits électriques à l'aide du langage naturel et une interface interactive de génération de code. Dans ce mémoire, nous comparerons les performances de divers LLM, tels que les modèles GPT d'Openai, les modèles Gemini de Google, les modèles Mistral de Mistral AI, et d'autres modèles pertinents, pour déterminer leur efficacité respective dans le contrôle des circuits électriques en utilisant le langage naturel comme interface.

En résumé, le travail proposé dans ce mémoire se positionne par rapport aux travaux existants en intégrant l'interaction en langage naturel pour le contrôle des circuits électriques, en utilisant un kit Arduino Uno pour la simulation des circuits électriques, en exploitant la bibliothèque Open Interpreter pour faciliter l'interaction entre le langage naturel et les objets programmables, et en comparant les performances des différents LLM pour le contrôle des circuits électriques. Ce travail vise à apporter une contribution significative à la littérature en proposant une méthode novatrice et efficace pour le contrôle interactif et dynamique des circuits électriques à l'aide du langage naturel.

II.5 Conclusion

Ce deuxième chapitre a permis de dresser un état de l'art exhaustif des deux piliers sur lesquels repose ce mémoire : les LLM (Large Language Models) et le contrôle des circuits électriques. La première partie du chapitre s'est attelée à démystifier les LLM, en explorant leur définition, leur architecture révolutionnaire basée sur les Transformers, ainsi que leurs capacités

et limitations. Cette analyse approfondie, appuyée par des exemples concrets tels que les modèles GPT d'OpenAI, Gemini de Google et Mistral de Mistral AI, a permis de mettre en lumière le potentiel disruptif des LLM pour le traitement du langage naturel. L'analyse comparative de leurs performances sur des benchmarks reconnus comme MMLU et HumanEval a confirmé leur aptitude à appréhender la complexité du langage naturel et à générer du code fonctionnel, validant ainsi leur pertinence pour le contrôle des circuits électriques par le langage naturel. La deuxième partie du chapitre a offert une plongée au cœur du contrôle des circuits électriques, en examinant ses principes fondamentaux, ses différentes typologies, ses outils et méthodes, ainsi que ses avantages et limitations. L'analyse croisée de ces deux domaines a permis de positionner clairement le travail de ce mémoire et de souligner son caractère novateur. En effet, l'intégration de l'interaction en langage naturel au cœur du contrôle des circuits électriques, via l'utilisation d'un kit Arduino Uno et de la bibliothèque Open Interpreter, représente une approche originale et prometteuse. La comparaison systématique des performances des différents LLM pour cette tâche spécifique constituera un apport significatif à la littérature scientifique. Ce chapitre a donc permis de poser les fondations théoriques et technologiques indispensables à la compréhension de ce travail de recherche. Le chapitre suivant s'appuiera sur ces bases solides pour présenter en détail la méthodologie adoptée pour la conception et l'implémentation du système de contrôle interactif et dynamique des circuits électriques par le langage naturel.

CHAPITRE III. IMPLEMENTATION D'UN SYSTEME DE CONTROLE ÉLECTRIQUE PAR LANGAGE NATUREL

III.1 Introduction

Ce troisième chapitre, intitulé "Implémentation d'un Système de Contrôle Électrique par Langage Naturel", traduit la théorie en pratique en concevant et implémentant un système permettant de contrôler des circuits électriques par le langage naturel, grâce aux grands modèles de langage (LLM). Dans un premier temps, nous présenterons les outils et données utilisés : le kit Arduino Uno pour la simulation des circuits, la bibliothèque Open Interpreter pour l'interaction entre langage naturel et objets programmables, ainsi que les LLM sélectionnés, en soulignant leurs caractéristiques et performances. Ensuite, nous exposerons les hypothèses de conception qui guident le développement du système, garantissant sa cohérence avec les objectifs visés. Enfin, nous détaillerons la méthodologie de conception et d'implémentation, rigoureuse et systématique, s'appuyant sur les bonnes pratiques d'ingénierie. Elle comprendra l'analyse des besoins et contraintes, la définition des spécifications, l'élaboration d'un modèle conceptuel, l'implémentation, la validation et les tests. Ce chapitre vise à fournir une description complète du processus de développement, permettant de comprendre les choix techniques, les défis et les solutions apportées. Il pose ainsi les bases pour l'analyse des résultats et la discussion des perspectives, qui seront abordées ultérieurement.

III.2 Présentation des outils et des données utilisées

Dans le cadre de ce projet, plusieurs outils et ressources seront utilisés pour développer et tester un système de contrôle électrique par langage naturel. Cette section détaille les outils sélectionnés, leur rôle et leur mise en œuvre dans le projet, ainsi que les données utilisées pour évaluer le système.

Le fait d'utiliser un LLM pour générer ce code ajoute une couche de flexibilité et de personnalisation, permettant aux utilisateurs d'adapter les comportements des dispositifs électromécaniques en fonction de leurs besoins spécifiques, sans nécessiter de programmation manuelle complexe.

III.2.1 L'Arduino Uno pour le prototypage et la simulation des circuits électriques

III.2.1.1 Description de l'Arduino Uno et de ses composants

L'Arduino Uno est une carte de développement open-source basée sur le microcontrôleur ATmega328P de Microchip, initialement commercialisée en 2010 par Arduino.cc. La carte est équipée de 14 broches d'entrée/sortie (E/S) numériques, dont six capables de sortie PWM, et de six broches d'E/S analogiques. Elle est programmable via l'environnement de développement intégré (IDE) Arduino, en utilisant un câble USB (Bus Série Universel) de type B. L'alimentation peut être fournie par un câble USB ou un connecteur cylindrique acceptant des tensions comprises entre 7 et 20 volts. De plus, l'ATmega328 sur la carte est préprogrammé avec un chargeur d'amorçage qui permet le téléchargement de nouveau code sans l'utilisation d'un programmeur matériel externe. (Margolis, s. d.)

La conception matérielle de référence est distribuée sous une licence Creative Commons Attribution Share-Alike 2.5 et est disponible sur le site Web d'Arduino. Le nom « Uno », signifiant « un » en italien, a été choisi pour marquer une refonte majeure du matériel et du logiciel Arduino, succédant à la version Duemilanove. (*UNO R3 | Arduino Documentation*, s. d.)

III.2.1.2 Justification du choix de l'Arduino Uno pour le contrôle électrique

Le choix de l'Arduino Uno pour le contrôle électrique dans ce projet repose sur plusieurs critères déterminants. Premièrement, sa simplicité d'utilisation et sa large compatibilité avec divers capteurs et actionneurs électriques en font un outil idéal pour les projets de prototypage rapide et de simulation des circuits électriques de toutes les manières acceptables grâce à ses broches PWM. Deuxièmement, l'IDE (Environnement de Développement Intégré) Arduino, basé sur le langage C/C++, offre une interface de programmation accessible et flexible, facilitant l'intégration avec d'autres bibliothèques et logiciels. Enfin, la vaste communauté d'utilisateurs et la disponibilité de ressources en ligne (tutoriels, forums, documentation) renforcent son utilité pour les ingénieurs et les techniciens cherchant à développer et tester rapidement leurs concepts.

III.2.1.3 Méthode d'installation et de configuration de l'Arduino Uno

L'installation de l'Arduino Uno commence par la connexion de la carte à un ordinateur via un câble USB. Ensuite, il est nécessaire d'installer l'IDE Arduino, téléchargeable gratuitement depuis le site officiel d'Arduino. Une fois l'IDE installé, la configuration du kit implique la sélection du type de carte (Arduino Uno) et du port série approprié dans l'IDE. La première programmation de la carte peut être réalisée avec un sketch de base, tel que le classique "Blink", pour tester les fonctionnalités de base de la carte et vérifier que l'installation et la configuration ont été correctement effectuées. (*Programming Arduino*, 2012)

La carte Arduino Uno est munie de diverses broches et connecteurs qui permettent l'interfaçage avec différents composants électriques et électroniques. Par exemple, les broches VIN et 5V permettent d'alimenter la carte via des sources externes, tandis que les broches GND servent de terre commune pour les circuits connectés. Les broches numériques et analogiques peuvent être configurées pour lire ou envoyer des signaux, mais dans ce projet nous n'aurons qu'à utiliser les broches PWM, qui sont les seules broches donnant une plus large manœuvre de contrôle des circuits électriques, ce qui permet un contrôle précis des composants connectés, tels que des LED, des moteurs et des capteurs. *(UNO R3 | Arduino Documentation, s. d.)*

L'utilisation de l'Arduino Uno dans le cadre de ce projet de mémoire s'avère particulièrement appropriée en raison de ses caractéristiques techniques robustes et de sa capacité à s'intégrer dans un environnement de prototypage rapide et flexible. Ces qualités facilitent le développement de systèmes de contrôle interactifs et dynamiques, répondant ainsi aux exigences du projet de démonstration de la preuve de concept pour un contrôle de circuits électriques par le langage naturel et les grands modèles de langage (LLM).

III.2.2 La bibliothèque Open Interpreter pour l'interaction entre le langage naturel et les objets programmables

III.2.2.1 Présentation de la bibliothèque Open Interpreter et de ses fonctionnalités

Open Interpreter est une bibliothèque logicielle open-source qui permet de créer des interfaces entre le langage naturel et les objets programmables. Elle offre des fonctionnalités de traitement automatique du langage naturel (NLP), notamment l'analyse syntaxique, l'analyse sémantique et la reconnaissance d'entités nommées, ainsi que des méthodes d'interprétation et d'exécution de commandes. Open Interpreter permet aux modèles de langage de grande taille (LLM) d'exécuter du code (Python, JavaScript, Shell, etc.) localement. Il propose une interface en langage naturel pour les capacités générales de l'ordinateur, facilitant ainsi la création et l'édition de médias, le contrôle de navigateurs Web, et l'analyse de grands ensembles de données. *(GitHub - OpenInterpreter/open-interpreter: A natural language interface for computers, s. d.)*

III.2.2.2 Méthode d'intégration de la bibliothèque Open Interpreter avec l'Arduino Uno

Pour intégrer la bibliothèque Open Interpreter avec l'Arduino Uno, il est nécessaire de développer une interface de communication entre l'ordinateur et la carte Arduino en utilisant des protocoles de communication série tels que UART (Émetteur-récepteur Asynchrone Universel) ou USB. Cette interface permet d'envoyer des commandes en langage naturel depuis l'ordinateur vers

la carte Arduino et de recevoir des réponses et des données de capteurs en retour. L'installation de la bibliothèque se fait via le terminal avec la commande ``pip install open-interpreter``. Une fois installée, Open Interpreter peut être exécuté en lançant la commande ``interpreter``, ce qui ouvre une session interactive où des instructions en langage naturel peuvent être traduites en commandes exécutables par l'Arduino par l'intermédiaire de l'exécution de code.(*GitHub - OpenInterpreter/open-interpreter: A natural language interface for computers, s. d.*)

III.2.2.1 Développement d'un protocole de communication entre le langage naturel et les objets programmables

Le protocole de communication développé doit définir les formats de messages et les séquences d'échange entre l'utilisateur, la bibliothèque Open Interpreter et les objets programmables. Ce protocole doit assurer une interopérabilité maximale entre les différents composants du système, tout en garantissant la fiabilité et la sécurité des échanges. Les messages envoyés à l'Arduino peuvent inclure des instructions pour contrôler des composants tels que des LED, des servomoteurs, des capteurs ultrasoniques et de température, ainsi que des moteurs pas à pas. Le code Python généré par le LLM est exécuté par Open Interpreter, qui envoie ensuite les commandes via le port série USB à l'Arduino. Le code Arduino, quant à lui, reste constant et assure l'exécution des commandes reçues, en renvoyant des informations au LLM pour confirmer l'exécution correcte des instructions ou pour générer de nouvelles commandes en cas d'échec.

L'intégration d'Open Interpreter permet de combiner la puissance des modèles de langage avec la flexibilité de l'environnement de développement local, surmontant ainsi les limitations des interpréteurs de code hébergés, tels que l'absence d'accès à Internet, les restrictions de taille de fichier et les limites de temps d'exécution. Cette combinaison offre une interface intuitive et conviviale pour les ingénieurs et les techniciens, leur permettant de surveiller et de contrôler en temps réel des dispositifs électromécaniques programmables, en utilisant des commandes en langage naturel.

III.2.3 Utilisation de la bibliothèque pySerial pour la communication série

III.2.3.1 Aperçu de pySerial et justification de son choix

PySerial est une bibliothèque Python qui permet de gérer facilement les communications via des ports série. Dans le cadre de ce projet, l'utilisation de pySerial est cruciale pour établir la communication entre le programme Python, généré par les grands modèles de langage (LLM), et la carte Arduino Uno via le port série USB. Cette bibliothèque fournit une interface uniforme et flexible qui fonctionne sur différentes plateformes telles que Windows, Linux, macOS, et d'autres systèmes compatibles POSIX, garantissant ainsi une compatibilité étendue pour des environnements de développement variés.(*Welcome to pySerial's documentation — pySerial 3.4 documentation, s. d.*)

L'objectif de ce projet étant de démontrer un contrôle interactif et dynamique de circuits électriques via des commandes en langage naturel, pySerial joue un rôle clé en facilitant l'envoi et la réception des instructions entre les composants matériels et le code Python exécuté. Le choix de pySerial est donc justifié par sa compatibilité, sa flexibilité, et sa simplicité d'intégration dans des environnements de développement basés sur Python.

III.2.3.2 Fonctionnalités et avantages techniques de pySerial

Les fonctionnalités principales de pySerial incluent(*Welcome to pySerial's documentation — pySerial 3.4 documentation, s. d.*) :

- Support multiplateforme : la bibliothèque est compatible avec les systèmes d'exploitation les plus courants, garantissant une large adoption.
- Gestion des paramètres du port : elle permet de configurer des éléments comme le débit en bauds, les bits de données, la parité, et les bits d'arrêt directement via des propriétés Python, facilitant le réglage fin des communications.
- Prise en charge des flux de contrôle : pySerial prend en charge les protocoles RTS/CTS et Xon/Xoff pour gérer les flux de données, un avantage pour des communications série complexes.
- Fonctionnalités de lecture-écriture : elle offre une API simple pour lire et écrire des données en binaire, avec des méthodes comme `readline` pour simplifier les échanges.

Ces avantages rendent pySerial particulièrement bien adaptée pour une interface avec des microcontrôleurs comme l'Arduino, où la gestion des ports série est cruciale pour l'exécution fluide des commandes et la réception des données de retour.

III.2.3.3 Intégration de pySerial dans le projet

Dans ce projet, pySerial est utilisée pour établir la communication entre le code Python généré par le LLM et le Arduino Uno via le port série USB.

Le processus suit les étapes suivantes :

- Initialisation du port série : Le programme Python, après génération par le LLM, ouvre une connexion série à un débit en bauds spécifié, communément fixé à 9600 bauds pour l'Arduino Uno.
- Envoi des commandes : Une fois le port série ouvert, le programme Python utilise les méthodes `write()` de pySerial pour envoyer les commandes au microcontrôleur. Ces commandes sont préalablement formatées sous forme binaire ou textuelle en fonction des exigences du circuit.

- Réception et analyse des données : L' Arduino Uno, après exécution des commandes, envoie un retour d'information à travers le même port série, que pySerial capture via la méthode ``read()`` ou ``readline()``. Ces retours sont ensuite traités par le LLM pour vérifier si les commandes ont été exécutées correctement.
- Réaction aux erreurs : En cas d'échec d'exécution, le LLM génère un nouveau code Python qui est à nouveau transmis à l'Arduino. Cette boucle continue jusqu'à ce que les actions spécifiées par l'utilisateur soient réalisées correctement.

III.2.3.4 Méthode d'installation et de configuration de pySerial

L'installation de pySerial se fait simplement via la commande `'pip install pyserial'` dans un environnement Python

Une fois installé, il est essentiel de configurer correctement le port série pour correspondre aux paramètres de l' Arduino Uno. Typiquement, ces paramètres incluent le débit en bauds, le nombre de bits de données (généralement 8), les bits d'arrêt (1 ou 2), et le contrôle de la parité (aucune, pour la plupart des cas).

III.2.3.5 Impact de pySerial sur l'interactivité et la dynamique du projet

L'intégration de pySerial améliore l'interactivité du système en offrant une communication en temps réel entre l'utilisateur, le modèle de langage, et les circuits électriques commandés. Elle permet à l'utilisateur de contrôler directement des composants tels que des LED, des servomoteurs, et les autres dispositifs ayant des circuits électriques par des commandes en langage naturel, tout en garantissant un retour d'information immédiat sur l'exécution des actions. Cette boucle de rétroaction dynamique est essentielle pour atteindre l'objectif du projet, qui est de démontrer un contrôle électrique interactif basé sur le langage naturel.

Grâce à cette communication efficace et fiable entre le LLM, le code Python et l'Arduino via pySerial, il devient possible de créer un environnement flexible et adaptable, réduisant les barrières techniques pour les ingénieurs et techniciens qui souhaitent contrôler des systèmes électromécaniques complexes de manière intuitive.

III.2.3 Analyse Comparative des LLM pour des Applications de contrôle électrique

III.2.3.1 Critères de sélection des LLM pour le contrôle électrique

Les modèles de langage (LLM) utilisés pour le contrôle électrique doivent répondre à des critères stricts pour garantir une performance optimale dans des environnements industriels et de recherche. Ces critères incluent :

- Compréhension et interprétation des commandes en langage naturel : Les LLM doivent démontrer une capacité élevée à comprendre et interpréter des commandes formulées en langage naturel afin de permettre une interaction fluide et intuitive avec les systèmes électromécaniques.
- Rapidité d'exécution : La latence doit être minimale pour assurer des réponses rapides aux commandes, ce qui est crucial pour les applications en temps réel.
- Précision et fiabilité : Les modèles doivent fournir des réponses précises et fiables pour éviter des erreurs potentielles dans le contrôle des circuits électriques.
- Compatibilité avec la bibliothèque Open Interpreter et l'environnement Arduino : Les LLM doivent être compatibles avec les outils et bibliothèques utilisés dans ce projet, notamment Open Interpreter et Arduino, pour faciliter l'intégration et le déploiement.

III.2.3.2 Présentation des LLM sélectionnés et de leurs caractéristiques techniques

Pour ce projet, plusieurs LLM ont été sélectionnés en raison de leurs performances, leurs disponibilités et de leurs caractéristiques techniques spécifiques pour faire cette preuve de concept. Les performances des modèles choisis sur le benchmarks MMLU et HumanEval sont illustré sur la figure III-1.

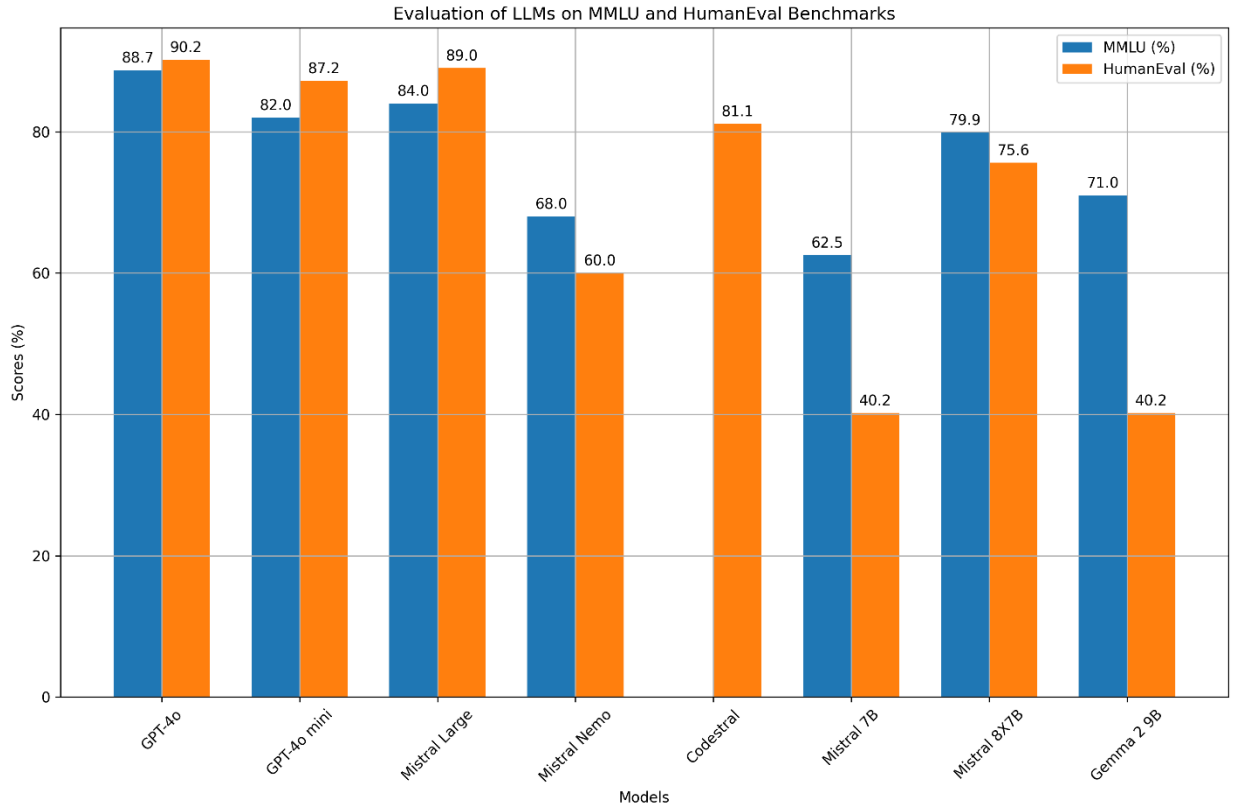


Figure III- 1: Performances des LLM selectionner sur les benchmarks HumanEval et MMLU

GPT-4o(*Hello GPT-4o*, 2024) :

- Performances sur les benchmarks : MMLU 88.7, HumanEval 90.2
- Capacité à comprendre des commandes complexes et à générer des codes précis rapidement.

GPT-4o mini (*Hello GPT-4o* / *OpenAI*, 2024):

- Performances sur les benchmarks : MMLU 82.0, HumanEval 87.2
- Version optimisée pour une exécution rapide avec une légère réduction de la précision. Ce modèle fait partie des modèles qui ont le meilleurs chiffres performances/coûts

Mistral Large(*Mistral AI API* / *Mistral AI Large Language Models*, 2024) :

- Performances sur les benchmarks : MMLU 84.0, HumanEval 89

- Capacité à comprendre des commandes complexes et à générer des codes précis rapidement.

Mistral Nemo (AI, 2024b):

- Nombre de paramètres : 12 milliards
- Performances sur les benchmarks : MMLU 68, HumanEval 60
- Modèle léger et efficace pour des tâches nécessitant moins de ressources computationnelles.

Codestral(AI, 2024a) :

- Nombre de paramètres : 22 milliards
- Performances sur les benchmarks : HumanEval 81.1
- Modèle léger et efficace spécialisé en code, pour des tâches nécessitant moins de ressources computationnelles. Il n'est pas nécessaire d'évaluer ce modèle sur le MMLU car il est spécialisé en code.

Mistral 7B(Jiang et al., 2023) :

- Nombre de paramètres : 7 milliards
- Performances sur les benchmarks : MMLU 62.5, HumanEval 40.2
- Modèle léger et efficace pour des tâches nécessitant moins de ressources computationnelles.

Mistral 8X7B (Jiang et al., 2024) :

- Nombre de paramètres : 12 milliards
- Performances sur les benchmarks : MMLU 79.9, HumanEval 75.6
- Bon compromis entre précision et efficacité, idéal pour des applications plus intensives.

Gemma 2 9B (Gemma Team et al., 2024) :

- Nombre de paramètres : 9 milliards
- Performances sur les benchmarks : MMLU 71, HumanEval 40.2
- Modèle léger et efficace pour des tâches nécessitant moins de ressources computationnelles.

III.2.3.3 Méthode de comparaison des performances des LLM sélectionnés dans le cadre du contrôle électrique

Pour évaluer les performances des LLM sélectionnés, une série d'expériences contrôlées sera mise en œuvre. Ces expériences utiliseront des jeux de données et des métriques d'évaluation adaptés aux tâches spécifiques de contrôle électrique par langage naturel. Les étapes suivantes seront suivies :

III.2.3.3.1 Conception des expériences

- Définition des scénarios de contrôle électrique typiques, incluant des commandes pour le contrôle de LEDs, de servomoteurs, et d'autres actionneurs ayant des circuits électriques.
- Création de jeux de données de commandes en langage naturel représentant ces scénarios.

III.2.3.3.2 Métriques d'évaluation :

- Précision : Mesure de la conformité des actions exécutées par rapport aux commandes données.
- Rapidité : Temps de réponse entre la réception de la commande et l'exécution de l'action.
- Fiabilité : Taux de réussite des commandes exécutées correctement sans erreurs.

III.2.3.3.3 Exécution des tests

- Chaque LLM sera testé en exécutant un ensemble de commandes standardisées.
- Les performances seront enregistrées et comparées en utilisant les métriques définies.

III.2.3.3.4 Analyse des résultats

- Les résultats des tests seront analysés pour déterminer les modèles les plus performants.
- Les forces et faiblesses de chaque modèle seront identifiées et documentées.

Cette méthodologie permettra une évaluation rigoureuse et objective des capacités des LLM à contrôler des circuits électriques via des commandes en langage naturel, fournissant ainsi une base solide pour leur utilisation dans des applications industrielles et de recherche.

III.3 Hypothèses et méthodologie de Conception et d'Implémentation d'un système de Contrôle Électrique Par Langage Naturel

Le développement d'un système de contrôle électrique par langage naturel implique plusieurs étapes clés, allant de la conception initiale à l'implémentation et la validation du système.

Cette section décrira en détail chacune de ces étapes, en mettant l'accent sur la méthodologie scientifique et les bonnes pratiques d'ingénierie.

III.3.1 Hypothèses de Conception

Pour concevoir un système de contrôle électrique par langage naturel, plusieurs hypothèses fondamentales doivent être posées :

Hypothèse de Compréhension du Langage Naturel :

- Les grands modèles de langage (LLM) sont capables de comprendre et d'interpréter correctement les commandes en langage naturel, en tenant compte des nuances syntaxiques et sémantiques.
- Les LLM peuvent générer du code exécutable en réponse aux commandes de l'utilisateur, permettant ainsi le contrôle des circuits électriques.

Hypothèse de Fiabilité des LLM :

- Les LLM utilisés (GPT4o, GPT4o mini, Mistral large, Codestral, etc...) sont suffisamment robustes et précis pour générer des commandes correctes et exécutables.
- Les LLM peuvent s'adapter et corriger les erreurs en temps réel, en cas de mauvaise interprétation des commandes.

Hypothèse de Compatibilité Technique :

- Les LLM sont compatibles avec les interfaces de communication utilisées (programme Python, Open Interpreter) et peuvent interagir efficacement avec les cartes de contrôle (Arduino UNO).
- Les composants électriques (LED, servomoteurs, capteurs, moteurs pas à pas) sont compatibles avec les commandes générées par les LLM et peuvent être contrôlés de manière fiable.

Hypothèse d'Adaptabilité : Le système sera suffisamment flexible pour s'adapter à diverses configurations de circuits électriques et à différents contextes d'utilisation.

III.3. 2 Méthodologie de Conception et d'Implémentation

III.2.2.1 Conception du système de contrôle

- Analyse des besoins et des contraintes du système de contrôle : L'analyse des besoins consiste à identifier les fonctionnalités et les performances attendues du système de

contrôle, en se basant sur les attentes des utilisateurs et les spécifications du projet. Les contraintes à prendre en compte incluent les limitations techniques, économiques, et environnementales, ainsi que les exigences en matière de sécurité et de fiabilité.

- Définition des spécifications fonctionnelles et techniques du système : À partir de l'analyse des besoins et des contraintes, il est nécessaire de définir les spécifications fonctionnelles et techniques du système de contrôle. Les spécifications fonctionnelles décrivent les différentes fonctionnalités du système et leurs interactions, tandis que les spécifications techniques détaillent les caractéristiques et les performances attendues du système en termes de précision, de rapidité, de consommation d'énergie, etc.
- Élaboration d'un modèle conceptuel du système de contrôle : Le modèle conceptuel du système de contrôle est une représentation abstraite du système, qui permet de décrire son architecture, ses composants et leurs interactions. Ce modèle doit être suffisamment détaillé pour guider l'implémentation du système, tout en étant assez général pour faciliter la compréhension et la communication entre les différents acteurs du projet.

III.3.2.2 Implémentation du système de contrôle

- Développement d'algorithmes de traitement du langage naturel pour la compréhension et l'interprétation des commandes : L'implémentation du système de contrôle requiert le développement d'algorithmes de traitement du langage naturel, qui permettent de comprendre et d'interpréter les commandes écrites en langage naturel. Ces algorithmes doivent être capables d'analyser la syntaxe et la sémantique des phrases, d'identifier les intentions de l'utilisateur, et de traduire ces intentions en actions concrètes sur les circuits électriques. Dans notre cas, ces algorithmes sont des LLM déjà entraînés et ayant subi un réglage fin.
- Intégration des LLM sélectionnés dans le système de contrôle : Les LLM sélectionnés pour le contrôle électrique doivent être intégrés au système de contrôle, en veillant à assurer une communication efficace et fiable entre les différents composants du système. Cette intégration implique la mise en place d'interfaces de communication, la configuration des paramètres de fonctionnement des LLM, et la vérification de leur compatibilité avec les autres composants du système.
- Programmation des cartes de contrôle et des actionneurs électriques : La programmation des cartes de contrôle et des actionneurs électriques consiste à écrire le code nécessaire pour piloter les circuits électriques en fonction des commandes interprétées par les algorithmes de traitement du langage naturel. Cette programmation doit prendre en compte les spécificités techniques des cartes de contrôle et des actionneurs, ainsi que les contraintes de sécurité et de fiabilité du système.

III.3.2.3 Validation et tests du système de contrôle

- Développement d'un plan de test pour évaluer les performances du système de contrôle : Le plan de test doit décrire les différentes étapes de la validation et des tests du système de contrôle, en précisant les objectifs, les méthodes, et les critères d'évaluation pour chaque étape. Ce plan doit également définir les scénarios de test à mettre en œuvre, en fonction des spécifications fonctionnelles et techniques du système.
- Réalisation de tests unitaires et d'intégration pour vérifier le bon fonctionnement du système : Les tests unitaires et d'intégration visent à vérifier le bon fonctionnement de chaque composant du système de contrôle, ainsi que leur interaction et leur coordination. Ces tests doivent être réalisés dans des conditions contrôlées, en utilisant des jeux de données et des scénarios de test adaptés.
- Évaluation de la précision, de la fiabilité et de l'efficacité du système de contrôle : L'évaluation des performances du système de contrôle implique la mesure de sa précision, de sa fiabilité, et de son efficacité dans différentes situations de contrôle. Ces mesures doivent être comparées aux spécifications techniques du système, afin de vérifier si les objectifs de performance sont atteints et d'identifier d'éventuelles pistes d'amélioration.

En suivant cette méthodologie, le développement d'un système de contrôle électrique par langage naturel sera mené de manière rigoureuse et systématique, en veillant à respecter les bonnes pratiques d'ingénierie et à garantir la qualité et la fiabilité du système. Cette approche permettra également de préparer efficacement la partie expérimentale du projet, en fournissant une base solide pour la conception et la mise en œuvre des expériences à réaliser.

III.4 Conclusion

Ce chapitre a marqué une étape importante dans ce mémoire, en passant de la théorie à la pratique. La conception et l'implémentation d'un système de contrôle électrique innovant, basé sur l'interaction en langage naturel grâce aux LLM, ont été détaillées. Le choix s'est porté sur l'Arduino Uno pour sa simplicité et sa polyvalence, couplé à la bibliothèque Open Interpreter pour une interaction intuitive. Une sélection rigoureuse de LLM, incluant GPT-4o et Mistral, a été opérée en fonction de leurs performances sur des benchmarks et de leur adéquation au projet. Les hypothèses de conception, réalistes quant aux capacités des LLM, ont été explicitées, garantissant la cohérence de la démarche. La méthodologie de développement, structurée en trois phases (conception, implémentation, validation), a assuré la robustesse du système. Ce chapitre, en présentant un système fonctionnel de contrôle par langage naturel, ouvre la voie à des analyses approfondies des résultats et à l'exploration de pistes d'amélioration, sujets qui seront abordés dans le chapitre suivant.

CHAPITRE IV. ANALYSE ET INTERPRETATION DES RESULTATS

IV.1 Introduction

Ce quatrième chapitre se penche sur l'analyse et l'interprétation des résultats obtenus lors de l'implémentation du système de contrôle des circuits électriques par le langage naturel. Il s'agit de confronter les performances des différents modèles LLM testés aux hypothèses de conception formulées précédemment. L'analyse, à la fois qualitative et quantitative, se basera sur des métriques rigoureuses (taux de succès, erreur temporelle, coût d'inférence). Le chapitre détaillera l'architecture et l'implémentation du système, en mettant en évidence les interactions entre les composants matériels et logiciels. Les limites inhérentes à l'approche, les difficultés techniques rencontrées, et les solutions mises en œuvre seront également abordées. Enfin, ce chapitre explorera les perspectives d'avenir de cette technologie prometteuse, en proposant des pistes d'amélioration et des axes de recherche pour optimiser les performances et la fiabilité du système.

IV.2 Évaluation des Hypothèses de Conception

Avant d'analyser en profondeur les performances des modèles LLM dans le contrôle interactif des circuits électriques, il est crucial de confronter les hypothèses de conception, formulées au Chapitre III (point III.2.1), aux résultats expérimentaux obtenus. Cette démarche de validation permet de juger de la pertinence des hypothèses initiales et d'identifier les ajustements nécessaires à l'approche.

IV.2.1 Hypothèse 1: Compréhension du Langage Naturel

- Énoncé de l'hypothèse: Les grands modèles de langage (LLM) sont capables de comprendre et d'interpréter correctement les commandes formulées en langage naturel, en saisissant les nuances syntaxiques et sémantiques.
- Résultats obtenus et discussion: Le Tableau IV-1 présente les taux de succès des différents LLM testés sur un jeu de 25 instructions de contrôle. Les taux de succès élevés, notamment pour GPT-4o (96%) et GPT-4o mini (100%), suggèrent une aptitude à interpréter correctement les instructions en langage naturel et à les traduire en structures de code Python appropriées (boucles, conditions, appels de fonctions).

Cependant, l'analyse du code généré révèle des limites dans la gestion des nuances temporelles précises et dans le choix des structures de code optimales. Par exemple, parfois certains modèles ont eu recours à une limitation de la durée d'une action au lieu d'une boucle infinie lorsque cette dernière était requise.

Ces observations soulèvent la question de la robustesse de la compréhension du langage naturel par les LLM. Si la traduction syntaxique semble généralement réussie, la compréhension sémantique, notamment des nuances temporelles, peut être améliorée. (Voir Tableau IV-1)

IV.2.2 Hypothèse 2: Fiabilité des LLM

- Énoncé de l'hypothèse: Les LLM utilisés sont suffisamment robustes et précis pour générer des commandes correctes et exécutables. De plus, les LLM peuvent s'adapter et corriger les erreurs en temps réel en cas de mauvaise interprétation des commandes.
- Résultats obtenus et discussion: Bien que les taux de succès soient globalement élevés, l'analyse des erreurs absolues moyennes (MAE) sur le temps (Tableau IV-1) révèle une variabilité significative dans la précision temporelle des modèles. L'observation du système en fonctionnement a également mis en évidence des difficultés d'adaptation et de correction d'erreurs en temps réel.

L'hypothèse de fiabilité des LLM est donc partiellement confirmée. La génération de code fonctionnel est possible, mais des limitations persistent en termes de précision temporelle et de capacité d'adaptation aux erreurs d'interprétation. Ces limitations sont à mettre en lien avec la complexité inhérente à la traduction du langage naturel, souvent imprécis, en un code précis et exempt d'erreurs. (Voir Tableau IV-1)

IV.2.3 Hypothèse 3: Compatibilité Technique

- Énoncé de l'hypothèse: Les LLM sont compatibles avec les interfaces de communication utilisées (programme Python, Open Interpreter) et peuvent interagir efficacement avec les cartes de contrôle (Arduino UNO).
- Résultats obtenus et discussion: L'implémentation du système a démontré la faisabilité de la communication entre les LLM et l'Arduino UNO via un programme Python et la bibliothèque PySerial pour la communication série. L'intégration d'Open Interpreter s'est avérée plus problématique en raison d'incompatibilités avec certaines API des modèles LLM.

La compatibilité technique entre les LLM et l'Arduino UNO est donc validée via un programme Python dédié. La difficulté rencontrée avec Open Interpreter souligne la nécessité d'adapter les interfaces de communication pour une meilleure intégration des outils et des modèles LLM.

IV.2.4 Hypothèse 4: Adaptabilité

- Énoncé de l'hypothèse: Le système sera suffisamment flexible pour s'adapter à diverses configurations de circuits électriques et à différents contextes d'utilisation.

- Résultats obtenus et discussion: Le système a été testé avec succès sur un nombre limité de configurations de circuits impliquant des LEDs et un servomoteur (cf. Annexe A - Liste des Instructions). L'adaptation à de nouvelles configurations ne nécessite pas de modification manuelle du programme Python pour prendre en compte les changements de câblage et de composants.

L'adaptabilité du système est donc vérifiée à petite échelle. Des efforts supplémentaires sont nécessaires pour améliorer la flexibilité du système et permettre une adaptation automatique à de nouvelles configurations de circuits.

IV.3 Analyse Quantitative des Performances des Modèles de Langage (LLM)

Cette section est dédiée à l'analyse quantitative des performances des différents modèles de langage (LLM) testés pour le contrôle interactif des circuits électriques. L'objectif est de mesurer et de comparer de manière objective la capacité des modèles à interpréter les instructions en langage naturel et à générer un code Python fonctionnel pour exécuter les actions spécifiées sur le circuit. Cette évaluation permet de comparer objectivement les performances de notre jeu de données d'instructions avec les benchmarks utilisés, tels que HumanEval et MMLU, comme illustré dans la figure IV-1.

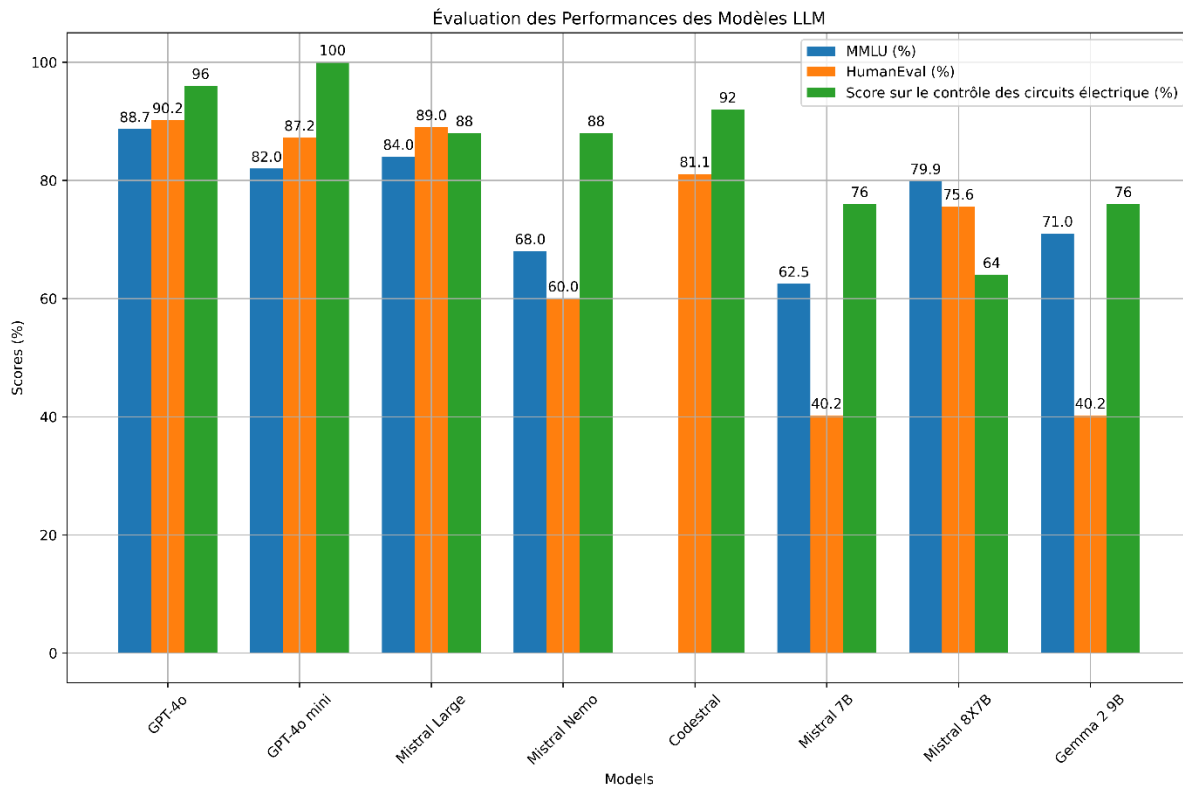


Figure IV- 1: Performances des modèles Evaluer sur notre jeu d'instructions comparer à HUMANEval et MMLU

IV.3.1 Présentation des Métriques d'Évaluation

Trois métriques principales ont été retenues pour évaluer les performances des LLM sur le contrôle dynamique des circuits électriques :

- Taux de succès (%): Mesure le pourcentage d'instructions du jeu de données pour lesquelles le code généré par le LLM a permis d'exécuter correctement l'action spécifiée sur le circuit électrique, sans erreur.
- Erreur absolue moyenne (MAE) sur le temps (en secondes): Représente la moyenne des différences absolues entre le temps d'exécution effectif du code généré par le LLM et le temps spécifié dans l'instruction pour chaque action. Cette mesure, calculée uniquement sur les instructions réussies, évalue la précision temporelle du contrôle.
- Coût d'inférence (\$/M tokens): Applicable uniquement aux modèles LLM commerciaux ayant une API payante, cette métrique mesure le coût financier lié à l'utilisation du modèle pour générer du code, exprimé en dollars par million de tokens.

IV.3.2 Présentation du Jeu d'Instructions de Contrôle Dynamique

Un jeu de 25 instructions de contrôle dynamique, formulées en langage naturel, a été conçu pour évaluer les LLM (cf. Annexe A). Ces instructions visent à contrôler des composants électroniques (LEDs et un servomoteur) connectés à une carte Arduino UNO R3, simulant ainsi un système programmable. Les instructions couvrent une gamme de complexité, allant de commandes simples ("Allumer la LED 1") à des séquences d'actions plus complexes impliquant la synchronisation, la variation temporelle et la gestion d'intervalles.

IV.3.3 Résultats Quantitatifs et Comparaison des Modèles

Le Tableau IV-1 synthétise les résultats obtenus par les différents LLM testés, selon les métriques définies précédemment.

Tableau IV- 1: Performances des Modèles LLM sur notre Jeu d'Instructions

Modèle	Taux de Succès (%)	MAE (secondes)	Coût d'Inférence (\$/M tokens)
GPT-4o	96	4.5	20
GPT-4o mini	100	10.7	0.75
Mistral Large	88	19.9	16
Mistral Nemo	88	20.1	-

Codestral	92	6.4	-
Mistral 8X7B	76	16.7	-
Mistral 7B v2	64	14.6	-
Gemma 2 9B	76	18.5	-

Analyse quantitative:

- Taux de succès : GPT-4o et GPT-4o mini excellent avec des taux de succès quasi-parfaits (96% et 100% respectivement). Codestral (92%) confirme également sa performance dans la génération de code. Les autres modèles présentent des taux de succès plus variables, allant de 88% (Mistral Large et Nemo) à 64% (Mistral 7B v2).
- MAE temporelle : GPT-4o se distingue avec une MAE temporelle très faible (4.5 secondes), suggérant une bonne gestion des contraintes temporelles. Les autres modèles présentent des MAE plus élevées, indiquant une moindre précision dans la gestion du timing des actions sur le circuit.
- Coût d'inférence : Parmi les modèles payants, GPT-4o mini offre le meilleur compromis entre performance et coût avec un taux de succès parfait et un coût d'inférence de 0.75 \$/M tokens.

IV.4 Analyse Qualitative des Performances

IV.4.1 Remarques Générales

L'analyse qualitative des performances des LLM dans le contrôle des circuits électriques par langage naturel a révélé des observations intéressantes concernant le fonctionnement du système et les comportements des différents modèles. Globalement, les LLM ont démontré une capacité prometteuse à traduire des instructions en langage naturel en code Python fonctionnel, permettant ainsi de piloter des circuits électriques connectés à une carte Arduino Uno.

IV.4.2 Analyse des Difficultés Rencontrées par les Modèles

Malgré des performances encourageantes, l'analyse qualitative a également mis en lumière certaines difficultés rencontrées par les LLM lors de l'interprétation des instructions et de la génération de code.

IV.4.2.1 Gestion des Nuances Temporelles

La gestion des nuances temporelles s'est avérée être un défi pour certains modèles. Par exemple, l'instruction "Allumer la LED 1 pendant 5 secondes puis l'éteindre" a été correctement interprétée par la plupart des modèles. Cependant, des variations ont été observées dans la précision de la durée d'allumage, certains modèles introduisant une marge d'erreur significative. Cette observation suggère que la compréhension fine des durées et des intervalles temporels par les LLM peut encore être améliorée.

Il a été remarqué que GPT-4o utilise davantage de marqueurs temporels précis pour respecter les durées spécifiées dans les instructions, ce qui pourrait expliquer sa meilleure performance en termes d'erreur absolue moyenne (MAE) sur le temps. Par exemple, pour l'instruction "Faire varier la luminosité de la LED 2 de 0% à 100% sur 10 secondes", GPT-4o a généré un code utilisant la fonction `'time.sleep()'` avec des intervalles de temps calculés précisément pour chaque niveau de luminosité, tandis que d'autres modèles ont opté pour des approximations moins précises.

IV.4.2.2 Interprétation des Boucles Infinies

Un autre défi rencontré concerne l'interprétation des boucles infinies. Les instructions exigeant une action continue, telle que "Faire clignoter la LED 3 indéfiniment", ont parfois été mal interprétées par certains modèles, notamment les plus petits. Au lieu de générer une boucle infinie, certains modèles ont préféré générer un code avec une durée limitée, par exemple "Faire clignoter la LED 3 pendant 10 minutes". Cette observation souligne la difficulté des LLM à appréhender le concept d'infini dans un contexte de contrôle temporel.

Le Tableau IV-2 illustre avec des exemples concrets des instructions exigeant des boucles infinies.

Tableau IV- 2: Instructions de Contrôle Dynamique Exigeant des Boucles Infinies

N°	Instructions de Contrôle Dynamique
6.	Synchroniser le mouvement du servo moteur et l'allumage de la LED 1, le servo allant de 0 à 180 degrés tandis que la LED passe de 0% à 100% de façon synchrone inverse au servo par pas de 50 (Device: [4, 1], Action: [1, 0], Param: [[0, 180], [0, 255]], Time: 10s)
15.	Synchroniser l'allumage de la LED 3 avec le mouvement du servo moteur allant de 0 à 180 degrés. La LED passe de 0% à 100% en synchronisation inverse avec le servo. (Device: [4, 3], Action: [1, 0], Param: [[0, 180], [0, 255]], Time: 10s)

22.	Synchroniser l'allumage de la LED 3 avec le mouvement du servo moteur de 0 à 180 degrés toutes les 4 secondes. La LED passe de 0% à 100% de luminosité. (Device: [4, 3], Action: [1, 0], Param: [[0, 180], [0, 255]], Time: 4s)
-----	---

IV.4.2.3 Synchronisation des Actions

Certains modèles, surtout les plus petits, ont également eu du mal à gérer les instructions impliquant la synchronisation de plusieurs actions sur des durées précises. Par exemple, l'instruction "Allumer la LED 1 et la LED 2 simultanément pendant 3 secondes puis les éteindre" a parfois généré un code où les LEDs s'allumaient avec un léger décalage temporel, ou avec des durées d'allumage différentes.

IV.4.3 Impact du Prompt Engineering

L'analyse qualitative a également mis en évidence l'importance du prompt engineering, c'est-à-dire la manière de formuler les instructions initiales données au modèle LLM, sur la qualité du code généré. Il a été observé que la précision du prompt joue un rôle crucial dans la capacité du LLM à interpréter correctement les nuances des instructions.

Par exemple, la fonction `'time.sleep(0.1)'`, ajoutée au prompt système pour donner le temps au matériel de réagir, a parfois induit des erreurs dans le timing des actions. Certains modèles ont généré un code qui durait plus longtemps que prévu en raison de l'accumulation des pauses `'time.sleep(0.1)'`. Cependant, certains modèles, notamment GPT-4o, ont réussi à contourner ce problème en calculant précisément le temps nécessaire pour chaque action, démontrant une meilleure compréhension du contexte temporel.

De plus, modifier le prompt système en insistant sur la précision des délais a permis de réduire l'erreur de l'écart temporel sur les instructions. Par exemple, ajouter la phrase "Il est crucial de respecter précisément les délais spécifiés" au prompt a conduit les modèles à générer un code plus précis en termes de timing. Cette observation souligne l'importance d'un prompt clair et précis pour guider le LLM vers une meilleure compréhension des exigences de l'utilisateur.

IV.4.4 Précision du Langage Naturel

Enfin, l'analyse qualitative a confirmé l'importance d'un langage naturel précis et non ambigu pour formuler les instructions de contrôle. Lorsque les instructions étaient trop vagues ou incomplètes, les modèles pouvaient générer un code incorrect ou inapproprié. Par exemple, l'instruction "Faire tourner le servomoteur" est ambiguë, car elle ne précise ni la direction ni l'angle de rotation. Dans ce cas, les modèles ont généré des codes différents, certains faisant tourner le servomoteur dans un sens, d'autres dans l'autre.

IV.5 Architecture et Implémentation du Système

IV.5.1 Diagramme du Système

La Figure IV-2 illustre l'architecture du système de contrôle interactif des circuits électriques par langage naturel. Ce diagramme schématique met en évidence les interactions entre les différents composants du système.

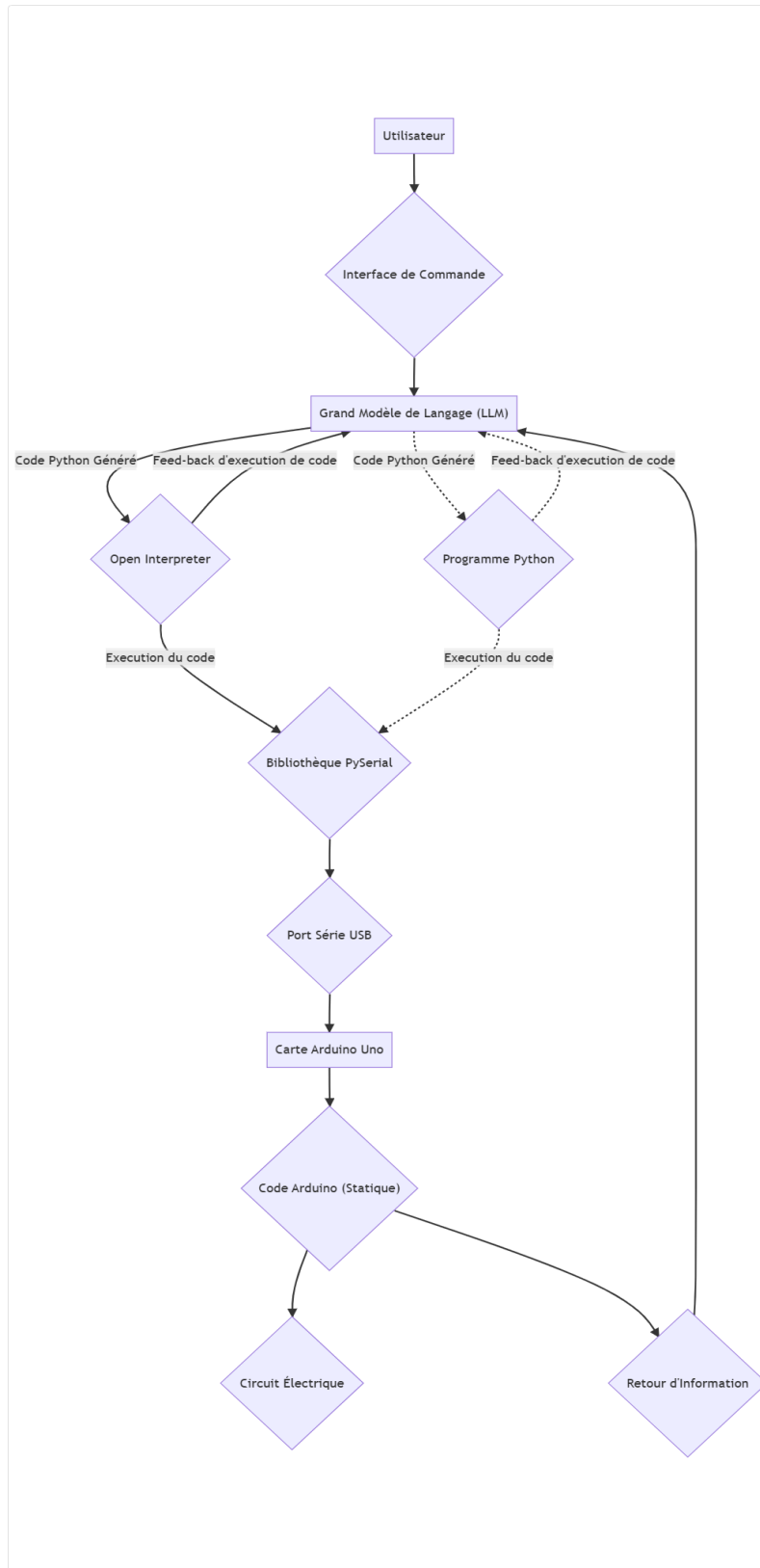


Figure IV- 2: Diagramme Schématique du Système

IV.5.1.1 Fonctionnement du Système

Le processus de contrôle débute avec l'utilisateur qui fournit une instruction en langage naturel via l'interface de commande. Le modèle LLM sélectionné reçoit cette instruction et la traite pour générer un code Python correspondant. Ce code est ensuite exécuté par l'interpréteur Python, soit directement, soit via la bibliothèque Open Interpreter si le modèle LLM le nécessite.

Le code Python généré contient des commandes destinées à la carte Arduino Uno, transmises via le port série USB grâce à la bibliothèque PySerial. Ces commandes sont formatées sous forme de matrices de contrôle, comme illustré dans le Tableau IV-3.

Tableau IV- 3: Exemple des matrices de commande serie

Commande Série	Explication
1,0,128	LED 1, action 0 (luminosité), valeur 128
4,1,90	Servomoteur, action 1 (rotation), angle 90
2,0,255	LED 2, action 0 (luminosité), valeur 255

L'Arduino Uno, programmé avec un code statique, reçoit et interprète ces matrices de commande pour piloter les circuits électriques connectés. Les ports PWM (Pulse Width Modulation) de l'Arduino sont utilisés pour contrôler les LEDs et le servomoteur, offrant une flexibilité accrue pour la gestion de la luminosité et de la position.

L'Arduino, après avoir exécuté les commandes, envoie un retour d'information au LLM via le port série. Ce retour confirme l'exécution des commandes ou signale d'éventuelles erreurs. Si les commandes ne sont pas exécutées correctement, le LLM génère un nouveau code Python, et la boucle se répète jusqu'à ce que l'instruction de l'utilisateur soit réalisée avec succès.

IV.5.2 Description Détaillée des Composants Matériels et Logiciels

IV.5.2.1 Composants Matériels

Le système de contrôle s'appuie sur un ensemble de composants matériels pour interagir avec le monde physique et exécuter les commandes générées par les LLM.

- Carte Arduino Uno R3 : Cœur du système de contrôle, la carte Arduino Uno (déjà présentée au Chapitre III.2.1) est responsable de l'exécution des commandes reçues du programme Python et du pilotage des composants électroniques connectés.
- LEDs : Des LEDs de différentes couleurs ont été utilisées pour visualiser les actions de contrôle, permettant de valider le fonctionnement du système et d'observer les résultats des commandes.

- Servomoteur : Un servomoteur a été intégré au circuit pour tester le contrôle de la position angulaire via les sorties PWM de l'Arduino.
- Plaque d'Essai : Une plaque d'essai a été utilisée pour faciliter les connexions physiques entre l'Arduino Uno et les composants électroniques, simplifiant ainsi le prototypage et l'expérimentation.
- Câbles de Connexion : Des câbles coaxiaux et flexibles de connexion pour plaque d'essai ont été utilisés pour assurer les connexions électriques entre les différents composants du circuit.
- Alimentation : Une alimentation 5V a été utilisée pour alimenter l'ensemble du circuit, assurant un fonctionnement stable de l'Arduino et des composants électroniques.

IV.5.2.2 Composants Logiciels et Environnement de Développement

La mise en œuvre du système de contrôle repose sur un environnement de développement logiciel robuste et des bibliothèques spécialisées pour la communication et le traitement du langage naturel.

- IDE Arduino: L'environnement de développement intégré (IDE) Arduino, présenté au Chapitre III.2.1, a été utilisé pour programmer la carte Arduino Uno avec le code statique nécessaire à la réception et à l'exécution des matrices de commande.
- Python: Le langage de programmation Python a été choisi pour développer le programme qui interagit avec les LLM, génère le code Python, et gère la communication série avec l'Arduino Uno. L'environnement Anaconda Python a été utilisé pour faciliter la gestion des dépendances et des bibliothèques Python.
- Bibliothèque PySerial: La bibliothèque PySerial, détaillée au Chapitre III.2.3, a été utilisée pour établir la communication série entre le programme Python et l'Arduino Uno via le port USB.
- Bibliothèque Open Interpreter: La bibliothèque Open Interpreter, présentée au point III.2.2, a été utilisée pour faciliter l'interaction entre certains modèles LLM et le programme Python, permettant l'exécution de code Python localement.
- Visual Studio Code: Le logiciel Visual Studio Code, avec l'extension Jupyter Notebook, a été utilisé comme environnement de développement pour le programme Python. L'utilisation de Jupyter Notebook a permis de structurer le code, de documenter les étapes de développement, et de visualiser les résultats de manière interactive.
- API Hugging Face: L'accès à l'API de Hugging Face a permis d'utiliser et d'évaluer des modèles LLM open-source, offrant une variété de choix pour comparer les performances.

- Chat Mistral: L'accès à l'interface Chat Mistral a permis d'utiliser les modèles LLM développés par Mistral AI, complétant ainsi la gamme de modèles disponibles pour l'évaluation.
- AI Studio de Google: L'accès à la plateforme AI Studio de Google a permis d'utiliser les modèles LLM développés par Google, enrichissant davantage la comparaison des performances.
- Ordinateur Lenovo T440: Un ordinateur portable Lenovo T440 (CPU: 2.49 GHz, RAM: 8Go DDR3 1600 MHz, GPU: 2Go) a été utilisé comme plateforme de développement et d'exécution du système de contrôle.

IV.5.3 Description des Interfaces de Communication

Le système de contrôle s'appuie sur une chaîne d'interfaces de communication pour assurer le dialogue entre l'utilisateur, le modèle LLM, le programme Python, et l'Arduino Uno.

IV.5.3.1 Communication Langage Naturel - LLM

L'utilisateur interagit avec le système en formulant des instructions en langage naturel. Ces instructions sont transmises au LLM via une interface de commande. Le processus de conversion de l'instruction en un format compréhensible par le LLM dépend du modèle utilisé. Certains modèles, notamment les plus récents, ont la capacité de traiter directement le langage naturel sans nécessiter de prétraitement spécifique.

IV.5.3.2 Communication LLM - Code Python

Le LLM, après avoir interprété l'instruction, génère un code Python correspondant. Ce processus de génération de code s'appuie sur l'apprentissage du modèle sur un vaste corpus de données textuelles, incluant des exemples de code Python. La qualité du code généré dépend de la capacité du LLM à comprendre l'instruction et à traduire les concepts exprimés en langage naturel en structures de code Python appropriées.

IV.5.3.3 Communication Python - Arduino

La communication entre le programme Python et l'Arduino Uno s'effectue via le port série USB, gérée par la bibliothèque PySerial. Le code Python envoie des matrices de commande à l'Arduino, comme illustré dans le Tableau IV-3. Ces matrices contiennent les informations nécessaires pour piloter les composants électroniques: l'identifiant du composant, le type d'action à réaliser (par exemple, luminosité pour une LED, rotation pour un servomoteur), et la valeur associée à l'action.

IV.5.3.4 Communication Arduino - Composants

L'Arduino Uno, après avoir reçu la matrice de commande, utilise ses broches digitales et PWM pour piloter les composants électroniques connectés. Le code Arduino statique associe chaque identifiant de composant à une broche spécifique et traduit les actions et les valeurs reçues en signaux de contrôle appropriés. Par exemple, pour contrôler la luminosité d'une LED connectée à une broche PWM, l'Arduino envoie un signal PWM dont le rapport cyclique est proportionnel à la valeur de luminosité spécifiée dans la matrice de commande.

IV.5.4 Caractéristiques des Modèles LLM

Les modèles LLM actuels se distinguent par leur capacité à comprendre divers types de langage, allant du langage naturel, souvent ambigu, aux langages mathématiques et informatiques plus formels. Cette polyvalence est un atout majeur pour le contrôle des circuits électriques, car elle permet d'utiliser des instructions en langage naturel, plus intuitives pour les utilisateurs non-programmeurs, tout en garantissant une traduction précise en commandes exécutables par l'Arduino.

IV.6 Limites, Difficultés Rencontrées, et Solutions

IV.6.1 Limites Intrinsèques à l'Approche

Le développement d'un système de contrôle des circuits électriques par langage naturel soulève des défis inhérents à la nature même du langage et à la fiabilité des modèles de langage (LLM) utilisés.

IV.6.1.1 Complexité du Langage Naturel

La complexité et l'ambiguïté inhérentes au langage naturel constituent un défi majeur pour l'interprétation précise des instructions par les LLM. Bien que ces modèles aient fait des progrès significatifs dans la compréhension du langage, des erreurs d'interprétation peuvent survenir, notamment lorsque les instructions contiennent des nuances subtiles, des expressions idiomatiques, ou un manque de précision dans les détails techniques.

Par exemple, une instruction telle que "Faire clignoter la LED rapidement" peut être interprétée différemment selon le modèle LLM, car la notion de "rapidité" est subjective et dépend du contexte. De même, une instruction imprécise comme "Ajuster la position du servomoteur" nécessite des informations supplémentaires pour être traduite en code Python fonctionnel. Le modèle LLM devra interpréter l'intention de l'utilisateur, ce qui peut conduire à une interprétation qui ne reflète pas l'intention de l'utilisateur, si l'instruction n'est pas suffisamment explicite.

IV.6.1.2 Fiabilité et Robustesse des Modèles LLM

Malgré des performances remarquables dans de nombreuses tâches, les LLM actuels ne sont pas infaillibles. Ils peuvent générer du code incorrect, incomplet ou inapproprié, ce qui peut entraîner des dysfonctionnements dans le système de contrôle des circuits électriques.

Comme observé lors de l'analyse qualitative des résultats, certains modèles ont eu du mal à interpréter correctement les instructions impliquant des boucles infinies, des synchronisations complexes, ou des délais précis. De plus, les LLM peuvent être sensibles aux erreurs de formulation dans les instructions en langage naturel. Une instruction ambiguë ou mal formulée peut conduire à la génération d'un code Python erroné, impactant ainsi le fonctionnement du système.

IV.6.2 Difficultés Techniques et Solutions

Le développement du système a nécessité de surmonter plusieurs difficultés techniques liées à l'intégration des LLM, de l'Arduino Uno, et des bibliothèques logicielles utilisées.

IV.6.2.1 Difficultés liées à Open Interpreter

L'utilisation de la bibliothèque Open Interpreter, destinée à faciliter l'interaction entre certains modèles LLM et l'exécution locale de code Python, s'est heurtée à des limitations. Des incompatibilités avec les API de certains modèles LLM ont été constatées, empêchant une intégration directe.

Pour contourner cette difficulté, une interface Python personnalisée a été développée, permettant de gérer les interactions entre les LLM et l'Arduino Uno de manière plus flexible et adaptable aux différents modèles testés. Cette solution a permis de surmonter les limitations d'Open Interpreter et de garantir une meilleure interopérabilité entre les composants du système.

IV.6.2.2 Difficultés liées à l'Arduino Uno

Les limitations matérielles de l'Arduino Uno, notamment sa mémoire et sa puissance de calcul limitées, ont imposé des contraintes sur la complexité des instructions et du code Python généré. Des instructions impliquant des traitements lourds ou des synchronisations très précises pouvaient engendrer des ralentissements ou des erreurs d'exécution sur l'Arduino.

Pour pallier ces limitations, des stratégies de simplification du code ont été adoptées. Par exemple, le nombre de boucles imbriquées a été réduit, et des approximations ont été utilisées pour gérer les délais dans certains cas. De plus, la sélection du prompt système fourni au LLM a pris en compte ces limitations, en privilégiant un prompt système pouvant guider les modèles et faire en sorte qu'il soit capables de générer du code Python concis et efficace, adapté aux ressources de l'Arduino Uno.

IV.6.2.3 Problèmes de Communication Série

La communication série entre le programme Python et l'Arduino Uno, gérée par la bibliothèque PySerial, a fonctionné de manière fiable dans la plupart des cas. Cependant, quelques erreurs de communication ont été rencontrées, comme dans certains cas le glissement inattendu du servomoteur lorsqu'il est commandé. Mais cela nécessite une étude plus approfondi pour en connaître la cause.

IV.6.3 Contraintes de Ressources et de Temps

Le développement du système a été contraint par des limitations de ressources et de temps, impactant la portée de l'évaluation et l'automatisation des tests.

IV.6.3.1 Impact sur le Jeu d'Instructions

La conception du jeu d'instructions de contrôle dynamique a été influencée par les contraintes de temps et de ressources disponibles pour l'expérimentation. Il n'a pas été possible de tester les LLM sur un très large éventail de configurations de circuits électriques et de scénarios de contrôle complexes.

Le jeu d'instructions s'est concentré sur un nombre limité de composants électroniques (LEDs et un servomoteur), et les instructions ont été conçues pour être relativement concises et simples à implémenter sur l'Arduino Uno. Une exploration plus approfondie de la capacité des LLM à gérer des circuits plus complexes et des instructions plus nuancées nécessiterait davantage de temps et de ressources.

IV.6.3.2 Impact sur le Choix des Modèles LLM

Le choix des modèles LLM à évaluer a été contraint par le budget alloué au projet. L'utilisation de modèles LLM commerciaux performants, tels que GPT-4o, implique des coûts d'inférence liés à l'utilisation de leur API. Il n'a donc pas été possible d'évaluer tous les modèles LLM disponibles sur le marché. La sélection des modèles s'est concentrée sur un ensemble représentatif de modèles open-source et commerciaux, permettant de comparer les performances et le coût d'utilisation.

IV.6.3.3 Impact sur l'Automatisation des Tests

L'évaluation des performances des LLM et l'analyse des résultats ont été réalisées de manière manuelle en raison du manque de temps et de ressources pour développer un processus entièrement automatisé. L'automatisation des tests nécessiterait de créer un environnement de test simulé, de développer des scripts d'évaluation, et d'intégrer des outils de reporting automatique.

Malgré l'absence d'automatisation complète, des efforts ont été déployés pour simplifier le processus d'évaluation, notamment en utilisant le logiciel Visual Studio Code et l'extension Jupyter Notebook pour structurer le code, documenter les étapes, et visualiser les résultats de manière interactive.

IV.7 Perspectives d'Avenir

L'utilisation des LLM pour le contrôle interactif et dynamique des circuits électriques par langage naturel ouvre des perspectives d'avenir prometteuses. Les résultats encourageants obtenus lors de cette expérimentation suggèrent de multiples axes de recherche et développement pour améliorer les performances, la fiabilité, et l'adaptabilité du système.

IV.7.1 Modèles LLM Spécialisés pour le Contrôle de Circuits Électriques

L'une des perspectives les plus prometteuses est le développement de modèles LLM spécifiquement entraînés et optimisés pour le contrôle des circuits électriques. Contrairement aux modèles LLM à usage général utilisés dans cette expérimentation, des modèles spécialisés pourraient être entraînés sur des corpus de données comprenant des schémas de circuits électriques, des codes Arduino, des spécifications de composants, et des exemples d'instructions en langage naturel relatives au contrôle des circuits.

Ce réglage fin sur des données spécifiques au domaine du contrôle des circuits électriques permettrait d'améliorer la précision du code Python généré, de réduire les erreurs d'interprétation, et d'augmenter la robustesse du système. De plus, l'apprentissage sur des exemples de code Arduino bien structurés permettrait aux LLM de générer un code plus efficace et optimisé pour les ressources limitées de la carte Arduino Uno.

IV.7.2 Intégration de la Multimodalité

L'intégration de la multimodalité dans le système de contrôle constitue une avancée significative. L'utilisation de modèles LLM multimodaux, capables de traiter des entrées textuelles, audio, images et vidéo, tels que GPT-4o avec sa version conversationnelle, ouvrirait de nouvelles possibilités.

Par exemple, l'utilisateur pourrait interagir avec le système de contrôle par la voix, en formulant des instructions verbales. Le modèle LLM multimodal transcrirait la parole en texte, l'interpréterait, et générerait le code Python correspondant. De plus, l'intégration d'une caméra permettrait au LLM d'analyser visuellement l'état des circuits électriques et de réagir en conséquence.

Cette approche multimodale serait particulièrement bénéfique dans des environnements industriels, où une surveillance auditive et visuelle des systèmes est cruciale pour détecter les anomalies et prévenir les incidents.

IV.7.3 Contrôle de Bas Niveau par les LLM

Actuellement, le système de contrôle utilise les LLM pour générer du code Python qui pilote l'Arduino Uno, qui à son tour contrôle les circuits électriques. Une perspective d'avenir est de permettre aux LLM de contrôler directement les circuits électriques au niveau des impulsions électriques, sans passer par l'intermédiaire de l'Arduino.

Ce contrôle de bas niveau par les LLM offrirait une plus grande flexibilité et précision. Les LLM pourraient ajuster les impulsions électriques en temps réel, en fonction des entrées sensorielles et des objectifs de contrôle, permettant des ajustements dynamiques et des optimisations impossibles à réaliser avec un contrôle de niveau supérieur.

IV.7.4 Amélioration du Raisonnement des Modèles LLM

Pour augmenter la fiabilité et la robustesse du système de contrôle, il est essentiel d'améliorer les capacités de raisonnement des LLM. Les erreurs d'interprétation et la génération de code incorrect découlent souvent d'une compréhension superficielle des instructions et d'un manque de capacité à raisonner sur les relations causales entre les actions et leurs effets sur les circuits électriques.

Des efforts de recherche sont en cours pour développer des LLM dotés de capacités de raisonnement plus poussées, capables de modéliser le fonctionnement des circuits électriques, de simuler les conséquences des actions, et d'anticiper les erreurs potentielles. L'intégration de telles capacités dans le système de contrôle permettrait de diminuer les risques d'erreurs et de garantir un contrôle plus fiable et prévisible.

IV.7.5 Apprentissage Continu et Collaboration

L'apprentissage continu est une caractéristique clé des LLM qui permettra au système de contrôle de s'améliorer continuellement. En collectant les données d'utilisation, les retours d'expérience, et les exemples de code corrects et incorrects, les LLM pourront être affinés et adaptés aux besoins spécifiques du contrôle des circuits électriques.

De plus, les LLM peuvent être utilisés pour collaborer avec les ingénieurs et les chercheurs en automatisation. En analysant les données des systèmes électriques, les LLM pourraient identifier des tendances, détecter des anomalies, et proposer des améliorations en termes de conception et de contrôle.

IV.8 Conclusion

Ce chapitre a permis d'analyser les performances du système de contrôle interactif et dynamique de circuits électriques par langage naturel, implémenté à l'aide de différents modèles de langage (LLM). L'évaluation des hypothèses de conception a confirmé le potentiel des LLM à interpréter des instructions en langage naturel et à générer du code Python fonctionnel pour piloter un circuit électrique via une carte Arduino Uno. L'analyse quantitative a mis en évidence les taux de succès élevés des modèles GPT-4o et GPT-4o mini, ainsi que la bonne précision temporelle de GPT-4o. L'analyse qualitative a révélé des défis persistants dans la gestion des nuances temporelles, des boucles infinies, et de la synchronisation d'actions, soulignant l'importance du prompt engineering et de la précision du langage naturel. Malgré les limitations intrinsèques à l'approche, liées à la complexité du langage naturel et à la fiabilité des LLM, des solutions techniques ont été mises en œuvre pour surmonter les difficultés rencontrées. Le développement d'une interface Python personnalisée et l'adoption de stratégies de simplification du code ont permis d'améliorer l'interopérabilité et de garantir un fonctionnement stable du système. Ce chapitre a démontré la faisabilité du concept de contrôle de circuits électriques par langage naturel, ouvrant la voie à des recherches futures axées sur le développement des LLM spécialisés, l'intégration de la multimodalité, l'amélioration du raisonnement des modèles, et l'apprentissage continu. L'application de cette technologie prometteuse dans des environnements industriels et de recherche laisse entrevoir des avancées significatives dans l'interaction homme-machine et l'automatisation des systèmes électriques.

CONCLUSION GENERALE

Ce mémoire, a exploré l'utilisation des grands modèles de langage (LLM) pour révolutionner le contrôle des circuits électriques en utilisant le langage naturel comme interface. Le travail a débuté par une exploration historique du contrôle des circuits électriques et des modèles d'apprentissage automatique, soulignant la convergence de ces deux domaines et l'émergence d'approches novatrices pour un contrôle intelligent. L'avènement des LLM, dotés de capacités avancées de compréhension et de génération de langage naturel, offre un potentiel disruptif pour l'interaction homme-machine, ouvrant la voie à une nouvelle ère de contrôle intuitif et flexible des systèmes électriques. Un état de l'art a ensuite permis de décrypter les fondements théoriques et techniques des LLM, mettant en évidence leur architecture basée sur les Transformers, leur apprentissage sur des corpus massifs et leurs performances notables en traitement du langage et en génération de code. Cette analyse a validé la pertinence des LLM pour le contrôle des circuits électriques par le langage naturel. Ce mémoire a abouti à la conception et l'implémentation d'un système de contrôle interactif et dynamique. Ce système, reposant sur le kit Arduino Uno pour la simulation de circuits et la bibliothèque PySerial pour la communication série. Des instructions en langage naturel ont été utilisées pour contrôler des circuits électriques des composants connectés à l'Arduino. L'évaluation systématique a confirmé la faisabilité du contrôle interactif de circuits électriques par le langage naturel grâce aux LLM. Les résultats ont mis en avant des taux de succès élevés, atteignant 100% pour GPT-4o mini, et une précision temporelle satisfaisante, particulièrement avec GPT-4o. Ces observations confirment les hypothèses de compréhension du langage naturel et, partiellement, de fiabilité des LLM. La compatibilité technique a été validée par l'implémentation réussie du système, et la modularité du code a permis de s'adapter à différentes configurations de circuits, attestant de l'adaptabilité de l'approche. Cependant, des limitations persistent dans la gestion des nuances temporelles complexes, l'interprétation des boucles infinies pour certains modèles et la synchronisation précise des actions pour les petits modèles, soulignant la complexité de la traduction du langage naturel en un code exempt d'erreurs. La fiabilité des LLM n'est donc pas totalement confirmée, nécessitant des efforts pour accroître leur robustesse et leur capacité à corriger les erreurs d'interprétation en temps réel. Des contraintes de ressources et de temps ont limité la portée de l'évaluation, restreignant la complexité des instructions et le nombre de modèles testés. Ce travail de mémoire ouvre la voie à de multiples axes de recherche. Le développement de LLM spécialisés pour le contrôle électrique, l'intégration de la multimodalité (textuelle, audio, visuelle), la possibilité d'un contrôle direct au niveau des impulsions électriques, et l'amélioration du raisonnement des LLM sont des pistes prometteuses pour des systèmes plus performants et fiables. La collaboration entre LLM et experts humains, par le biais de l'apprentissage continu, est essentielle pour optimiser le contrôle et s'adapter aux besoins spécifiques des différents domaines d'application. Ce mémoire, en démontrant la faisabilité du contrôle interactif des circuits par langage naturel, ouvre un champ de recherche fertile et ouvre la voie à une automatisation des systèmes électriques plus intuitive et conviviales.

BIBLIOGRAPHIE

- AI, M. (2024a, mai 29). *Codestral : Hello, World!* <https://mistral.ai/fr/news/codestral/>
- AI, M. (2024b, juillet 18). *Mistral NeMo*. <https://mistral.ai/fr/news/mistral-nemo/>
- Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., & Sanghai, S. (2023). *GQA : Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints* (No. arXiv:2305.13245). arXiv. <http://arxiv.org/abs/2305.13245>
- A.M Turing. (1950). *I.—COMPUTING MACHINERY AND INTELLIGENCE*. <http://archive.org/details/MIND--COMPUTING-MACHINERY-AND-INTELLIGENCE>
- Aspray, W. F. (1985). The Scientific Conceptualization of Information : A Survey. *IEEE Annals of the History of Computing*, 7(2), 117-140. <https://doi.org/10.1109/MAHC.1985.10018>
- Aspray, W. (avec Internet Archive). (1990). *John von Neumann and the origins of modern computing*. Cambridge, Mass. : MIT Press. <http://archive.org/details/johnvonneumannor0000aspr>
- Asservissement (automatique). (2024). In *Wikipédia*. [https://fr.wikipedia.org/w/index.php?title=Asservissement_\(automatique\)&oldid=216448625](https://fr.wikipedia.org/w/index.php?title=Asservissement_(automatique)&oldid=216448625)
- Atzori, L., Iera, A., & Morabito, G. (2010). The Internet of Things : A survey. *Computer Networks*, 54(15), 2787-2805. <https://doi.org/10.1016/j.comnet.2010.05.010>
- BALEMBA, S. (2024). *BKS00/CONTROLE-DYNAMIQUE-DES-CIRCUITS-ELECTRIQUES-GRACE-AUX-LLM-LARGE-LANGUAGE-MODELS-*. <https://github.com/BKS00/CONTROLE-DYNAMIQUE-DES-CIRCUITS-ELECTRIQUES-GRACE-AUX-LLM-LARGE-LANGUAGE-MODELS-/tree/main>
- Bennett, S. (1986). *A History of Control Engineering, 1800-1930*. IET.

Bipolar junction transistor. (2024). In *Wikipedia*.
https://en.wikipedia.org/w/index.php?title=Bipolar_junction_transistor&oldid=12407854

56

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., ... Amodei, D. (2020). *Language Models are Few-Shot Learners* (No. arXiv:2005.14165). arXiv.
<http://arxiv.org/abs/2005.14165>

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. de O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., ... Zaremba, W. (2021). *Evaluating Large Language Models Trained on Code* (No. arXiv:2107.03374). arXiv. <http://arxiv.org/abs/2107.03374>

Chen, W. (2023). *LLM-enabled Incremental Learning Framework for Hand Exoskeleton Control*.
<https://doi.org/10.36227/techrxiv.23939520.v1>

Codestral: Hello, World! | Mistral AI | Frontier AI in your hands. (2024, mai 29).
<https://mistral.ai/fr/news/codestral/>

Craig, J. J. (2005). *Introduction to Robotics : Mechanics and Control*. Pearson/Prentice Hall.

Crevier, D. (1993). *Ai : The Tumultuous History Of The Search For Artificial Intelligence*. Basic Books.

Cui, H., Du, Y., Yang, Q., Shao, Y., & Liew, S. C. (2024). *LLMind : Orchestrating AI and IoT with LLM for Complex Task Execution* (No. arXiv:2312.09007). arXiv.
<http://arxiv.org/abs/2312.09007>

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of Deep*

- Bidirectional Transformers for Language Understanding* (No. arXiv:1810.04805). arXiv.
<http://arxiv.org/abs/1810.04805>
- Dorf, R. C., & Bishop, R. H. (2008). *Modern Control Systems*. Prentice Hall.
- Fakih, M., Dharmaji, R., Moghaddas, Y., Araya, G. Q., Ogundare, O., & Faruque, M. A. A. (2024). LLM4PLC : Harnessing Large Language Models for Verifiable Programming of PLCs in Industrial Control Systems. *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice*, 192-203.
<https://doi.org/10.1145/3639477.3639743>
- File:Compact and electric valve actuator.jpg*—Wikipedia. (2017, avril 14).
https://commons.wikimedia.org/wiki/File:Compact_and_electric_valve_actuator.jpg
- Franklin, G. F. (avec Internet Archive). (2010). *Feedback control of dynamic systems*. Upper Saddle River, N.J. ; Harlow : Pearson.
http://archive.org/details/feedbackcontrolo0000fran_w1i0
- Gemma Team, Riviere, M., Pathak, S., Sessa, P. G., Hardin, C., Bhupatiraju, S., Hussenot, L., Mesnard, T., Shahriari, B., Ramé, A., Ferret, J., Liu, P., Tafti, P., Friesen, A., Casbon, M., Ramos, S., Kumar, R., Lan, C. L., Jerome, S., ... Andreev, A. (2024). *Gemma 2 : Improving Open Language Models at a Practical Size* (No. arXiv:2408.00118). arXiv.
<http://arxiv.org/abs/2408.00118>
- GitHub—OpenInterpreter/open-interpreter : A natural language interface for computers*. (s. d.). Consulté 22 août 2024, à l'adresse <https://github.com/OpenInterpreter/open-interpreter>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- He, R., Ravula, A., Kanagal, B., & Ainslie, J. (2021). *RealFormer : Transformer Likes Residual Attention* (No. arXiv:2012.11747). arXiv. <http://arxiv.org/abs/2012.11747>

Hello GPT-4o. (2024). <https://openai.com/index/hello-gpt-4o/>

Hello GPT-4o | OpenAI. (2024). <https://openai.com/index/hello-gpt-4o/>

Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., & Steinhardt, J. (2021).

Measuring Massive Multitask Language Understanding (No. arXiv:2009.03300). arXiv.

<http://arxiv.org/abs/2009.03300>

Howard, J., & Ruder, S. (2018). *Universal Language Model Fine-tuning for Text Classification*

(No. arXiv:1801.06146). arXiv. <http://arxiv.org/abs/1801.06146>

Hughes, T. P. (1993). *Networks of Power : Electrification in Western Society, 1880-1930*. JHU Press.

Intel 4004. (2024). In *Wikipédia*.

https://fr.wikipedia.org/w/index.php?title=Intel_4004&oldid=218902033

Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., Casas, D. de las, Bressand,

F., Lengyel, G., Lample, G., Saulnier, L., Lavaud, L. R., Lachaux, M.-A., Stock, P., Scao,

T. L., Lavril, T., Wang, T., Lacroix, T., & Sayed, W. E. (2023). *Mistral 7B* (No.

arXiv:2310.06825). arXiv. <http://arxiv.org/abs/2310.06825>

Jiang, A. Q., Sablayrolles, A., Roux, A., Mensch, A., Savary, B., Bamford, C., Chaplot, D. S.,

Casas, D. de las, Hanna, E. B., Bressand, F., Lengyel, G., Bour, G., Lample, G., Lavaud,

L. R., Saulnier, L., Lachaux, M.-A., Stock, P., Subramanian, S., Yang, S., ... Sayed, W. E.

(2024). *Mixtral of Experts* (No. arXiv:2401.04088). arXiv. <http://arxiv.org/abs/2401.04088>

Jobin, A., Ienca, M., & Vayena, E. (s. d.). *Artificial Intelligence : The global landscape of ethics guidelines*.

Kagermann, H., Helbig, J., & Wahlster, W. (2013). *Recommendations for Implementing the*

Strategic Initiative INDUSTRIE 4.0 : Securing the Future of German Manufacturing

- Industry ; Final Report of the Industrie 4.0 Working Group*. Forschungsunion.
- Kalman1960.pdf. (s. d.). Consulté 25 septembre 2024, à l'adresse <https://www.unitedthc.com/DSP/Kalman1960.pdf>
- Lascar, O. (2022). *Enquête sur Elon Musk, l'homme qui défie la science*. Alisio.
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *nature*, 521(7553), 436-444.
- Lee, E. A. (2008). Cyber Physical Systems : Design Challenges. *2008 11th IEEE International Symposium on Object and Component-Oriented Real-Time Distributed Computing (ISORC)*, 363-369. <https://doi.org/10.1109/ISORC.2008.25>
- Margolis, M. (s. d.). *Arduino Cookbook*.
- Marr, B. (2019). *Artificial Intelligence in Practice : How 50 Successful Companies Used AI and Machine Learning to Solve Problems*. John Wiley & Sons.
- Minaee, S., Mikolov, T., Nikzad, N., Chenaghlu, M., Socher, R., Amatriain, X., & Gao, J. (2024, février 9). *Large Language Models : A Survey*. arXiv.org. <https://arxiv.org/abs/2402.06196v2>
- Mistral AI API | Mistral AI Large Language Models*. (2024). <https://docs.mistral.ai/api/>
- Naveed, H., Khan, A. U., Qiu, S., Saqib, M., Anwar, S., Usman, M., Akhtar, N., Barnes, N., & Mian, A. (2024). *A Comprehensive Overview of Large Language Models* (No. arXiv:2307.06435). arXiv. <http://arxiv.org/abs/2307.06435>
- Ogata, K. (2010). *Modern Control Engineering*. Prentice Hall.
- OpenAI, Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., Avila, R., Babuschkin, I., Balaji, S., Balcom, V., Baltescu, P., Bao, H., Bavarian, M., Belgum, J., ... Zoph, B. (2024). *GPT-4 Technical Report* (No. arXiv:2303.08774). arXiv. <http://arxiv.org/abs/2303.08774>

Pearl, J. (1988). *Probabilistic Reasoning in Intelligent Systems : Networks of Plausible Inference*. Morgan Kaufmann Publishers.

Perceptron. (2024). In *Wikipédia*.
<https://fr.wikipedia.org/w/index.php?title=Perceptron&oldid=216487433>

Perceptron multicouche. (2023). In *Wikipédia*.
https://fr.wikipedia.org/w/index.php?title=Perceptron_multicouche&oldid=205213828

Programming Arduino : Getting Started with Sketches (First edition). (2012). McGraw-Hill Education. <https://www.accessengineeringlibrary.com/content/book/9780071784221>

Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning*, 1(1), 81-106.
<https://doi.org/10.1007/BF00116251>

Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (s. d.). *Improving Language Understanding by Generative Pre-Training*.

Régulateur à boules. (2024). In *Wikipédia*.
https://fr.wikipedia.org/w/index.php?title=R%C3%A9gulateur_%C3%A0_boules&oldid=214156361

Régulateur PID. (2024). In *Wikipédia*.
https://fr.wikipedia.org/w/index.php?title=R%C3%A9gulateur_PID&oldid=216330043

Relais électromécanique. (2024). In *Wikipédia*.
https://fr.wikipedia.org/w/index.php?title=Relais_%C3%A9lectrom%C3%A9canique&oldid=218892685

Riordan, M., & Hoddeson, L. (1997). *Crystal Fire : The Birth of the Information Age*. Norton.

Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386-408.

<https://doi.org/10.1037/h0042519>

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533-536. <https://doi.org/10.1038/323533a0>

Schmidhuber, J. (2015). Deep Learning in Neural Networks : An Overview. *Neural Networks*, 61, 85-117. <https://doi.org/10.1016/j.neunet.2014.09.003>

Sontag, E. D. (2013). *Mathematical Control Theory : Deterministic Finite Dimensional Systems*. Springer Science & Business Media.

Strubell, E., Ganesh, A., & McCallum, A. (2019). *Energy and Policy Considerations for Deep Learning in NLP* (No. arXiv:1906.02243). arXiv. <http://arxiv.org/abs/1906.02243>

Test de Turing. (2024). In *Wikipédia*. https://fr.wikipedia.org/w/index.php?title=Test_de_Turing&oldid=218446803

Transformeur. (2024). In *Wikipédia*. <https://fr.wikipedia.org/w/index.php?title=Transformeur&oldid=218399794>

Transformeur génératif pré-entraîné. (2024). In *Wikipédia*. https://fr.wikipedia.org/w/index.php?title=Transformeur_g%C3%A9n%C3%A9ratif_pr%C3%A9-entra%C3%AEn%C3%A9&oldid=216490180

Triode (électronique). (2023). In *Wikipédia*. [https://fr.wikipedia.org/w/index.php?title=Triode_\(%C3%A9lectronique\)&oldid=210741125](https://fr.wikipedia.org/w/index.php?title=Triode_(%C3%A9lectronique)&oldid=210741125)

UNO R3 | *Arduino Documentation*. (s. d.). Consulté 25 septembre 2024, à l'adresse <https://docs.arduino.cc/hardware/uno-rev3/>

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2023). *Attention Is All You Need* (No. arXiv:1706.03762). arXiv.

<http://arxiv.org/abs/1706.03762>

Welcome to pySerial's documentation—pySerial 3.4 documentation. (s. d.). Consulté 25 septembre 2024, à l'adresse <https://pyserial.readthedocs.io/en/latest/>

Xia, Y., Shenoy, M., Jazdi, N., & Weyrich, M. (2023). Towards autonomous system : Flexible modular production system enhanced with large language model agents. *2023 IEEE 28th International Conference on Emerging Technologies and Factory Automation (ETFA)*, 1-8. <https://doi.org/10.1109/ETFA54631.2023.10275362>

ANNEXES

Annexe A: Liste du jeu d'instructions utilisé pour évaluer les modèles sur le contrôle dynamique des circuits électriques

Instructions de Contrôle Dynamique
1. Augmenter progressivement la luminosité de la LED 1 de 0% à 100% en 5 secondes. (Device: 1, Action: 0, Param: [0, 255], Time: 5s)
2. Faire varier l'angle du servo moteur de 0 à 180 degrés et revenir à 90 degrés en 10 secondes. (Device: 4, Action: 1, Param: [0, 180, 90], Time: 10s)
3. Alternier l'allumage des LED 1 et 2 avec des intervalles de 2 secondes pendant 10 secondes. (Device: [1, 2], Action: 0, Param: [255, 0], Interval: 2s, Time: 10s)
4. Faire clignoter la LED 3 cinq fois avec un intervalle de 1 seconde entre chaque allumage. (Device: 3, Action: 0, Param: 255, Interval: 1s, Repeats: 5)
5. Diminuer progressivement la luminosité de la LED 2 de 100% à 0% en 3 secondes. (Device: 2, Action: 0, Param: [255, 0], Time: 3s)
6. Synchroniser le mouvement du servo moteur et l'allumage de la LED 1, le servo allant de 0 à 180 degrés tandis que la LED passe de 0% à 100% de façon synchrone inverse au servo par pas de 50 (Device: [4, 1], Action: [1, 0], Param: [[0, 180], [0, 255]], Time: 10s)
7. Faire varier la luminosité des LED 1 et 2 en alternance toutes les 0,5 secondes pendant 10 secondes. (Device: [1, 2], Action: 0, Param: [255, 0], Interval: 0.5s, Time: 10s)
8. Faire passer la LED 3 de 0% à 100% de luminosité en 2 secondes, puis revenir à 0% en 2 secondes. Répéter 3 fois. (Device: 3, Action: 0, Param: [0, 255], Time: 4s, Repeats: 3)
9. Augmenter progressivement la luminosité de la LED 2 de 0% à 50% en 1 seconde, puis de 50% à 100% en 2 secondes. (Device: 2, Action: 0, Param: [0, 128, 255], Time: 3s)
10. Faire varier l'angle du servo moteur de 90 à 0 degrés, puis de 0 à 90 degrés en 4 secondes. (Device: 4, Action: 1, Param: [90, 0, 90], Time: 4s)
11. Éteindre la LED 1 pendant 2 secondes, puis allumer la LED 3 à 100% pendant 2 secondes. Répéter 4 fois. (Device: [1, 3], Action: [0, 0], Param: [0, 255], Interval: 2s, Repeats: 4)
12. Alternier l'allumage des LED 1 et 2 toutes les 1,5 secondes pendant 15 secondes, avec une LED allumée à 100% et l'autre éteinte. (Device: [1, 2], Action: 0, Param: [255, 0], Interval: 1.5s, Time: 15s)
13. Diminuer la luminosité de la LED 1 de 100% à 0% en 4 secondes, puis augmenter la luminosité de la LED 2 de 0% à 100% en 4 secondes. (Device: [1, 2], Action: 0, Param: [255, 0, 255], Time: 8s)
14. Faire varier l'angle du servo moteur de 0 à 90 degrés en 3 secondes, puis de 90 à 180 degrés en 3 secondes. (Device: 4, Action: 1, Param: [0, 90, 180], Time: 6s)
15. Synchroniser l'allumage de la LED 3 avec le mouvement du servo moteur allant de 0 à 180 degrés. La LED passe de 0% à 100% en synchronisation inverse avec le servo. (Device: [4, 3], Action: [1, 0], Param: [[0, 180], [0, 255]], Time: 10s)
16. Faire clignoter la LED 2 à 100% de luminosité toutes les 1,5 secondes pendant 12 secondes, puis éteindre la LED 2 pendant 3 secondes. (Device: 2, Action: 0, Param: 255, Interval: 1.5s, Time: 15s)

B

17. Augmenter la luminosité de la LED 3 de 0% à 100% en 6 secondes, puis diminuer à 0% en 4 secondes. Répéter 3 fois. (Device: 3, Action: 0, Param: [0, 255], Time: 10s, Repeats: 3)
18. Alternier l'angle du servo moteur entre 0 et 180 degrés toutes les 2 secondes pendant 20 secondes. (Device: 4, Action: 1, Param: [0, 180], Interval: 2s, Time: 20s)
19. Faire clignoter la LED 1 à 50% de luminosité et la LED 2 à 75% toutes les 2 secondes pendant 8 secondes. (Device: [1, 2], Action: 0, Param: [128, 192], Interval: 2s, Time: 8s)
20. Augmenter la luminosité de la LED 2 de 0% à 100% en 5 secondes, puis faire clignoter la LED 2 à 100% de luminosité toutes les 1,5 secondes pendant 10 secondes. (Device: 2, Action: [0, 0], Param: [255, 0], Interval: 1.5s, Time: 15s)
21. Faire varier l'angle du servo moteur de 90 à 0 degrés, puis de 0 à 90 degrés en 8 secondes. (Device: 4, Action: 1, Param: [90, 0, 90], Time: 8s)
22. Synchroniser l'allumage de la LED 3 avec le mouvement du servo moteur de 0 à 180 degrés toutes les 4 secondes. La LED passe de 0% à 100% de luminosité. (Device: [4, 3], Action: [1, 0], Param: [[0, 180], [0, 255]], Time: 4s)
23. Diminuer la luminosité de la LED 1 de 100% à 0% en 5 secondes, puis augmenter la luminosité de la LED 2 de 0% à 100% en 5 secondes. (Device: [1, 2], Action: 0, Param: [255, 0, 255], Time: 10s)
24. Synchroniser le mouvement du servo moteur entre 30 et 150 degrés toutes les 5 secondes avec l'allumage de la LED 1 à 100%. (Device: [4, 1], Action: [1, 0], Param: [[30, 150], [255]], Interval: 5s, Time: 10s)
25. Synchroniser l'allumage de la LED 2 avec le mouvement du servo moteur entre 0 et 180 degrés toutes les 6 secondes. La LED passe de 0% à 100% de luminosité. (Device: [2, 4], Action: [0, 1], Param: [[0, 180], [0, 255]], Interval: 6s, Time: 12s)

Annexe B: Prompt système qui guide les modèles LLM

Voici un prompt détaillé structuré pour guider un LLM dans la génération de code Python destiné à contrôler un Arduino en utilisant des matrices de commandes.

Role et Objectif :

Tu es un assistant IA expert en programmation et en électromécanique, capable de générer du code Python qui communique avec un Arduino via le port série. Ton objectif est de générer du code Python qui contrôle dynamiquement des composants électroniques connectés à un Arduino, tels que des LEDs et un servomoteur, en se basant sur des instructions données en langage naturel. Les commandes Python que tu génères doivent envoyer des matrices de commandes structurées à un Arduino, qui exécutera ces commandes selon un code Arduino préexistant.

Contexte :

Tu travailles dans le cadre d'un projet de recherche visant à démontrer une preuve de concept pour le contrôle interactif et dynamique de circuits électriques programmables à l'aide de modèles de langage naturel (LLM). Le projet utilise un kit Arduino UNO pour contrôler divers composants, notamment des LEDs, un servomoteur, des capteurs ultrasoniques et de température, ainsi que des moteurs pas à pas. Le code Arduino est statique et reçoit des commandes structurées sous forme de matrices via le port série. Ton rôle est de générer le code Python qui transforme les instructions utilisateur en matrices de commandes compréhensibles par l'Arduino.

Voici le code Arduino qui sert de base :

```

```cpp
#include <Servo.h>

Servo myservo;
String command = "";

void setup() {
 Serial.begin(9600);
 Serial.println("Arduino is ready");

 pinMode(5, OUTPUT); // LED 1
 pinMode(3, OUTPUT); // LED 2
 pinMode(11, OUTPUT); // LED 3
 myservo.attach(6); // Servo moteur connecté à la broche 6
}

void executeCommand(int device, int action, int* params, int paramCount) {
 switch (device) {
 case 1:
 if (action == 0 && paramCount > 0) analogWrite(5, params[0]); // LED 1
 break;
 case 2:
 if (action == 0 && paramCount > 0) analogWrite(3, params[0]); // LED 2
 break;
 case 3:
 if (action == 0 && paramCount > 0) analogWrite(11, params[0]); // LED 3
 break;
 case 4:
 if (action == 1 && paramCount > 0) myservo.write(params[0]); // Servo
moteur
 break;
 default:
 Serial.println("Invalid device or action");
 }
}

```



```

 break;
 }
}

void loop() {
 while (Serial.available()) {
 char c = Serial.read();
 if (c == '\n') {
 Serial.print("Received command: ");
 Serial.println(command);

 int parts[5] = {0};
 int partIndex = 0;
 int commaIndex = 0;
 while ((commaIndex = command.indexOf(',')) > 0 && partIndex < 5) {
 parts[partIndex++] = command.substring(0, commaIndex).toInt();
 command = command.substring(commaIndex + 1);
 }
 parts[partIndex] = command.toInt();

 if (partIndex >= 1) {
 executeCommand(parts[0], parts[1], parts + 2, partIndex - 1);
 } else {
 Serial.println("Command format error");
 }

 command = "";
 } else {
 command += c;
 }
 }
}
...

```

Ce code Arduino permet de contrôler trois LEDs et un servomoteur en utilisant des commandes formatées. Tu dois générer un code Python qui envoie des commandes appropriées via le port série en fonction des instructions utilisateur.

**### \*\*Exemple :\*\***

**\*\*Instruction Utilisateur :\*\***

*"Fais clignoter la LED 1 cinq fois avec une intensité croissante de 0 à 255, puis fais pivoter le servomoteur de 0 à 180 degrés pendant que la LED 2 reste allumée à moitié de son intensité maximale."*

**\*\*Code Python que tu dois générer :\*\***

```

```python
import serial
import time

# Connexion au port série
ser = serial.Serial('COM8', 9600) # Remplacez 'COM3' par le port série
approprié

def envoyer_commande(device, action, params):
    commande = f"{device},{action}," + ",".join(map(str, params)) + "\n"
    ser.write(commande.encode())
    time.sleep(0.1)

try:
    # Faire clignoter la LED 1 cinq fois avec intensité croissante
    for i in range(5):
        for intensity in range(0, 256, 51):
            envoyer_commande(1, 0, [intensity])
            time.sleep(0.5)
        envoyer_commande(1, 0, [0])

    # Allumer la LED 2 à moitié de son intensité maximale
    envoyer_commande(2, 0, [128])

    # Faire pivoter le servomoteur de 0 à 180 degrés
    for angle in range(0, 181, 30):
        envoyer_commande(4, 1, [angle])
        time.sleep(1)

    # Éteindre la LED 2 après le mouvement du servomoteur
    envoyer_commande(2, 0, [0])

finally:
    ser.close()
```

```

**### \*\*Instruction :\*\***

Utilise cette structure pour générer du code Python capable de contrôler les composants connectés à l'Arduino, en transformant les instructions en langage naturel en matrices de commandes compréhensibles par le code Arduino fourni. Assure-toi que le code Python suit une logique cohérente, respecte les capacités du matériel et gère les tâches de manière dynamique et interactive.

INSTRUCTION DE L'UTILISATEUR :

Annexe C: Performances des modèles sur l'écart temporel par instruction

| N° de l' instruction | temps exiger | ECART TEMPOREL DES MODÈLES (en secondes) |             |               |              |           |              |               |            |
|----------------------|--------------|------------------------------------------|-------------|---------------|--------------|-----------|--------------|---------------|------------|
|                      |              | GPT-4o                                   | GPT-4o mini | Mistral large | Mistral Nemo | Codestral | Mistral 8X7B | Mistral 7B v2 | Gemma 2 9B |
| 1                    | 5            | 6,1                                      | 13,6        | 0,2           | 4,8          | 29,7      | 24,8         | 3,9           | 50,4       |
| 2                    | 10           | 0,3                                      | 11,7        | 8             | 20,3         | 11,4      | 3,8          | 40,8          | 4,8        |
| 3                    | 10           | 3,4                                      | 3,4         | 3,3           | 32,3         | 3,3       | 33,5         | -             | 12,2       |
| 4                    | 5            | 6,2                                      | 6,2         | 6,9           | 6,1          | 6,1       | 6,1          | -             | 1,1        |
| 5                    | 3            | 0,5                                      | 11,9        | 0,1           | 28,1         | 0,1       | 1,3          | 0,7           | 8,3        |
| 6                    | -            | -                                        | -           | -             | -            | -         | -            | -             | -          |
| 7                    | 10           | 19,2                                     | 0,1         | 19            | 4,3          | 4,5       | 4,7          | -             | -          |
| 8                    | 12           | 7,1                                      | 31,7        | -             | 1,3          | 8,1       | 13,8         | 13,2          | -          |
| 9                    | 3            | -                                        | 3,7         | 29,9          | 28,9         | 6,5       | 5            | 0,9           | 1,1        |
| 10                   | 4            | 2,3                                      | 2,3         | 26,8          | 8,3          | 5,2       | 9,6          | 30            | 7,2        |
| 11                   | 16           | 0,9                                      | 0,9         | 1,4           | 5,1          | 0,9       | 0,5          | 1             | 0,9        |
| 12                   | 15           | 19,6                                     | 19,8        | 19,4          | 36,1         | 2,5       | -            | -             | 35,2       |
| 13                   | 8            | 0,9                                      | 23,3        | 171,6         | 6,1          | 3         | 0,7          | 2             | 5,8        |
| 14                   | 6            | 0,2                                      | 7,6         | 2,6           | 18,2         | 7,3       | -            | 39,7          | 2,3        |
| 15                   | -            | -                                        | -           | -             | -            | -         | -            | -             | -          |
| 16                   | 15           | 1                                        | 1           | 1             | 26,8         | 1         | 26,8         | -             | -          |
| 17                   | 30           | 7,2                                      | 49,6        | -             | 22,3         | -         | 3,9          | -             | -          |
| 18                   | 20           | 1,2                                      | 1,2         | 1,2           | 64,7         | 0         | 40,6         | -             | 64,7       |
| 19                   | 8            | 0,9                                      | 0,9         | 9,8           | -            | 1,8       | -            | -             | -          |
| 20                   | 15           | 2,5                                      | 0,6         | -             | 33,2         |           | 37,8         | -             | 60,3       |
| 21                   | 8            | 3,2                                      | 11,8        | 33,5          | 16,3         | 21,1      | 29           | 20            | 12         |
| 22                   | -            | -                                        | -           | -             | -            | -         | -            | -             | -          |
| 23                   | 10           | 3,5                                      | 12,8        | 4,4           | -            | 2,7       | 42           | 8,7           | 12,3       |
| 24                   | -            | -                                        | -           | -             | -            | -         | -            | -             | -          |
| 25                   | -            | -                                        | -           | -             | -            | -         | -            | -             | -          |

Annexe D : Notebook des tests de performances des Modèles LLM sur les Instructions de Contrôle Dynamique des Circuits Électriques (BALEMBA, 2024)